

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



- Machine Learning (CO3107)

News Category Classification

Giảng viên hướng dẫn: Trương Vĩnh Lân

Students: 2311041 - Huỳnh Huy Hoàng
2311359 - Trần Nguyễn Đức Hưng
2312709 - Phan Gia Phúc
2211572 - Nguyễn Gia Khiêm
2313674 - Nguyễn Mạnh Trung

TP. HỒ CHÍ MINH, 05/2026



Danh sách thành viên & Phân công công việc

STT	Họ và tên	MSSV	Nhiệm vụ
1	Huỳnh Huy Hoàng	2311041	báo cáo, pipeline học máy
2	Trần Nguyễn Đức Hưng	2311359	Báo cáo, eda & tiền xử lý, pipeline học máy và pipeli
3	Phan Gia Phúc	2312709	Báo cáo,pipeline học máy
4	Nguyễn Gia Khiêm	2211572	Báo cáo, pipeline học máy
4	Nguyễn Mạnh Trung	2313674	Báo cáo, pipeline học máy

Table 1: Member list & Task Assignment



Mục lục

1	Giới thiệu bộ dữ liệu News Category Dataset	6
2	Exploratory Data Analysis(EDA) - Phân tích dữ liệu khám phá	7
2.1	Phạm vi EDA	7
2.2	Tổng quan dữ liệu và chất lượng dữ liệu	7
2.3	Phân phối dữ liệu	8
2.4	Phân tích độ dài văn bản	11
2.5	Phân tích độ dài văn bản theo 15 category	13
2.6	Phân tích từ và cụm từ phổ biến và vocabulary	14
2.7	Các từ phổ biến theo category	16
2.8	Phân tích từ khóa đặc trưng bằng TF-IDF	20
3	Tiền xử lý dữ liệu	22
3.1	Lựa chọn thuộc tính	22
3.2	Xử lý giá trị thiếu	22
3.3	Kiểm tra và loại bỏ dữ liệu trùng lặp	22
3.4	Lựa chọn các nhãn phổ biến	23
3.5	Làm sạch văn bản	23
3.6	Loại bỏ stopwords và lemmatization	23
3.6.1	Chia tập dữ liệu	24
4	Biểu diễn văn bản	25
4.1	Bag-of-Words (BoW)	25
4.2	TF-IDF	25
4.3	Mô hình BERT (một ứng dụng trong Xử lý Ngôn ngữ Tự nhiên) . .	27
5	Huấn luyện và đánh giá mô hình phân loại	29
5.1	Thuật toán Random Forest	29
5.2	LightGBM	31
5.3	Gradient Boosting	32
5.4	Thuật toán CatBoost	33
5.5	Thuật toán Linear SVC	34
6	Pipeline học máy	36
6.1	TF-IDF và LinearSVC	36
6.1.1	Trích xuất đặc trưng	36
6.1.2	Huấn luyện mô hình	37
6.1.3	Đánh giá mô hình	37
6.1.4	Các thử nghiệm đã thực hiện	39
6.2	BERT/RoBERTa Embedding và LinearSVC	42

6.2.1	Tiền xử lí dữ liệu	42
6.2.2	Trích xuất đặc trưng	42
6.2.3	Huấn luyện mô hình	43
6.2.4	Đánh giá mô hình	44
6.2.5	Thử nghiệm	46
6.3	Sbert và Catboost	47
6.3.1	Biểu diễn văn bản bằng Sentence-BERT	47
6.3.2	Huấn luyện mô hình CatBoost	48
6.3.3	Đánh giá mô hình	49
6.3.4	Các thử nghiệm và đánh giá	49
6.4	TF-IDF và CatBoost	50
6.4.1	Trích xuất đặc trưng	50
6.4.2	Huấn luyện mô hình	51
6.4.3	Các thử nghiệm đã thực hiện	51
6.4.4	Đánh giá mô hình	52
6.5	TF-IDF và Gradient Boosting	54
6.5.1	Pipeline mô hình	54
6.5.2	Tìm kiếm siêu tham số	54
6.5.3	Bộ tham số tốt nhất	55
6.5.4	Kết quả trên tập kiểm thử	55
6.5.5	Phân tích theo nhãn	55
6.5.6	Phân tích từ ma trận nhầm lẫn	56
6.5.7	Nhận xét tổng quan	58
6.6	TF-IDF và thuật toán Random Forest	58
6.6.1	Biểu diễn văn bản bằng TF-IDF	58
6.6.2	Huấn luyện mô hình Random Forest	59
6.6.3	Điều chỉnh siêu tham số	59
6.6.4	Đánh giá mô hình	60
6.7	TF-IDF và thuật toán LightGBM	61
6.7.1	Trích xuất đặc trưng	61
6.7.2	Huấn luyện mô hình	61
6.7.3	Tối ưu siêu tham số	62
6.7.4	Đánh giá mô hình	63
6.8	BoW và LinearSVC	65
6.8.1	Trích xuất đặc trưng	65
6.8.2	Huấn luyện mô hình	65
6.8.3	Đánh giá mô hình	66
6.8.4	Các thử nghiệm đã thực hiện	67
6.9	BoW và Random Forest	68
6.9.1	Trích xuất đặc trưng	68

6.9.2	Huấn luyện mô hình	69
6.9.3	Đánh giá mô hình	70
6.9.4	Các thử nghiệm đã thực hiện	71
6.10	BoW và LightGBM	72
6.10.1	Trích xuất đặc trưng	73
6.10.2	Huấn luyện mô hình	73
6.10.3	Đánh giá mô hình	74
6.10.4	Các thử nghiệm đã thực hiện	75
7	Pipeline học sâu	76
7.1	Tiền xử lý dữ liệu	76
7.1.1	Lựa chọn thuộc tính	76
7.1.2	Xử lý giá trị thiếu	77
7.1.3	Kiểm tra dữ liệu trùng lặp	77
7.1.4	Lựa chọn các nhãn phổ biến	78
7.1.5	Làm sạch văn bản	78
7.1.6	Mã hóa nhãn	78
7.1.7	Chia tập dữ liệu	79
7.1.8	Tokenization và padding	79
7.1.9	Xử lý mất cân bằng lớp	80
7.2	Xây dựng mô hình học sâu	80
7.2.1	Đầu vào của mô hình	80
7.2.2	Lớp Embedding	80
7.2.3	Spatial Dropout	81
7.2.4	Lớp BiLSTM	81
7.2.5	Cơ chế Masked Attention	82
7.2.6	Dropout sau Attention	82
7.2.7	Các lớp Dense và Dropout	83
7.2.8	Lớp đầu ra	83
7.3	Tổng quan kiến trúc mô hình	83
7.3.1	Biên dịch mô hình	84
7.4	Quá trình huấn luyện mô hình	84
7.4.1	Các callback sử dụng trong huấn luyện	85
7.4.2	Diễn biến quá trình huấn luyện	85
7.4.3	Biểu đồ loss trong quá trình huấn luyện	86
7.4.4	Biểu đồ accuracy trong quá trình huấn luyện	87
7.5	Đánh giá mô hình	88



8	Phân tích và thảo luận kết quả thực nghiệm	90
8.1	Hiệu quả tổng thể của các pipeline	91
8.2	Tác động của phương pháp biểu diễn văn bản	92
8.3	Tác động của thuật toán phân loại	93
8.4	Phân tích theo nhóm nhãn	93
8.5	So sánh học máy truyền thống và học sâu	94
8.6	Mức độ phù hợp giữa đặc trưng và thuật toán	94
8.7	Kết luận	95
9	Mã nguồn	96

1 Giới thiệu bộ dữ liệu News Category Dataset

News Category Dataset là một bộ dữ liệu tin tức được công bố trên Kaggle bởi Rishabh Misra. Bộ dữ liệu bao gồm khoảng 210 nghìn tiêu đề bài báo được thu thập từ HuffPost trong giai đoạn từ năm 2012 đến năm 2022. Mỗi bản ghi tương ứng với một bài viết và được gán nhãn theo chuyên mục tin tức, phù hợp cho các bài toán xử lý ngôn ngữ tự nhiên như phân loại văn bản, phân tích chủ đề và khai phá xu hướng nội dung báo chí.

Mỗi mẫu dữ liệu gồm các thuộc tính chính sau:

- **category**: chuyên mục của bài báo;
- **headline**: tiêu đề bài báo;
- **authors**: tác giả của bài viết;
- **link**: đường dẫn đến bài báo gốc;
- **short_description**: mô tả ngắn hoặc phần tóm tắt nội dung;
- **date**: ngày xuất bản bài báo.

Bộ dữ liệu có tổng cộng 42 nhãn chuyên mục khác nhau, bao gồm các nhóm như *Politics*, *Wellness*, *Entertainment*, *Travel*, *Style & Beauty*, *Business*, v.v. Nhờ có nhãn phân loại sẵn và phần mô tả ngắn đi kèm, bộ dữ liệu này thường được sử dụng để huấn luyện và đánh giá các mô hình phân loại tin tức dựa trên tiêu đề và nội dung tóm tắt.

Trong báo cáo này, bộ dữ liệu được sử dụng nhằm xây dựng mô hình phân loại chuyên mục tin tức. Đầu vào của mô hình có thể là tiêu đề bài báo, mô tả ngắn hoặc sự kết hợp của cả hai trường văn bản; đầu ra là nhãn chuyên mục tương ứng của bài viết.

2 Exploratory Data Analysis(EDA) - Phân tích dữ liệu khám phá

2.1 Phạm vi EDA

Phần EDA tập trung vào ba trường chính được sử dụng trong bài toán phân loại chủ đề tin tức: `category`, `headline` và `short_description`. Trong đó, `category` là nhãn cần dự đoán, còn `headline` và `short_description` cung cấp nội dung văn bản đầu vào cho mô hình.

Biến văn bản tổng hợp được tạo bằng cách ghép tiêu đề và mô tả:

```
text = headline + "+" + short_description
```

Cách ghép này giúp kết hợp thông tin ngắn gọn từ tiêu đề với ngữ cảnh bổ sung từ phần mô tả, tạo ra đầu vào đầy đủ hơn cho quá trình phân tích và huấn luyện mô hình.

2.2 Tổng quan dữ liệu và chất lượng dữ liệu

Table 2: Tổng quan dữ liệu sau bước chuẩn hoá EDA

Chỉ số	Giá trị
Số dòng	209.527
Missing sau fillna trong 3 biến	0
Số dòng chứa text rỗng	5
Số category	42
Dòng trùng toàn bộ trường EDA	471
Text trùng	485

Dữ liệu có quy mô lớn với hơn 209 nghìn mẫu và 42 category, phù hợp để xây dựng mô hình phân loại văn bản. Sau bước xử lý thiếu dữ liệu, ba trường chính không còn giá trị missing; tuy nhiên vẫn tồn tại một số dòng có `text` rỗng và các mẫu bị trùng lặp.

Số lượng `text` trùng là 485, không quá lớn so với toàn bộ dữ liệu nhưng vẫn cần lưu ý vì có thể gây rò rỉ thông tin nếu các mẫu trùng xuất hiện đồng thời ở tập train và validation/test. Do đó, nên kiểm tra hoặc loại bỏ trùng lặp trước khi chia dữ liệu.

2.3 Phân phối dữ liệu

Table 3: Top 20 category theo số lượng mẫu

Category	Count	Tỷ lệ
POLITICS	35.602	16,99%
WELLNESS	17.945	8,56%
ENTERTAINMENT	17.362	8,29%
TRAVEL	9.900	4,72%
STYLE & BEAUTY	9.814	4,68%
PARENTING	8.791	4,20%
HEALTHY LIVING	6.694	3,19%
QUEER VOICES	6.347	3,03%
FOOD & DRINK	6.340	3,03%
BUSINESS	5.992	2,86%
COMEDY	5.400	2,58%
SPORTS	5.077	2,42%
BLACK VOICES	4.583	2,19%
HOME & LIVING	4.320	2,06%
PARENTS	3.955	1,89%
THE WORLDPOST	3.664	1,75%
WEDDINGS	3.653	1,74%
WOMEN	3.572	1,70%
CRIME	3.562	1,70%
IMPACT	3.484	1,66%

Table 4: Bottom 20 category theo số lượng mẫu

Category	Count	Tỷ lệ
MEDIA	2.944	1,41%
WEIRD NEWS	2.777	1,33%
GREEN	2.622	1,25%
WORLDPOST	2.579	1,23%
RELIGION	2.577	1,23%
STYLE	2.254	1,08%
SCIENCE	2.206	1,05%
TECH	2.104	1,00%
TASTE	2.096	1,00%
MONEY	1.756	0,84%
ARTS	1.509	0,72%
ENVIRONMENT	1.444	0,69%
FIFTY	1.401	0,67%
GOOD NEWS	1.398	0,67%
U.S. NEWS	1.377	0,66%
ARTS & CULTURE	1.339	0,64%
COLLEGE	1.144	0,55%
LATINO VOICES	1.130	0,54%
CULTURE & ARTS	1.074	0,51%
EDUCATION	1.014	0,48%

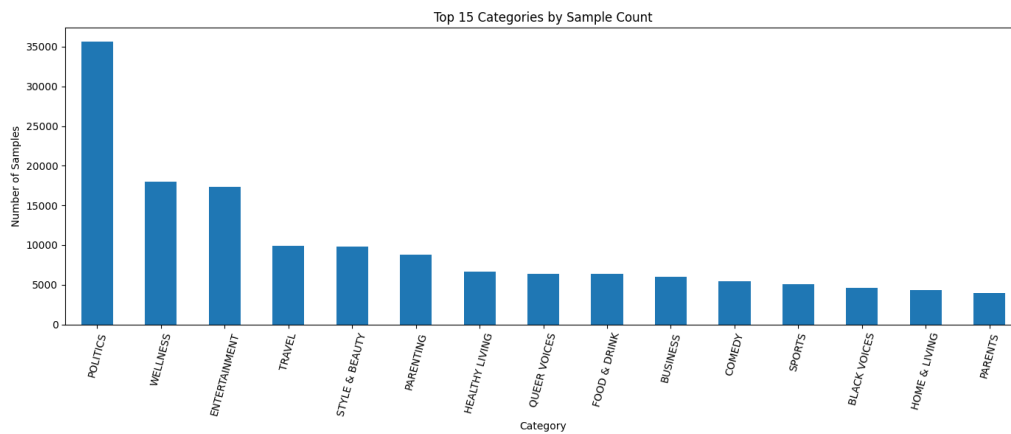


Figure 1: Phân phối 15 category lớn nhất theo số mẫu.

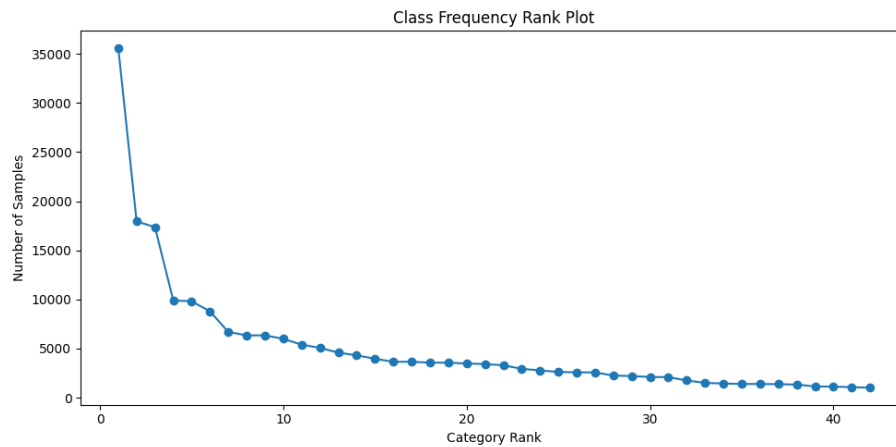


Figure 2: Đường cong tần suất category theo thứ hạng.

Lớp lớn nhất là **POLITICS** với 35.602 mẫu, trong khi lớp nhỏ nhất là **EDUCATION** với 1.014 mẫu. Tỷ lệ mất cân bằng giữa lớp lớn nhất và nhỏ nhất khoảng **35,11 lần**, cho thấy dữ liệu có sự chênh lệch rõ rệt giữa các category. Top 5 category chiếm khoảng 43,25% toàn bộ dữ liệu, còn Top 15 chiếm khoảng 70,69%.

Do các lớp nhỏ có số lượng mẫu hạn chế, mô hình dễ học kém ổn định nếu giữ toàn bộ 42 category. Vì vậy, trong pipeline huấn luyện, chỉ **giữ lại 15 category có số lượng mẫu cao nhất** để giảm mức độ mất cân bằng, tăng độ ổn định khi huấn luyện và tập trung vào các nhóm chủ đề có đủ dữ liệu đại diện. Tuy nhiên, ngay cả trong tập Top 15, phân phối nhãn vẫn còn chênh lệch đáng kể, đặc biệt lớp **POLITICS** chiếm tỷ trọng lớn hơn nhiều so với các lớp còn lại. Do mục tiêu của mô hình không chỉ là đạt accuracy cao mà còn cần nhận diện tương đối cân bằng giữa các category, pipeline có thể bổ sung **class weight** trong quá trình huấn luyện. Cách này giúp tăng trọng số lỗi cho các lớp ít mẫu hơn, từ đó cải thiện khả năng học của mô hình trên các category có tần suất thấp. Khi đánh giá, nên theo dõi thêm **macro-F1** và **per-class recall** thay vì chỉ dựa vào accuracy tổng thể.

2.4 Phân tích độ dài văn bản

Table 5: Thống kê độ dài headline, description và text ghép

Biến	Mean	Std	Min	P25	Median	P75	Max
Headline - số từ	9,60	3,07	0	8	10	12	44
Description - số từ	19,67	14,15	0	10	19	24	243
Text ghép - số từ	29,27	13,80	0	20	28	35	245
Headline - số ký tự	58,42	18,81	0	46	60	71	320
Description - số ký tự	114,21	80,84	0	59	120	134	1.472
Text ghép - số ký tự	173,62	78,55	1	123	171	208	1.487

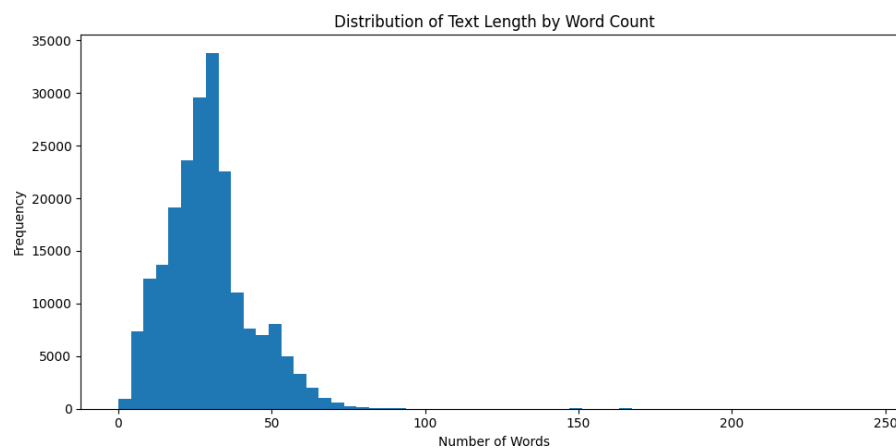


Figure 3: Phân phối độ dài text theo số từ.

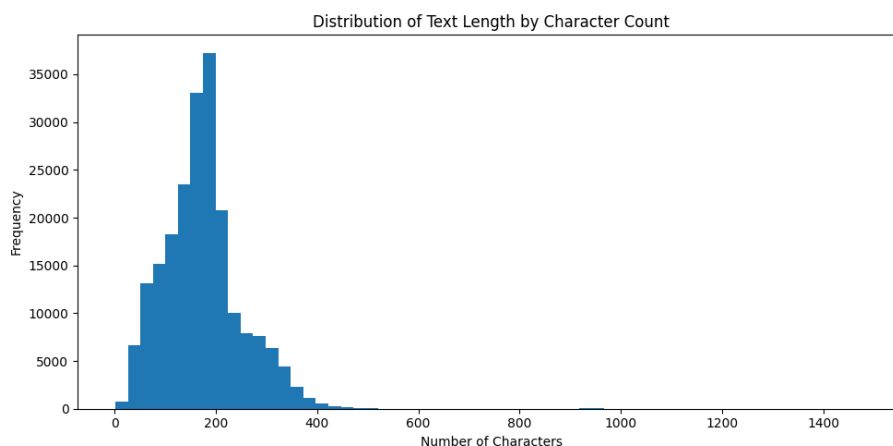


Figure 4: Phân phối độ dài text theo số ký tự.

Độ dài văn bản trong bộ dữ liệu nhìn chung khá ngắn, phù hợp với đặc trưng của dữ liệu tin tức gồm tiêu đề và mô tả ngắn. Trung vị của **headline** là 10 từ, trong khi **short_description** có trung vị 19 từ. Sau khi ghép hai trường này, **text** có trung vị khoảng 28 từ và trung bình khoảng 29 từ.

Phân phối độ dài có xu hướng lệch phải, tức là phần lớn mẫu có độ dài ngắn hoặc trung bình, nhưng vẫn tồn tại một số văn bản dài bất thường. Điều này cho thấy khi đưa dữ liệu vào mô hình, cần lựa chọn độ dài chuỗi tối đa hợp lý để vừa giữ được phần lớn thông tin, vừa tránh padding quá nhiều hoặc bị ảnh hưởng bởi các mẫu quá dài.

2.5 Phân tích độ dài văn bản theo 15 category

Table 6: Thống kê độ dài text theo Top 15 category

Category	Count	Mean	Median	Std	Min	Max
POLITICS	35.602	26,27	25,00	12,95	0	245
WELLNESS	17.945	39,04	36,00	11,75	3	171
ENTERTAINMENT	17.362	23,20	21,00	11,48	1	98
TRAVEL	9.900	34,76	33,00	13,26	1	175
STYLE & BEAUTY	9.814	32,17	32,00	8,17	2	165
PARENTING	8.791	38,46	35,00	11,37	5	150
HEALTHY LIVING	6.694	28,30	25,00	17,56	1	176
QUEER VOICES	6.347	29,78	27,00	14,64	0	85
FOOD & DRINK	6.340	26,18	25,00	10,63	4	133
BUSINESS	5.992	31,25	30,00	15,37	0	153
COMEDY	5.400	22,67	21,00	11,37	1	128
SPORTS	5.077	23,94	22,00	11,56	1	88
BLACK VOICES	4.583	27,50	27,00	11,38	2	110
HOME & LIVING	4.320	29,37	30,00	9,46	2	120
PARENTS	3.955	29,45	26,00	15,91	3	122

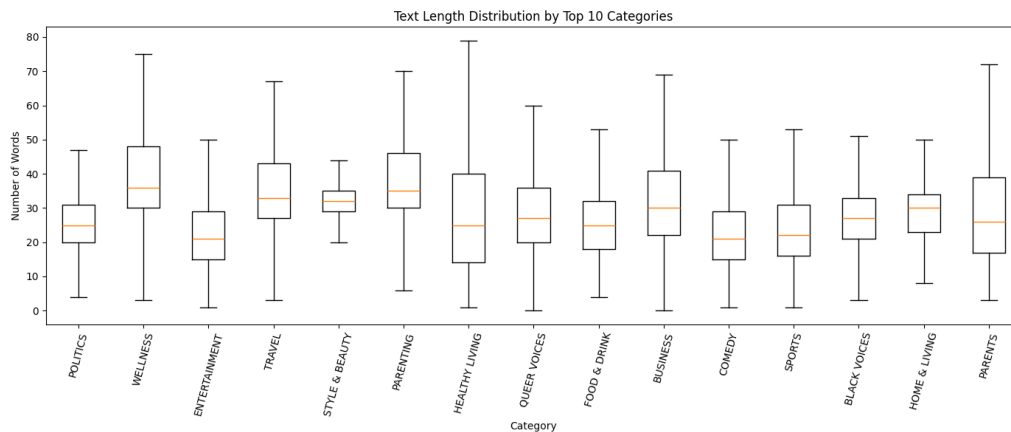


Figure 5: Boxplot độ dài text của Top 15 category, bỏ outlier để nhìn rõ trung vị/IQR.

Độ dài văn bản có sự khác biệt rõ rệt giữa các category. Các nhóm như **WELLNESS**, **PARENTING**, **TRAVEL** và **STYLE & BEAUTY** có độ dài trung bình cao hơn, cho thấy nội dung thường có phần mô tả dài hơn và giàu ngữ cảnh hơn. Ngược lại, các nhóm như **COMEDY**, **ENTERTAINMENT** và **SPORTS** có độ dài trung bình thấp hơn, phản ánh đặc trưng tiêu đề ngắn, trực tiếp và dễ nhận diện theo chủ đề.

Boxplot cho thấy phần lớn văn bản trong các category tập trung ở khoảng độ dài ngắn đến trung bình, nhưng vẫn tồn tại một số mẫu dài bất thường. Sự khác biệt này là cơ sở để lựa chọn độ dài chuỗi tối đa phù hợp khi huấn luyện mô hình, nhằm giữ lại phần lớn thông tin quan trọng nhưng không làm tăng quá nhiều padding.

2.6 Phân tích từ và cụm từ phổ biến và vocabulary

Văn bản được chuẩn hóa phục vụ cho EDA, trong đó có bao gồm loại bỏ stopwords trong Tiếng Anh.

Table 7: Tổng quan từ vựng sau bước làm sạch EDA

Chỉ số	Giá trị
Vocabulary size	107.075
Tổng số token sau cleaning	3.408.187

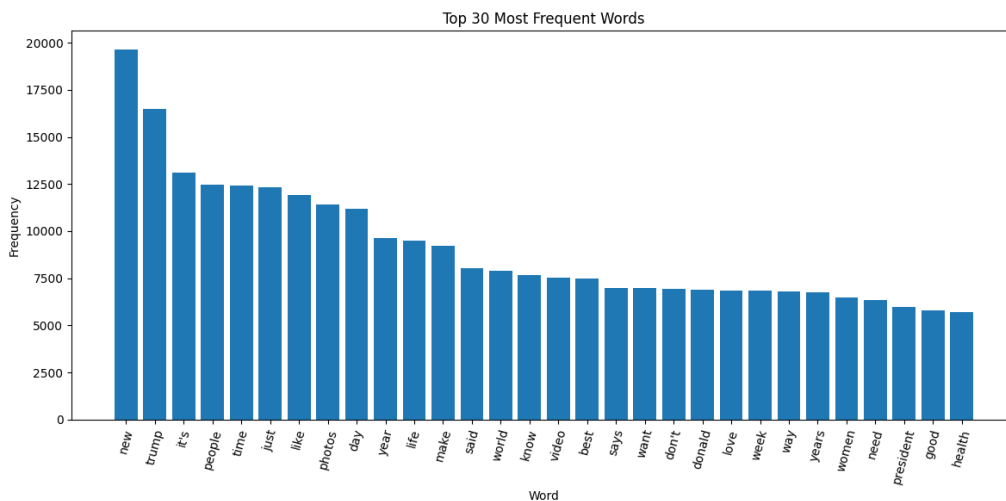


Figure 6: Top 30 token phổ biến nhất trong toàn bộ corpus sau cleaning.

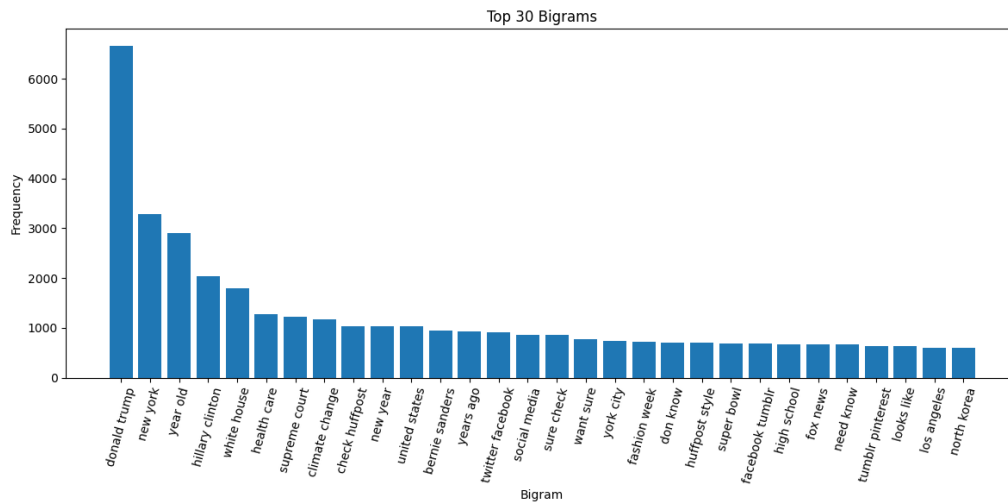


Figure 7: Top 30 bigram trong corpus.

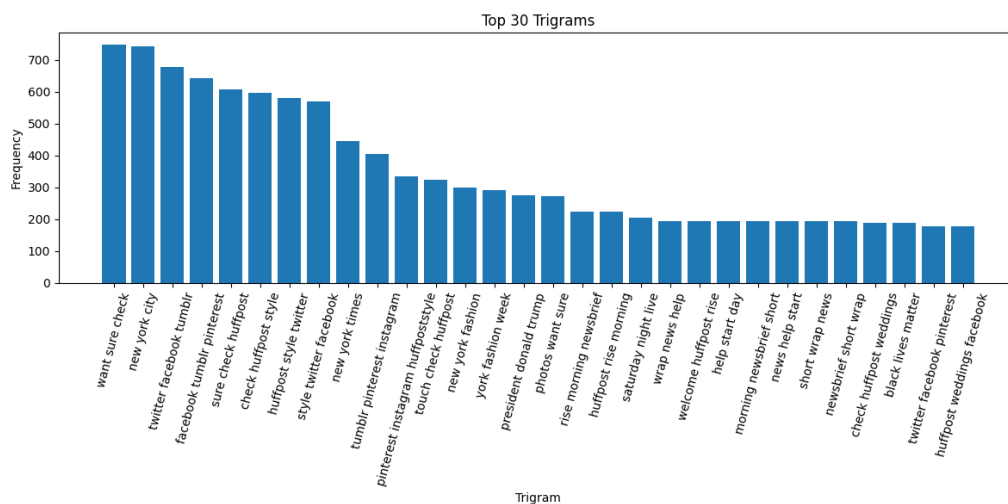


Figure 8: Top 30 trigram trong corpus.

Ở mức unigram, nhiều từ xuất hiện với tần suất cao như *new*, *people*, *time*, *just*, *like*, *day*, *year* còn khá chung chung và không mang nhiều ý nghĩa phân biệt chủ đề. Các từ này phản ánh đặc điểm ngôn ngữ phổ biến trong tin tức, nhưng nếu xét riêng lẻ thì chưa đủ mạnh để nhận diện category. Một số từ như *trump*, *photos*, *president*, *health* có giá trị ngữ nghĩa rõ hơn, tuy nhiên số lượng chưa nhiều trong nhóm từ phổ biến nhất.

Ở mức bigram, các cụm từ bắt đầu mang nhiều thông tin hơn, chẳng hạn *donald trump*, *new york*, *year old*, *hillary clinton*, *white house*. Những cụm này có

ngữ cảnh rõ ràng hơn từ đơn và liên quan trực tiếp đến một số nhóm chủ đề như chính trị, xã hội, đời sống hoặc tin tức Hoa Kỳ. Vì vậy, bigram cho thấy tín hiệu phân loại tốt hơn so với unigram.

Ở mức trigram, chỉ một số cụm thật sự có ý nghĩa chủ đề rõ ràng như *new york times*, *new york city* hoặc các cụm liên quan đến nhân vật/sự kiện. Bên cạnh đó vẫn xuất hiện nhiều cụm mang tính mẫu câu, nguồn tin hoặc metadata mạng xã hội như *twitter facebook tumblr*, *facebook tumblr pinterest*. Do đó, trigram có thể bổ sung ngữ cảnh, nhưng không phải toàn bộ cụm tần suất cao đều hữu ích cho phân loại.

2.7 Các từ phổ biến theo category

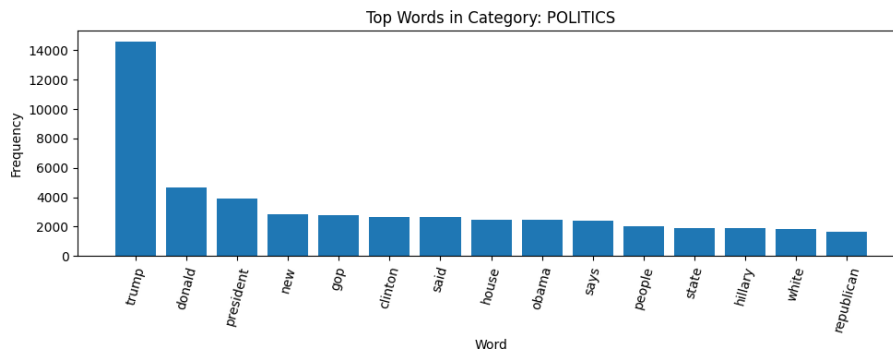


Figure 9: Top words theo category POLITICS

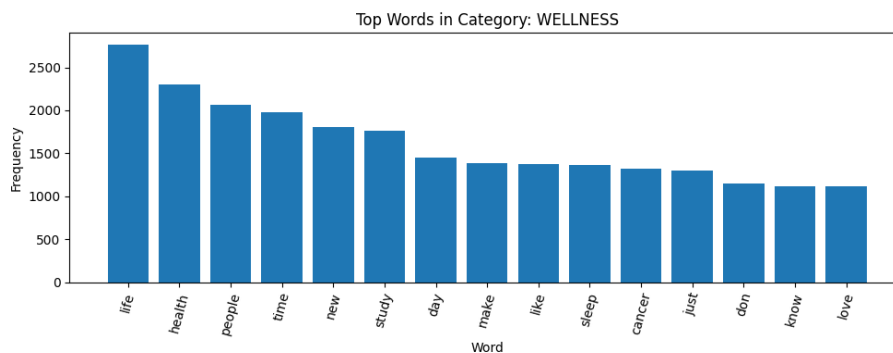


Figure 10: Top words theo category WELLNESS

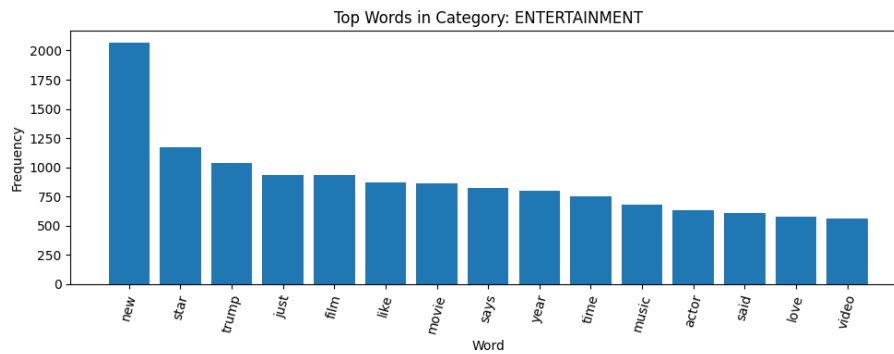


Figure 11: Top words theo category ENTERTAINMENT

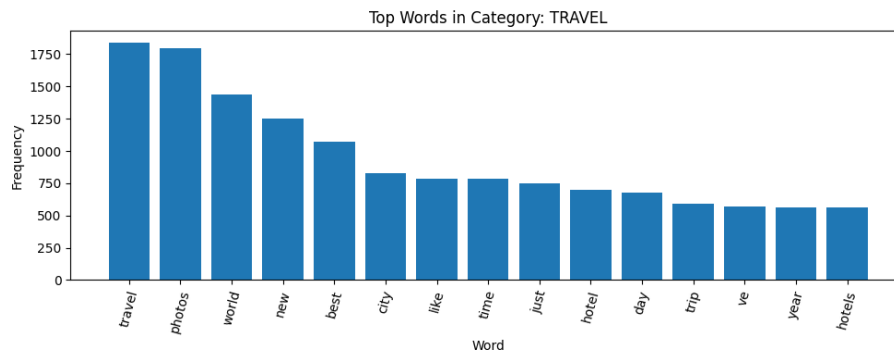


Figure 12: Top words theo category TRAVEL

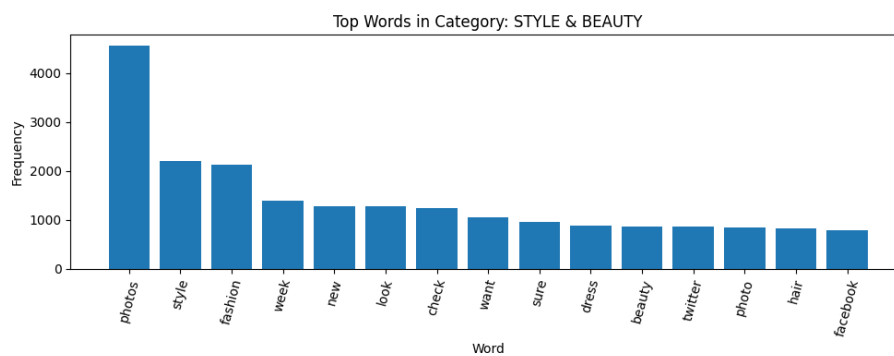


Figure 13: Top words theo category STYLE & BEAUTY

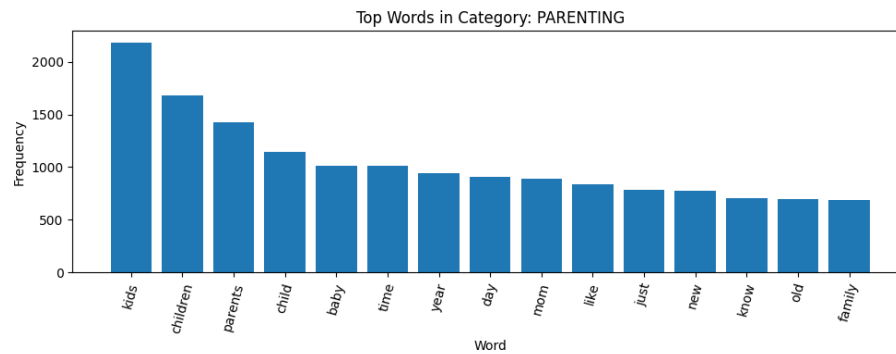


Figure 14: Top words theo category PARENTING

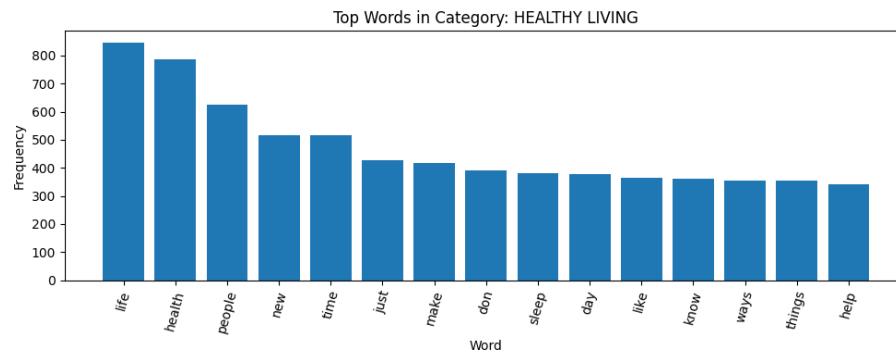


Figure 15: Top words theo category HEALTHY LIVING

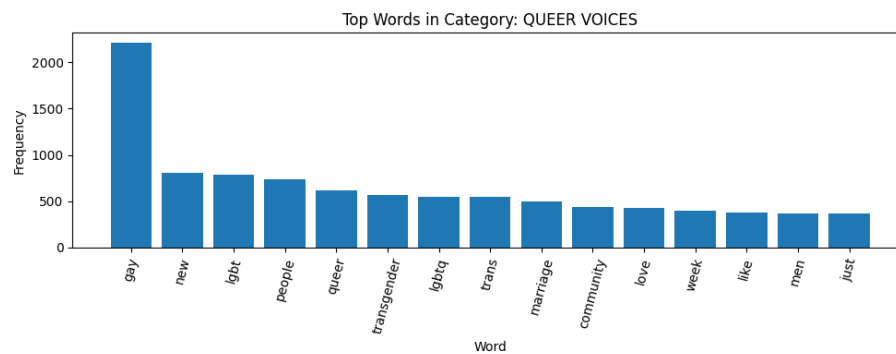


Figure 16: Top words theo category QUEER VOICES

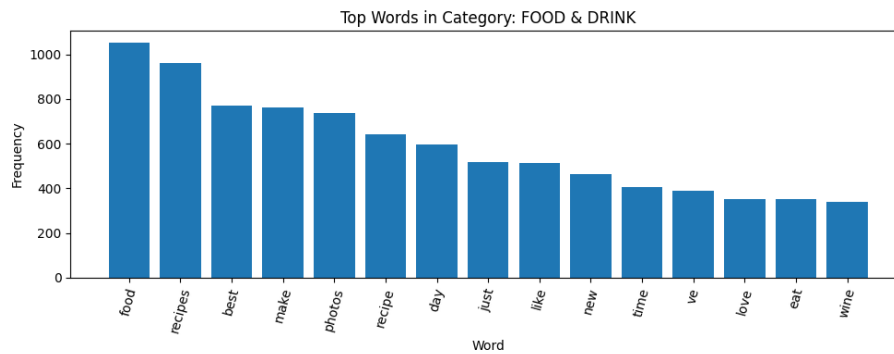


Figure 17: Top words theo category FOOD & DRINK

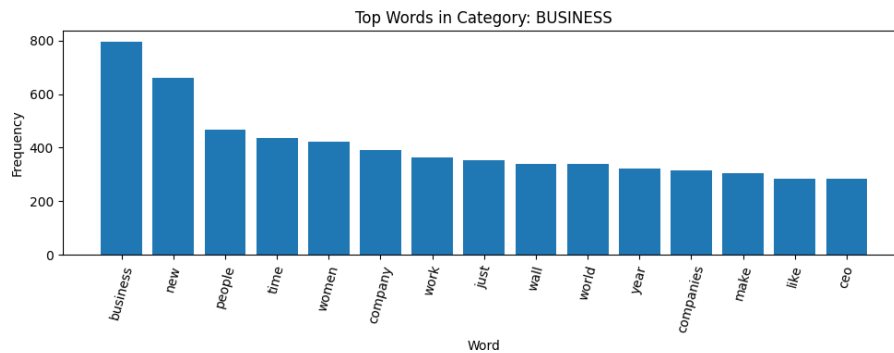


Figure 18: Top words theo category BUSINESS

Các biểu đồ top words theo từng category cho thấy mỗi nhóm chủ đề đều có một số từ khóa đặc trưng riêng, giúp phân biệt với các nhóm còn lại. Ví dụ, **POLITICS** nổi bật với các từ như *trump, donald, president, gop, clinton, obama*; **PARENTING** có các từ như *kids, children, parents, child, baby*; còn **FOOD & DRINK** xuất hiện nhiều các từ như *food, recipes, recipe, eat, wine*. Đây là những tín hiệu ngữ nghĩa khá rõ ràng, có khả năng hỗ trợ tốt cho mô hình phân loại.

Tuy nhiên, không phải toàn bộ từ phổ biến đều mang tính phân biệt cao. Một số từ như *new, people, time, just, like, day, year* xuất hiện ở nhiều category khác nhau, ví dụ trong **WELLNESS, BUSINESS, TRAVEL, ENTERTAINMENT** và **HEALTHY LIVING**. Những từ này phản ánh cách viết phổ biến trong dữ liệu tin tức nhưng nếu xét riêng lẻ thì không đủ mạnh để xác định chủ đề.

Một số category có mức độ đặc trưng từ khóa rất cao, chẳng hạn **QUEER VOICES** với các từ như *gay, lgbt, queer, transgender, lgbtq*; **STYLE & BEAUTY** với *photos, style, fashion, look, beauty*; và **POLITICS** với các tên riêng/chủ thể chính trị. Ngược lại, các nhóm như **WELLNESS, HEALTHY LIVING** hoặc

BUSINESS có nhiều từ chung hơn, nên mô hình cần dựa thêm vào ngữ cảnh câu và tổ hợp từ thay vì chỉ dựa vào tần suất từ đơn.

2.8 Phân tích từ khóa đặc trưng bằng TF-IDF

Bên cạnh tần suất xuất hiện của từ, TF-IDF được sử dụng để xác định các từ hoặc cụm từ nổi bật trong từng category. TF-IDF được tính riêng cho từng category bằng `TfidfVectorizer` với `ngram_range=(1,2)`, cho phép xét cả từ đơn và cụm hai từ. Sau khi vector hoá các văn bản trong một category, điểm TF-IDF của từng từ/cụm từ được lấy trung bình trên toàn bộ mẫu thuộc category đó. Những từ/cụm từ có TF-IDF trung bình cao nhất được xem là keyword đại diện cho category.

Table 8: Từ khóa TF-IDF nổi bật theo từng category

Category	Từ khóa TF-IDF nổi bật
POLITICS	<i>trump, donald, donald trump, president, clinton, gop, obama, new, house, said, says, hillary, people, state, white</i>
WELLNESS	<i>life, health, people, time, study, new, sleep, day, cancer, make, like, just, love, know, don</i>
ENTERTAINMENT	<i>new, star, trump, just, like, says, movie, film, year, time, music, season, actor, love, watch</i>
TRAVEL	<i>travel, photos, world, new, best, city, like, time, just, hotel, day, trip, hotels, vacation, year</i>
STYLE & BEAUTY	<i>photos, style, fashion, week, check, look, new, want, sure, twitter, hair, beauty, photo, facebook, dress</i>
PARENTING	<i>kids, children, parents, child, baby, time, year, mom, day, like, just, new, know, parenting, old</i>
HEALTHY LIVING	<i>health, life, people, new, sleep, time, just, make, know, don, ways, things, day, need, cancer</i>
QUEER VOICES	<i>gay, new, lgbt, queer, people, transgender, trans, lgbtq, marriage, love, community, like, men, sex, just</i>
FOOD & DRINK	<i>food, recipes, photos, best, make, recipe, day, just, like, new, time, eat, ve, love, wine</i>
BUSINESS	<i>business, new, women, people, time, company, work, world, just, wall, companies, ceo, year, america, make</i>

Kết quả TF-IDF cho thấy nhiều category có bộ từ khóa khá đặc trưng. Chẳng hạn, **POLITICS** nổi bật với các từ liên quan đến chính trị và nhân vật công chúng như *trump, donald trump, president, clinton, gop, obama*. Tương tự, **QUEER**



VOICES có các từ khóa rõ ràng như *gay, lgbt, queer, transgender, lgbtq*; còn **PARENTING** tập trung vào các từ như *kids, children, parents, child, baby*.

So với phân tích tần suất từ đơn, TF-IDF giúp làm nổi bật tốt hơn các từ có giá trị đại diện cho từng category. Các nhóm như **FOOD & DRINK, TRAVEL, STYLE & BEAUTY** có từ khóa khá trực quan và dễ phân biệt, ví dụ *recipes, travel, hotel, fashion, beauty*. Tuy nhiên, vẫn tồn tại một số từ chung như *new, people, time, just, like* xuất hiện trong nhiều category, nên mô hình cần học thêm ngữ cảnh và quan hệ giữa các từ thay vì chỉ dựa vào từng keyword riêng lẻ.

3 Tiền xử lý dữ liệu

Dữ liệu chứa nhiều ký tự đặc biệt, chữ số, dấu câu và khoảng trắng dư thừa,... Vì vậy cần thực hiện các bước tiền xử lý nhằm chuẩn hóa dữ liệu, giảm nhiễu và cải thiện chất lượng đầu vào cho các bước trích xuất đặc trưng và phân loại.

3.1 Lựa chọn thuộc tính

Từ bộ dữ liệu ban đầu, nghiên cứu sử dụng ba thuộc tính chính gồm:

- *category*: nhãn phân loại bài báo.
- *headline*: tiêu đề bài báo.
- *short_description*: mô tả ngắn của bài báo.

Hai trường *headline* và *short_description* được kết hợp để tạo thành một trường văn bản mới là *text*.

$$\text{text} = \text{headline} + \text{short_description} \quad (1)$$

Việc kết hợp này giúp văn bản đầu vào chứa đầy đủ thông tin ngữ nghĩa hơn so với chỉ sử dụng tiêu đề hoặc mô tả riêng lẻ.

3.2 Xử lý giá trị thiếu

Trong quá trình xử lý, các giá trị bị thiếu trong hai cột *headline* và *short_description* được thay thế bằng chuỗi rỗng nhằm tránh lỗi khi ghép văn bản và đảm bảo toàn bộ dữ liệu có thể được xử lý thống nhất.

$$\text{missing value} \rightarrow "" \quad (2)$$

3.3 Kiểm tra và loại bỏ dữ liệu trùng lặp

Sau khi tạo cột *text*, dữ liệu được kiểm tra các trường hợp trùng lặp nhằm giảm nhiễu và tránh ảnh hưởng đến quá trình huấn luyện mô hình.

Hai dạng trùng lặp được xem xét:

- Trùng lặp toàn bộ dòng dữ liệu.
- Trùng lặp nội dung văn bản trong cột *text*.

Trong nghiên cứu này, các văn bản trùng lặp trong cột *text* được loại bỏ bằng phương pháp:

$$df = df.drop_duplicates(subset = ["text"]) \quad (3)$$

Việc loại bỏ dữ liệu trùng giúp hạn chế hiện tượng mô hình học lặp lại cùng một mẫu dữ liệu, đồng thời giảm nguy cơ rò rỉ dữ liệu giữa tập huấn luyện và tập kiểm tra.

3.4 Lựa chọn các nhãn phổ biến

Bộ dữ liệu gốc chứa nhiều nhóm chủ đề khác nhau với phân bố không đồng đều. Để giảm mất cân bằng dữ liệu và tập trung vào các lớp có đủ số lượng mẫu, nghiên cứu lựa chọn 15 nhãn xuất hiện nhiều nhất trong tập dữ liệu.

Sau bước này, chỉ các mẫu thuộc 15 nhãn phổ biến nhất được giữ lại để phục vụ quá trình huấn luyện và đánh giá mô hình.

3.5 Làm sạch văn bản

Dữ liệu văn bản được chuẩn hóa nhằm loại bỏ các thành phần không cần thiết và giảm nhiễu trong quá trình xử lý ngôn ngữ tự nhiên. Các bước làm sạch bao gồm:

- Chuyển toàn bộ văn bản về chữ thường.
- Loại bỏ URL.
- Loại bỏ thẻ HTML.
- Loại bỏ ký tự đặc biệt.
- Chuẩn hóa khoảng trắng.

Sau bước này, văn bản được lưu vào cột *clean_text*.

3.6 Loại bỏ stopwords và lemmatization

Các stopwords tiếng Anh như *the*, *is*, *and* được loại bỏ vì chúng xuất hiện thường xuyên nhưng ít mang ý nghĩa phân loại.

Sau đó, văn bản được đưa qua quá trình *lemmatization* nhằm đưa từ về dạng gốc. Trước khi lemmatization, hệ thống thực hiện gán nhãn từ loại (POS tagging) để tăng độ chính xác khi xác định dạng chuẩn của từ.

Ví dụ:

- *running* → *run*
- *cars* → *car*
- *better* → *good*

Kết quả sau tiền xử lí được lưu trong cột *ml_clean_text*.

3.6.1 Chia tập dữ liệu

Dữ liệu sau tiền xử lí được chia thành ba tập:

- Tập huấn luyện (70%)
- Tập validation (15%)
- Tập kiểm tra (15%)

Quá trình chia dữ liệu sử dụng phương pháp phân tầng theo nhãn nhằm đảm bảo tỉ lệ các lớp trong từng tập dữ liệu gần tương đồng với tập dữ liệu ban đầu.

4 Biểu diễn văn bản

Để sử dụng dữ liệu văn bản trong các mô hình Machine Learning, cần chuyển đổi văn bản thành vector số. Trong project này sử dụng hai loại phương pháp:

- Truyền thống: Bag-of-Words (BoW), TF-IDF,
- Hiện đại (deep learning embeddings): sử dụng các mô hình học sâu để biểu diễn văn bản: BERT, SBERT

4.1 Bag-of-Words (BoW)

Bag-of-Words là phương pháp biểu diễn văn bản bằng cách đếm số lần xuất hiện của từng từ trong tài liệu.

Đặc điểm của BoW:

- Không quan tâm thứ tự từ
- Chỉ quan tâm tần suất xuất hiện
- Vector đầu ra có kích thước bằng số lượng từ vựng

Ưu điểm:

- Đơn giản
- Dễ triển khai

Nhược điểm:

- Không nắm bắt ngữ nghĩa
- Vector rất thưa (sparse)

4.2 TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) là một phương pháp biểu diễn văn bản nâng cao hơn Bag-of-Words. Phương pháp này không chỉ xét tần suất xuất hiện của từ trong một văn bản mà còn xét mức độ phổ biến của từ đó trong toàn bộ tập dữ liệu.

TF-IDF giúp giảm trọng số của các từ xuất hiện quá phổ biến và tăng trọng số cho các từ mang nhiều thông tin hơn.

Công thức tổng quát của TF-IDF được biểu diễn như sau:

```
docs = [
    "I am going to school to study for the final exam.",
    "The weather is nice today and I feel happy.",
    "I love programming in Python and exploring new libraries.",
    "Data science is an exciting field with many opportunities.",
]

bow = CountVectorizer()
vectors = bow.fit_transform(docs)

for i, vec in enumerate(vectors):
    print(f"Document {i+1}: {vec.toarray()}")
```

```
... Document 1: [[1 0 0 0 1 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 1 1 2 0 0 0]]
Document 2: [[0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 1 0]]
Document 3: [[0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 1 1 0 1 0 0 1 1 0 0 0 0 0 0 0 0]]
Document 4: [[0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 1 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0 1]]
```

Figure 19: Ví dụ minh họa cho BoW

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

Trong đó:

$$TF(t, d) = \frac{\text{số lần xuất hiện của từ } t \text{ trong văn bản } d}{\text{tổng số từ trong văn bản } d}$$

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

Với:

- t là một từ trong tập từ vựng.
- d là một văn bản.
- N là tổng số văn bản trong tập dữ liệu.
- $df(t)$ là số văn bản có chứa từ t .

Khác với BoW, vector TF-IDF không chỉ chứa số lần xuất hiện của từ mà chứa trọng số phản ánh mức độ quan trọng của từ trong văn bản.

Các giá trị trong vector TF-IDF thường là số thực thay vì số nguyên.

- Ưu điểm: biểu diễn tốt hơn BoW do có xét đến độ quan trọng của từ.
- Nhược điểm: vẫn chưa nắm bắt được ngữ nghĩa và quan hệ ngữ cảnh giữa các từ.

```
docs = [
    "I am going to school to study for the final exam.",
    "The weather is nice today and I feel happy.",
    "I love programming in Python and exploring new libraries.",
    "Data science is an exciting field with many opportunities.",
]

vectorizer = TfidfVectorizer()
tfidf_vectors = vectorizer.fit_transform(docs)

for i, vec in enumerate(tfidf_vectors):
    print(f"TF-IDF for Document {i+1}:")
    print(vec.toarray())
```

Figure 20: Ví dụ minh họa cho TF-IDF

```
... TF-IDF for Document 1:
[[0.29333722 0. 0. 0.29333722 0.
 0. 0. 0. 0.29333722 0.29333722 0.29333722
 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.29333722
 0. 0.29333722 0.23127044 0.58667444 0.
 0.]]

TF-IDF for Document 2:
[[0. 0. 0.30091213 0. 0. 0.
 0. 0.38166888 0. 0. 0. 0.
 0.38166888 0. 0.30091213 0. 0. 0.
 0. 0.38166888 0. 0. 0. 0.
 0. 0. 0.30091213 0. 0.38166888 0.38166888
 0.]]

TF-IDF for Document 3:
[[0. 0. 0.2855815 0. 0. 0.
 0.36222393 0. 0. 0. 0. 0.
 0. 0.36222393 0. 0.36222393 0.36222393 0.
 0.36222393 0. 0.36222393 0.36222393 0.
 0. 0. 0. 0. 0. 0.
 0.]]

TF-IDF for Document 4:
[[0. 0.34056989 0. 0.34056989 0. 0.34056989
 0. 0. 0.34056989 0. 0. 0.
 0. 0. 0.26850921 0. 0. 0.34056989
 0. 0. 0.34056989 0. 0. 0.
 0.34056989 0. 0. 0. 0. 0.
 0.34056989]]
```

Figure 21: Ví dụ minh họa cho TF-IDF

4.3 Mô hình BERT (một ứng dụng trong Xử lý Ngôn ngữ Tự nhiên)

BERT (Bidirectional Encoder Representations from Transformers) là mô hình biểu diễn ngôn ngữ do Google giới thiệu nhằm cải thiện hiệu quả của các bài toán xử lý ngôn ngữ tự nhiên (NLP). Khác với các mô hình trước đây chỉ khai thác ngữ cảnh một chiều (từ trái sang phải hoặc ngược lại), BERT sử dụng cơ chế học ngữ cảnh hai chiều dựa trên kiến trúc Transformer Encoder. Nhờ đó, mô hình có khả năng hiểu tốt hơn mối quan hệ giữa các từ trong câu và đạt kết quả vượt trội trên nhiều tác vụ NLP như Question Answering, Natural Language Inference hay Named Entity Recognition.

Kiến trúc của BERT được xây dựng từ nhiều lớp Bidirectional Transformer Encoder theo mô hình Transformer. Hai phiên bản chính được đề xuất gồm:

- **BERT_{BASE}**: gồm 12 lớp Transformer, kích thước hidden là 768 và 12 attention heads với tổng cộng 110 triệu tham số.

- **BERT_{LARGE}**: gồm 24 lớp Transformer, kích thước hidden là 1024 và 16 attention heads với khoảng 340 triệu tham số.

Dầu vào của BERT có thể là một câu đơn hoặc một cặp câu. Biểu diễn dầu vào được hình thành bằng cách cộng ba thành phần gồm token embedding, segment embedding và positional embedding. Trong đó, token đặc biệt [CLS] được thêm vào đầu chuỗi để biểu diễn toàn bộ câu trong các tác vụ phân loại, còn token [SEP] được dùng để phân tách các câu. BERT sử dụng phương pháp WordPiece Embedding với từ điển khoảng 30.000 từ nhằm xử lý hiệu quả các từ hiếm và từ ghép.

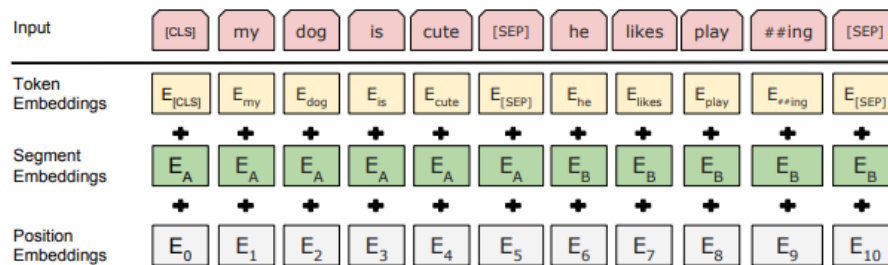


Figure 2: BERT input representation. The input embeddings is the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Quá trình tiền huấn luyện (pre-training) của BERT dựa trên hai nhiệm vụ chính:

1. **Masked Language Model (MLM)**: Một số token trong câu được che ngẫu nhiên và mô hình phải dự đoán lại các token bị ẩn. Cách tiếp cận này cho phép BERT học biểu diễn ngữ nghĩa hai chiều.
2. **Next Sentence Prediction (NSP)**: Mô hình dự đoán xem hai câu có liên tiếp nhau trong ngữ cảnh thực tế hay không nhằm học được mối quan hệ giữa các câu.

Dữ liệu dùng để huấn luyện gồm BooksCorpus với khoảng 800 triệu từ và English Wikipedia với khoảng 2,5 tỷ từ. Sau khi pre-training, mô hình được fine-tuning cho từng tác vụ cụ thể bằng cách bổ sung thêm một lớp đầu ra phù hợp. Điểm mạnh của BERT là có thể áp dụng cho nhiều bài toán khác nhau mà không cần thay đổi nhiều kiến trúc mạng.

Nhìn chung, BERT là một bước tiến quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên nhờ khả năng học ngữ cảnh hai chiều mạnh mẽ. Kiến trúc này không chỉ nâng cao hiệu năng cho nhiều tác vụ NLP mà còn tạo nền tảng cho hàng loạt mô hình tiên tiến sau này như RoBERTa, ALBERT hay DistilBERT.

5 Huấn luyện và đánh giá mô hình phân loại

5.1 Thuật toán Random Forest

Random Forest là một thuật toán học máy thuộc nhóm học có giám sát, được sử dụng rộng rãi cho cả bài toán phân loại và hồi quy. Thuật toán này được xây dựng dựa trên ý tưởng kết hợp nhiều cây quyết định thay vì chỉ sử dụng một cây đơn lẻ. Mỗi cây trong “rừng” sẽ đưa ra một dự đoán riêng, sau đó các dự đoán này được tổng hợp để tạo thành kết quả cuối cùng. Nhờ cơ chế đó, Random Forest thường cho kết quả ổn định hơn, giảm hiện tượng phụ thuộc quá mức vào một mô hình đơn lẻ và nâng cao khả năng tổng quát hóa trên dữ liệu mới.

Về nguyên lý hoạt động, Thuật toán Random Forest là một phương pháp học máy dựa trên kỹ thuật Ensemble Learning (Học kết hợp), cụ thể là cơ chế **Bagging** (Bootstrap Aggregating). Nguyên lý hoạt động dựa trên hai trụ cột ngẫu nhiên chính để xây dựng một tập hợp gồm T cây quyết định:

- **Lấy mẫu Bootstrap (Bootstrap Sampling):** Từ tập dữ liệu huấn luyện ban đầu $\mathcal{D}$ có N mẫu, thuật toán thực hiện lấy mẫu ngẫu nhiên có lặp (random sampling with replacement) để tạo ra T tập dữ liệu con $\{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_T\}$. Mỗi tập con $\mathcal{D}_t$ được dùng để huấn luyện một cây quyết định tương ứng.
- **Không gian đặc trưng ngẫu nhiên (Random Subspace):** Tại mỗi nút trong quá trình xây dựng cây, thay vì xét toàn bộ M đặc trưng đầu vào, thuật toán chỉ chọn ngẫu nhiên một nhóm con gồm m đặc trưng ($m < M$, thường $m = \sqrt{M}$ cho bài toán phân loại) để tìm điểm tách tối ưu.

Quyết định cuối cùng:

Trong bài toán phân loại (như phân loại danh mục tin tức trong bài tập này), dự đoán cuối cùng $\hat{y}$ cho một mẫu dữ liệu mới $\mathbf{x}$ được thực hiện thông qua cơ chế **Bỏ phiếu đa số (Majority Voting)**:

$$\hat{y} = \operatorname{argmax}_{c \in \mathcal{C}} \sum_{t=1}^T I(h_t(\mathbf{x}) = c) \quad (4)$$

Trong đó:

- $\mathcal{C}$ là tập hợp các nhãn lớp (các danh mục tin tức).
- $h_t(\mathbf{x})$ là kết quả dự đoán nhãn của cây quyết định thứ t .
- $I(\cdot)$ là hàm chỉ thị (Indicator function), nhận giá trị 1 nếu điều kiện đúng và 0 nếu sai.

Một số tham số quan trọng của Random Forest có ảnh hưởng trực tiếp đến hiệu năng mô hình:

- **n_estimators**: quy định số lượng cây trong rừng; số cây càng lớn thì mô hình thường ổn định hơn nhưng thời gian huấn luyện cũng tăng lên.
- **max_depth**: quyết định độ sâu tối đa của mỗi cây, ảnh hưởng đến khả năng học các mẫu phức tạp cũng như nguy cơ overfitting.
- **max_features**: xác định số lượng đặc trưng được chọn ngẫu nhiên tại mỗi lần phân chia nút; đây là yếu tố quan trọng giúp tạo ra sự khác biệt giữa các cây.
- **min_samples_split**: kiểm soát điều kiện tối thiểu để một nút tiếp tục được phân chia.
- **min_samples_leaf**: quy định số lượng mẫu tối thiểu tại một nút lá, từ đó ảnh hưởng đến độ phức tạp của cây.
- **bootstrap**: quyết định có sử dụng lấy mẫu ngẫu nhiên có lặp hay không; trong đa số trường hợp tham số này được bật để giữ đúng bản chất của Random Forest.

Ưu điểm của Random Forest gồm:

- Giảm overfitting tốt hơn so với cây quyết định đơn nhờ cơ chế kết hợp nhiều cây độc lập.
- Hoạt động khá ổn định trên dữ liệu nhiễu và dữ liệu có nhiều đặc trưng.
- Không yêu cầu chuẩn hóa dữ liệu quá nghiêm ngặt như một số thuật toán khác.
- Có khả năng đánh giá mức độ quan trọng của từng đặc trưng, hỗ trợ phân tích và giải thích mô hình.
- Có thể áp dụng hiệu quả cho nhiều bài toán thực tế, bao gồm cả phân loại văn bản sau bước vector hóa như Bag-of-Words hoặc TF-IDF.

Tuy nhiên, Random Forest cũng tồn tại một số nhược điểm:

- Khi số lượng cây lớn, mô hình có thể tiêu tốn nhiều bộ nhớ và thời gian huấn luyện hơn so với các phương pháp đơn giản.

- Cần điều chỉnh một số siêu tham số quan trọng để đạt hiệu quả tối ưu; nếu lựa chọn chưa phù hợp, mô hình có thể học chưa tốt hoặc tốn tài nguyên không cần thiết.
- Khó diễn giải hơn so với một cây quyết định đơn lẻ vì dự đoán cuối cùng là kết quả tổng hợp từ rất nhiều cây.
- Trong các bài toán có dữ liệu rất thưa và số chiều cực lớn, một số mô hình tuyến tính hoặc boosting đôi khi có thể cho hiệu quả tốt hơn.

Tóm lại, Random Forest là một thuật toán mạnh, linh hoạt và có tính ứng dụng cao trong nhiều lĩnh vực. Nhờ kết hợp nhiều cây quyết định thông qua cơ chế lấy mẫu ngẫu nhiên và chọn đặc trưng ngẫu nhiên, thuật toán này vừa cải thiện độ chính xác vừa giảm nguy cơ overfitting. Với các ưu điểm về độ ổn định, khả năng xử lý dữ liệu đa dạng và hỗ trợ đánh giá tầm quan trọng đặc trưng, Random Forest là một trong những thuật toán nền tảng và phổ biến nhất trong học máy hiện đại.

5.2 LightGBM

LightGBM (Light Gradient Boosting Machine) là một thuật toán học máy thuộc nhóm Gradient Boosting Decision Tree (GBDT), được phát triển bởi Microsoft nhằm tối ưu tốc độ huấn luyện và hiệu năng trên các bộ dữ liệu lớn. Thuật toán này thường được sử dụng trong các bài toán phân loại, hồi quy và xếp hạng nhờ khả năng xử lý dữ liệu nhanh và đạt độ chính xác cao.

Về nguyên lý, LightGBM hoạt động dựa trên kỹ thuật boosting, trong đó nhiều cây quyết định được huấn luyện tuần tự. Mỗi cây mới sẽ tập trung học các sai số mà các cây trước đó chưa dự đoán tốt. Kết quả cuối cùng được tổng hợp từ toàn bộ các cây quyết định để tạo thành mô hình mạnh hơn.

Khác với GBDT truyền thống, LightGBM sử dụng chiến lược phát triển cây theo chiều lá (leaf-wise growth) thay vì phát triển theo mức (level-wise growth). Ở mỗi bước, thuật toán sẽ mở rộng node có khả năng làm giảm hàm mất mát nhiều nhất. Điều này giúp mô hình hội tụ nhanh hơn và thường đạt độ chính xác cao hơn với cùng số lượng cây.

Ngoài ra, LightGBM còn sử dụng kỹ thuật histogram-based algorithm để tăng tốc quá trình tìm điểm chia tối ưu. Các giá trị liên tục của feature sẽ được chia thành các khoảng rời rạc (bins), từ đó giảm chi phí tính toán và bộ nhớ sử dụng. Thuật toán cũng hỗ trợ tốt với dữ liệu sparse như TF-IDF trong bài toán xử lý ngôn ngữ tự nhiên.

Một số siêu tham số quan trọng của LightGBM gồm:

- **num_leaves**: số lượng lá tối đa của mỗi cây quyết định.

- **learning_rate**: tốc độ học của mô hình.
- **n_estimators**: số lượng cây quyết định được sử dụng.
- **max_depth**: giới hạn độ sâu của cây để tránh overfitting.
- **min_data_in_leaf**: số lượng mẫu tối thiểu trong một lá.

Ưu điểm của LightGBM là tốc độ huấn luyện nhanh, sử dụng bộ nhớ hiệu quả và hoạt động tốt trên dữ liệu lớn hoặc dữ liệu có số chiều cao. Bên cạnh đó, mô hình thường đạt độ chính xác cao khi kết hợp với các phương pháp vector hóa văn bản như TF-IDF.

Tuy nhiên, LightGBM cũng có một số hạn chế như dễ bị overfitting nếu số lá cây quá lớn hoặc dữ liệu quá nhỏ. Ngoài ra, mô hình khá nhạy với việc lựa chọn siêu tham số nên thường cần kết hợp với các kỹ thuật tuning như Grid Search hoặc Randomized Search để đạt hiệu quả tối ưu.

5.3 Gradient Boosting

Khái niệm: Gradient Boosting là phương pháp *ensemble learning* kết hợp nhiều mô hình yếu, thường là cây quyết định nông, để tạo thành một mô hình mạnh. Các mô hình được huấn luyện tuần tự, trong đó mô hình sau có nhiệm vụ sửa sai cho mô hình trước.

Nguyên lý: Ở mỗi bước, thuật toán tối ưu hàm mất mát $L(y, F(x))$ bằng cách học phần sai số còn lại của mô hình hiện tại. Mô hình tổng quát có dạng:

$$F(x) = \sum_{m=1}^M \gamma_m h_m(x)$$

Và được cập nhật theo:

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$$

Trong đó $h_m(x)$ là mô hình yếu thứ m , γ_m là trọng số, và ν là *learning rate*.

Các tham số quan trọng:

- **n_estimators**: số lượng cây.
- **learning_rate**: tốc độ học.
- **max_depth**: độ sâu tối đa của cây.
- **subsample**: tỷ lệ lấy mẫu dữ liệu.

- `max_features`: số đặc trưng dùng khi chia nút.

Ưu điểm:

- Độ chính xác cao.
- Học tốt quan hệ phi tuyến.
- Hiệu quả trên dữ liệu dạng bảng.

Nhược điểm:

- Huấn luyện chậm do tính tuần tự.
- Dễ overfitting nếu tham số không phù hợp.
- Khó diễn giải hơn mô hình đơn giản.

5.4 Thuật toán CatBoost

CatBoost (*Categorical Boosting*) là một thư viện học máy mã nguồn mở do Yandex phát triển, dựa trên kỹ thuật *Gradient Boosting* với cây quyết định. Thuật toán này được sử dụng cho các bài toán phân loại, hồi quy và xếp hạng, đặc biệt phù hợp với dữ liệu có nhiều thuộc tính phân loại. CatBoost có khả năng xử lý trực tiếp các biến phân loại mà không cần thực hiện nhiều bước mã hóa thủ công như *One-Hot Encoding*, giúp đơn giản hóa quá trình tiền xử lý dữ liệu.

Về nguyên lý hoạt động, CatBoost xây dựng nhiều cây quyết định theo cách tuần tự. Mỗi cây mới được huấn luyện để giảm sai số dự đoán của các cây trước đó, từ đó cải thiện dần hiệu quả của mô hình. Bên cạnh đó, CatBoost sử dụng các kỹ thuật như *Ordered Boosting* và xử lý thống kê có thứ tự cho biến phân loại, giúp hạn chế hiện tượng *overfitting*, rò rỉ dữ liệu và cải thiện khả năng tổng quát hóa.

Một số siêu tham số quan trọng của CatBoost gồm:

- `iterations`: số lượng cây được xây dựng trong quá trình huấn luyện;
- `learning_rate`: tốc độ học, quyết định mức độ đóng góp của mỗi cây vào mô hình cuối;
- `depth`: độ sâu tối đa của mỗi cây quyết định;
- `loss_function`: hàm mất mát được sử dụng để tối ưu mô hình;
- `cat_features`: danh sách các thuộc tính phân loại được CatBoost xử lý tự động;

- `random_seed`: giá trị cố định giúp kết quả huấn luyện có thể tái lập.

Ưu điểm của CatBoost:

- Xử lý tốt dữ liệu phân loại mà không cần mã hóa thủ công phức tạp.
- Cho độ chính xác cao trên nhiều bài toán phân loại và hồi quy.
- Giảm nguy cơ *overfitting* nhờ cơ chế *Ordered Boosting*.
- Hỗ trợ huấn luyện trên cả CPU và GPU, giúp tăng tốc quá trình huấn luyện.
- Ít yêu cầu tiền xử lý dữ liệu hơn so với một số thuật toán khác.

Nhược điểm của CatBoost:

- Thời gian huấn luyện có thể lâu khi dữ liệu có kích thước rất lớn hoặc số lượng cây nhiều.
- Mô hình có nhiều siêu tham số, do đó cần điều chỉnh phù hợp để đạt hiệu quả tốt nhất.
- Khả năng diễn giải mô hình không trực quan bằng các mô hình đơn giản như cây quyết định đơn lẻ hoặc hồi quy tuyến tính.
- Với dữ liệu văn bản, CatBoost thường cần kết hợp với bước trích xuất đặc trưng như TF-IDF trước khi huấn luyện.

Trong bài toán phân loại văn bản tin tức, CatBoost được sử dụng sau bước trích xuất đặc trưng từ dữ liệu văn bản. Mô hình học mối quan hệ giữa đặc trưng đầu vào và nhãn chuyên mục, từ đó dự đoán loại tin tương ứng cho các văn bản mới. Nhờ khả năng huấn luyện ổn định, độ chính xác cao và hỗ trợ điều chỉnh nhiều siêu tham số, CatBoost là một lựa chọn phù hợp cho bài toán phân loại dữ liệu tin tức trong báo cáo này.

5.5 Thuật toán Linear SVC

Linear SVC (*Linear Support Vector Classification*) là một thuật toán phân loại thuộc nhóm *Support Vector Machine* (SVM), sử dụng siêu phẳng tuyến tính để phân tách các lớp dữ liệu. Thuật toán này đặc biệt phù hợp với các bài toán có dữ liệu nhiều chiều, chẳng hạn như phân loại văn bản sau khi dữ liệu được biểu diễn bằng các đặc trưng số như TF-IDF.

Nguyên lý chính của Linear SVC là tìm một siêu phẳng sao cho khoảng cách giữa siêu phẳng này và các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất. Khoảng cách đó được gọi là *margin*. Các điểm dữ liệu nằm gần biên phân tách nhất được

gọi là *support vectors*, vì chúng có vai trò quan trọng trong việc xác định ranh giới phân loại.

Với bài toán phân loại văn bản, mỗi văn bản được chuyển thành vector đặc trưng. Linear SVC sẽ học ranh giới tuyến tính giữa các nhóm văn bản thuộc các chuyên mục khác nhau. Do dữ liệu văn bản thường có số chiều rất lớn nhưng thưa, Linear SVC thường cho hiệu quả tốt và tốc độ huấn luyện nhanh hơn so với các SVM sử dụng kernel phi tuyến.

Một số tham số quan trọng của Linear SVC gồm:

- **C**: hệ số điều chuẩn, kiểm soát sự cân bằng giữa việc tối đa hóa margin và giảm lỗi phân loại;
- **loss**: hàm mất mát được sử dụng trong quá trình tối ưu;
- **penalty**: dạng điều chuẩn, thường là L2;
- **max_iter**: số vòng lặp tối đa trong quá trình huấn luyện;
- **random_state**: giá trị cố định giúp kết quả có thể tái lập.

Ưu điểm của Linear SVC:

- Hoạt động hiệu quả với dữ liệu có số chiều lớn, đặc biệt là dữ liệu văn bản.
- Tốc độ huấn luyện và dự đoán nhanh hơn so với SVM phi tuyến.
- Thường cho kết quả tốt khi dữ liệu có thể phân tách gần tuyến tính.
- Phù hợp với dữ liệu thưa như ma trận TF-IDF.
- Ít bị ảnh hưởng bởi số lượng đặc trưng lớn nếu dữ liệu đã được chuẩn hóa phù hợp.

Nhược điểm của Linear SVC:

- Chỉ xây dựng ranh giới tuyến tính nên có thể kém hiệu quả với dữ liệu có quan hệ phi tuyến phức tạp.
- Không trực tiếp cung cấp xác suất dự đoán như một số mô hình khác.
- Hiệu quả phụ thuộc vào việc lựa chọn tham số C.
- Cần chuyển đổi dữ liệu văn bản thành vector đặc trưng trước khi huấn luyện.

Nhờ khả năng xử lý tốt dữ liệu văn bản nhiều chiều và tốc độ huấn luyện nhanh, Linear SVC là một lựa chọn phù hợp để so sánh với các mô hình phân loại khác.

6 Pipeline học máy

6.1 TF-IDF và LinearSVC

Trong giai đoạn này, mô hình học máy truyền thống được xây dựng nhằm phân loại văn bản đã qua tiền xử lý vào các nhãn chủ đề tương ứng. Dữ liệu đầu vào gồm ba tập riêng biệt: tập huấn luyện, tập validation và tập kiểm thử. Trường văn bản sử dụng cho mô hình là `ml_clean_text`, trong khi nhãn phân loại là `category`. Các giá trị rỗng trong văn bản được thay thế bằng chuỗi rỗng và toàn bộ nhãn được ép kiểu về chuỗi để đảm bảo tính nhất quán khi huấn luyện.

6.1.1 Trích xuất đặc trưng

Sau các bước thử nghiệm, cấu hình TF-IDF cuối cùng được lựa chọn như sau:

- `ngram_range=(1,2)`: sử dụng cả unigram và bigram nhằm khai thác thông tin từ đơn lẻ và cụm hai từ.
- `max_features=300000`: giới hạn số lượng đặc trưng tối đa ở mức 300.000 để cân bằng giữa hiệu quả mô hình và chi phí tính toán.
- `max_df=0.8`: loại bỏ các từ xuất hiện quá phổ biến trong tập dữ liệu, do các từ này thường ít mang ý nghĩa phân biệt.
- `sublinear_tf=True`: áp dụng biến đổi logarit lên tần suất từ nhằm giảm ảnh hưởng của những từ xuất hiện quá nhiều lần trong một văn bản.

Vectorizer chỉ được *fit* trên tập huấn luyện để tránh hiện tượng rò rỉ dữ liệu. Sau đó, cùng một vectorizer được dùng để biến đổi tập validation và tập test. Sau khi trích xuất đặc trưng, kích thước các ma trận TF-IDF thu được là:

- Tập huấn luyện: (103685, 300000)
- Tập validation: (22218, 300000)
- Tập kiểm thử: (22219, 300000)

Điều này cho thấy mỗi văn bản được biểu diễn trong không gian 300.000 chiều, tương ứng với 300.000 đặc trưng TF-IDF được lựa chọn.

6.1.2 Huấn luyện mô hình

Sau các thử nghiệm, cấu hình mô hình cuối cùng như sau:

- `C=0.5`: hệ số điều chuẩn, giúp cân bằng giữa việc tối ưu biên phân tách và hạn chế overfitting.
- `loss="squared_hinge"`: hàm mất mát bình phương hinge, cho kết quả ổn định hơn so với hinge trong các thử nghiệm.
- `class_weight="balanced"`: tự động điều chỉnh trọng số theo tần suất lớp, nhằm giảm ảnh hưởng của mất cân bằng dữ liệu.
- `max_iter=5000`: số vòng lặp tối đa cho quá trình tối ưu.
- `random_state=42`: cố định trạng thái ngẫu nhiên để đảm bảo khả năng tái lập kết quả.

Việc sử dụng `class_weight="balanced"` là cần thiết vì số lượng mẫu giữa các nhãn không đồng đều. Nếu không điều chỉnh trọng số lớp, mô hình có thể thiên lệch về các lớp phổ biến.

6.1.3 Đánh giá mô hình

Mô hình TF-IDF kết hợp LinearSVC được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 được ưu tiên xem xét vì phản ánh hiệu quả trung bình trên từng lớp, phù hợp với bài toán phân loại đa lớp có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.7613
Macro Precision	0.7030
Macro Recall	0.7010
Macro F1	0.6988
Weighted F1	0.7568

Table 9: Kết quả đánh giá tổng quát của mô hình TF-IDF kết hợp LinearSVC trên tập kiểm thử

Bảng 9 cho thấy mô hình đạt Accuracy 76.13% và Weighted F1 75.68%, thể hiện khả năng phân loại tương đối tốt trên toàn bộ tập kiểm thử. Tuy nhiên, Macro F1 đạt 69.88%, thấp hơn Weighted F1, cho thấy hiệu quả giữa các lớp chưa

đồng đều. Nói cách khác, mô hình hoạt động tốt với các lớp có đặc trưng rõ hoặc nhiều mẫu, nhưng suy giảm ở các lớp có nội dung giao thoa.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.5909	0.5866	0.5888	687
BUSINESS	0.6412	0.6641	0.6525	899
COMEDY	0.6329	0.5469	0.5868	810
ENTERTAINMENT	0.8120	0.7693	0.7901	2605
FOOD & DRINK	0.7540	0.8349	0.7924	951
HEALTHY LIVING	0.4650	0.2849	0.3533	1004
HOME & LIVING	0.7862	0.7886	0.7874	648
PARENTING	0.5925	0.7142	0.6476	1319
PARENTS	0.4239	0.3052	0.3549	593
POLITICS	0.8982	0.8757	0.8868	5341
QUEER VOICES	0.8206	0.7878	0.8039	952
SPORTS	0.7848	0.8436	0.8132	761
STYLE & BEAUTY	0.8434	0.8743	0.8586	1472
TRAVEL	0.8011	0.8411	0.8209	1485
WELLNESS	0.6977	0.7983	0.7446	2692

Table 10: Kết quả đánh giá chi tiết theo từng nhãn của mô hình TF-IDF kết hợp LinearSVC

Kết quả theo từng nhãn trong Bảng 10 cho thấy mô hình phân loại tốt nhất ở các lớp có ranh giới từ vựng rõ, gồm **POLITICS**, **STYLE & BEAUTY**, **TRAVEL**, **SPORTS** và **QUEER VOICES**. Các lớp này đều đạt F1-score trên 0.80, trong đó **POLITICS** cao nhất với 0.8868. Điều này cho thấy biểu diễn TF-IDF đặc biệt hiệu quả khi chủ đề có hệ từ khóa đặc trưng và ít giao thoa với các nhóm khác.

Ở chiều ngược lại, **HEALTHY LIVING** và **PARENTS** là hai lớp yếu nhất, với F1-score lần lượt 0.3533 và 0.3549. Điểm đáng chú ý là recall của hai lớp này rất thấp, chỉ 0.2849 và 0.3052, cho thấy nhiều mẫu thật thuộc hai lớp này không được nhận diện đúng. Đây là nguyên nhân chính kéo Macro F1 xuống thấp hơn Weighted F1.

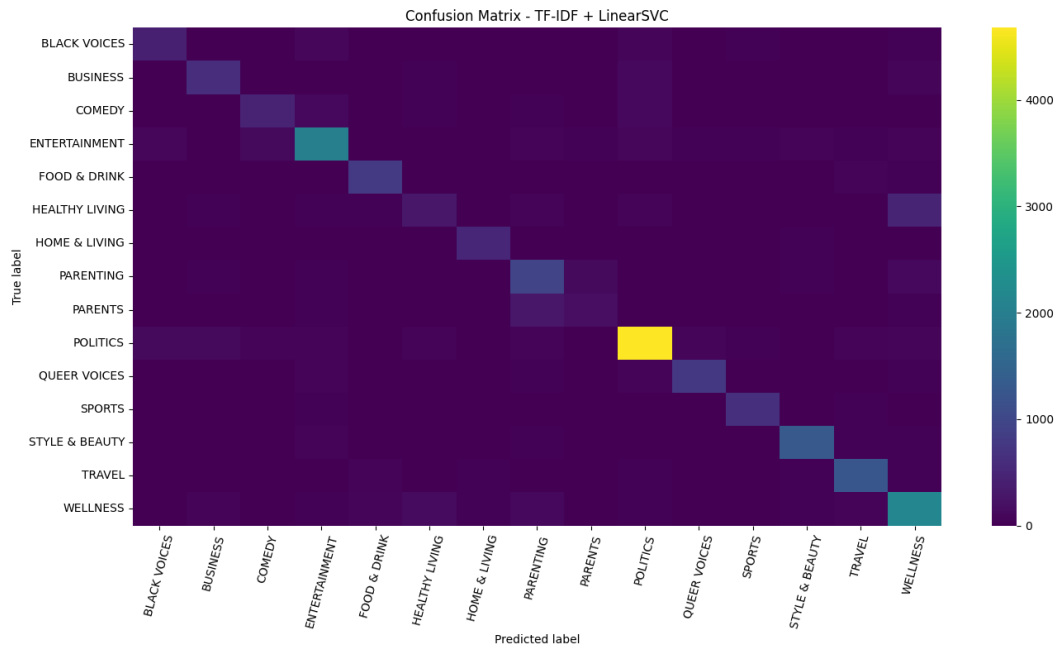


Figure 22: Ma trận nhầm lẫn của mô hình TF-IDF kết hợp LinearSVC trên tập kiểm thử

Ma trận nhầm lẫn ở Hình 22 củng cố các nhận xét trên. Phần lớn giá trị lớn nằm trên đường chéo chính, đặc biệt ở các lớp như **POLITICS**, **ENTERTAINMENT**, **STYLE & BEAUTY**, **TRAVEL** và **WELLNESS**, cho thấy mô hình dự đoán đúng phần lớn mẫu ở các nhóm này. Tuy nhiên, lỗi nhầm lẫn xuất hiện rõ ở các nhãn có ngữ nghĩa gần nhau như **HEALTHY LIVING**, **PARENTS**, **PARENTING** và **WELLNESS**. Các lớp này cùng liên quan đến sức khỏe, đời sống cá nhân, gia đình và nuôi dạy con cái nên thường chia sẻ nhiều từ vựng, khiến TF-IDF khó tách biệt hoàn toàn.

Tổng kết lại, mô hình TF-IDF kết hợp LinearSVC là một mô hình cơ sở hiệu quả, đạt kết quả tốt với các chủ đề có đặc trưng ngôn ngữ rõ ràng. Hạn chế chính của mô hình nằm ở khả năng phân biệt các lớp có nội dung giao thoa. Do đó, các hướng cải thiện tiếp theo nên tập trung vào bổ sung đặc trưng ngữ nghĩa, xử lý mất cân bằng dữ liệu và thử nghiệm các mô hình biểu diễn văn bản mạnh hơn.

6.1.4 Các thử nghiệm đã thực hiện

Để lựa chọn cấu hình phù hợp, nhiều nhóm thử nghiệm đã được thực hiện trên tập validation.

Thử nghiệm n-gram. Ba cấu hình n-gram được so sánh gồm unigram, unigram kết hợp bigram, và unigram kết hợp bigram trigram. Kết quả cho thấy cấu hình `ngram_range=(1,2)` đạt Macro F1 tốt nhất trên tập validation, với giá

trị 0.6901. Khi bổ sung trigram, số lượng đặc trưng tăng mạnh từ khoảng 1.11 triệu lên hơn 2.63 triệu, nhưng Macro F1 lại giảm xuống 0.6805. Điều này cho thấy các cụm 3 từ xuất hiện quá thừa và có thể làm tăng nhiễu trong không gian đặc trưng.

Cấu hình	Số đặc trưng	Accuracy	Macro F1
Unigram	54,811	0.7242	0.6657
Unigram + Bigram	1,113,497	0.7576	0.6901
Unigram + Bigram + Trigram	2,632,388	0.7538	0.6805

Table 11: Kết quả thử nghiệm các cấu hình n-gram trên tập validation

Thử nghiệm số lượng đặc trưng TFIDF. Các giá trị maxfeatures được thử nghiệm từ 10.000 đến 1.113.497. Kết quả cho thấy khi số lượng đặc trưng tăng, hiệu quả mô hình có xu hướng cải thiện do mô hình có thể biểu diễn được nhiều từ khóa và ngữ cảnh hơn trong tập dữ liệu. Đặc biệt, khi tăng từ các mức thấp như 10.000 hoặc 50.000 lên khoảng 300.000 đặc trưng, Macro F1 tăng tương đối rõ rệt vì nhiều từ mang tính phân biệt giữa các lớp được giữ lại.

Tuy nhiên, khi số đặc trưng vượt quá 300.000, mức cải thiện bắt đầu giảm dần. Nguyên nhân là phần lớn các đặc trưng được bổ sung ở giai đoạn này là các từ hoặc cụm từ hiếm, xuất hiện với tần suất thấp và đóng góp không đáng kể vào khả năng phân loại. Đồng thời, không gian đặc trưng quá lớn cũng làm ma trận TF-IDF trở nên rất thưa (sparse), dẫn đến tăng chi phí lưu trữ, thời gian huấn luyện và nguy cơ mô hình học nhiễu từ dữ liệu.

Cấu hình 1.000.000 đặc trưng đạt Macro F1 cao nhất trong nhóm thử nghiệm này, khoảng 0.6901, cho thấy việc mở rộng không gian đặc trưng vẫn mang lại thêm thông tin hữu ích. Tuy nhiên, mức cải thiện so với 300.000 đặc trưng là không đáng kể trong khi chi phí tính toán và bộ nhớ tăng lên đáng kể. Vì vậy, mức 300.000 đặc trưng được lựa chọn như một điểm cân bằng hợp lý giữa hiệu quả mô hình và tài nguyên tính toán.

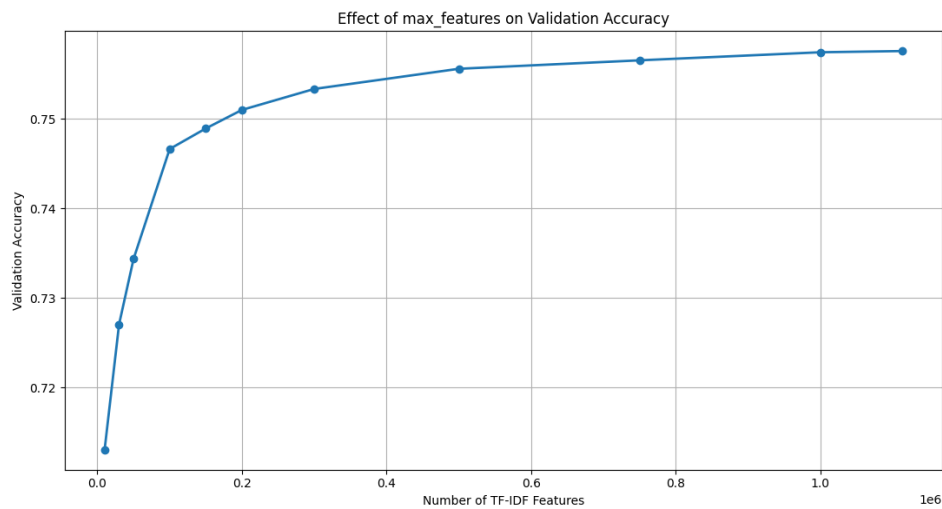


Figure 23: Ảnh hưởng của số lượng đặc trưng TF-IDF đến Accuracy trên tập validation

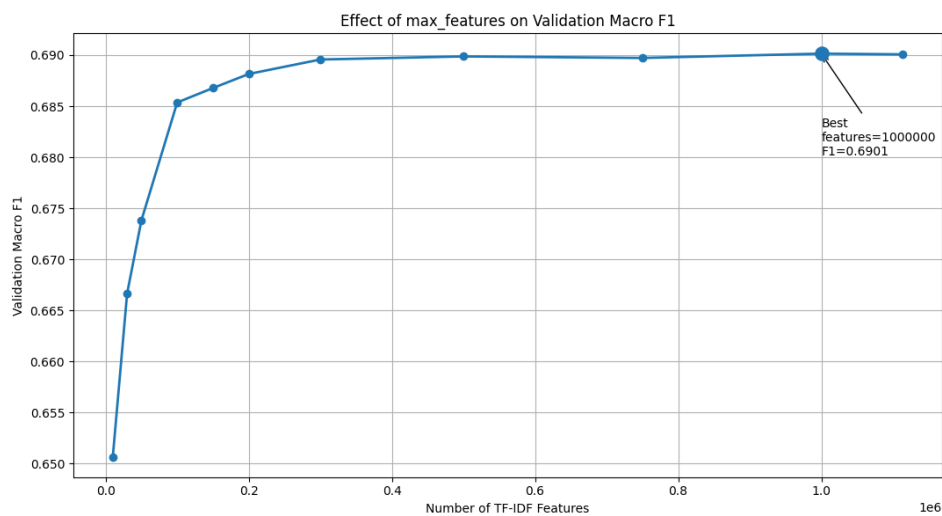


Figure 24: Ảnh hưởng của số lượng đặc trưng TF-IDF đến Macro F1 trên tập validation

Thử nghiệm tham số của LinearSVC. Các giá trị C được thử gồm 0.01, 0.1, 0.5, 1.0, 2.0, 5.0 và 10.0, kết hợp với hai hàm mất mát `hinge` và `squared_hinge`. Kết quả tốt nhất trên validation đạt được với $C=0.5$ và `loss="squared_hinge"`, với Accuracy bằng 0.7542 và Macro F1 bằng 0.6909. Các giá trị C quá nhỏ làm mô hình bị underfitting, trong khi các giá trị quá lớn không cải thiện kết quả và có xu hướng làm giảm Macro F1.

C	Loss	Accuracy	Macro F1
0.5	squared_hinge	0.7542	0.6909
1.0	squared_hinge	0.7534	0.6896
2.0	squared_hinge	0.7507	0.6849
1.0	hinge	0.7467	0.6846
0.1	squared_hinge	0.7430	0.6792

Table 12: Một số kết quả tốt nhất khi tinh chỉnh tham số LinearSVC trên tập validation

Nhìn chung, các cấu hình sử dụng `squared_hinge` cho hiệu quả tốt hơn so với `hinge`, thể hiện qua cả Accuracy và Macro F1. Điều này cho thấy hàm mất mát `squared_hinge` phù hợp hơn với bài toán phân loại văn bản trong thí nghiệm này.

Cấu hình tốt nhất là `C = 0.5` kết hợp với `loss = squared_hinge`, đạt Accuracy 0.7542 và Macro F1 0.6909. Khi tăng `C` lên 1.0 hoặc 2.0, hiệu quả không cải thiện mà giảm nhẹ, cho thấy mô hình bắt đầu nhạy hơn với dữ liệu huấn luyện và không mang lại lợi ích rõ rệt trên tập validation. Ngược lại, khi `C` quá nhỏ như 0.1, mô hình bị điều chuẩn mạnh hơn, làm giảm khả năng học các đặc trưng phân biệt giữa các lớp.

6.2 BERT/RoBERTa Embedding và LinearSVC

6.2.1 Tiền xử lý dữ liệu

Khác với mô hình TF-IDF, mô hình BERT/RoBERTa không cần tiền xử lý văn bản quá mạnh. Do RoBERTa đã được tiền huấn luyện trên văn bản tự nhiên và sử dụng tokenizer dạng subword, mô hình cần giữ lại ngữ cảnh, cấu trúc câu và các từ chức năng để tạo embedding chính xác.

Quy trình tiền xử lý cho BERT/RoBERTa nhìn chung tương tự phần trước, nhưng không loại bỏ stopwords và không thực hiện lemmatization. Các stopwords như *not*, *with*, *about* tuy ít quan trọng trong TF-IDF nhưng vẫn góp phần thể hiện quan hệ ngữ nghĩa trong câu. Ngoài ra, tokenizer subword của RoBERTa đã có khả năng xử lý các biến thể từ vựng, nên lemmatization thủ công là không cần thiết..

6.2.2 Trích xuất đặc trưng

Sau khi tiền xử lý, văn bản được đưa vào tokenizer của `roberta-base`. Tokenizer có nhiệm vụ chuyển văn bản thành các token phù hợp với đầu vào của mô hình RoBERTa. Các cấu hình chính được sử dụng gồm:

- `max_length=128`: giới hạn độ dài tối đa của văn bản là 128 token.

- `padding=True`: thêm padding để các văn bản trong cùng một batch có cùng độ dài.
- `truncation=True`: cắt bớt văn bản nếu vượt quá độ dài tối đa.
- `batch_size=32`: xử lý 32 văn bản trong mỗi batch.

Sau khi đi qua mô hình **roberta-base**, mỗi token trong văn bản được biểu diễn bằng một vector embedding. Tuy nhiên, bài toán phân loại cần một vector duy nhất đại diện cho toàn bộ văn bản. Vì vậy, nhóm sử dụng phương pháp *mean pooling*, tức là lấy trung bình embedding của các token hợp lệ để tạo ra vector biểu diễn cho cả văn bản.

Trong quá trình mean pooling, **attention_mask** được sử dụng để phân biệt token thật và token padding. Chỉ các token thật mới được đưa vào phép tính trung bình, còn các token padding bị bỏ qua. Cách làm này giúp vector biểu diễn văn bản không bị ảnh hưởng bởi các giá trị padding được thêm vào khi chuẩn hóa độ dài câu.

Sau bước trích xuất đặc trưng, mỗi văn bản được biểu diễn bởi một vector có 768 chiều. Kích thước các tập embedding thu được là:

- Train: (103683, 768)
- Validation: (22218, 768)
- Test: (22218, 768)

Các vector embedding sau đó được chuẩn hóa bằng **StandardScaler**. Bộ chuẩn hóa chỉ được *fit* trên tập train, sau đó dùng *transform* cho tập validation và test. Cách làm này giúp tránh rò rỉ dữ liệu, vì thông tin thống kê của validation và test không được đưa vào quá trình huấn luyện.

6.2.3 Huấn luyện mô hình

Sau khi có vector embedding, nhóm sử dụng mô hình **LinearSVC** để phân loại văn bản. LinearSVC là mô hình SVM tuyến tính, phù hợp với dữ liệu dạng vector số có kích thước cố định. Trong trường hợp này, mỗi văn bản được biểu diễn bằng một vector embedding 768 chiều, nên LinearSVC có thể học ranh giới phân tách giữa các lớp dựa trên các đặc trưng ngữ nghĩa đã được RoBERTa trích xuất.

Cấu hình cuối cùng của mô hình gồm:

- `C=0.01`: tham số điều chuẩn được chọn sau quá trình thử nghiệm.
- `loss="squared_hinge"`: hàm mất mát dùng cho mô hình SVM tuyến tính.

- `class_weight="balanced"`: cân bằng trọng số giữa các lớp.
- `max_iter=5000`: số vòng lặp tối đa cho quá trình tối ưu.
- `dual=False`: phù hợp khi số lượng mẫu lớn hơn số chiều đặc trưng.
- `random_state=42`: đảm bảo khả năng tái lập kết quả.

Tham số `class_weight="balanced"` được sử dụng vì dữ liệu có hiện tượng mất cân bằng lớp.

6.2.4 Đánh giá mô hình

Tương tự pipeline TF-IDF kết hợp LinearSVC, mô hình RoBERTa Embedding kết hợp LinearSVC được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 tiếp tục được xem là chỉ số quan trọng do bài toán có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.7686
Macro Precision	0.7057
Macro Recall	0.7264
Macro F1	0.7138
Weighted F1	0.7691

Table 13: Kết quả đánh giá tổng quát của mô hình RoBERTa Embedding kết hợp LinearSVC trên tập kiểm thử

Bảng 13 cho thấy mô hình đạt Accuracy 76.86%, Weighted F1 76.91% và Macro F1 71.38%. So với pipeline TF-IDF kết hợp LinearSVC, mô hình sử dụng RoBERTa Embedding cho kết quả Macro F1 cao hơn, cho thấy biểu diễn ngữ nghĩa từ mô hình Transformer giúp cải thiện khả năng phân loại cân bằng giữa các lớp.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.5086	0.5590	0.5326	687
BUSINESS	0.6000	0.6915	0.6425	898
COMEDY	0.5173	0.5716	0.5431	810
ENTERTAINMENT	0.8207	0.7628	0.7907	2605
FOOD & DRINK	0.7848	0.8707	0.8255	951
HEALTHY LIVING	0.6186	0.4442	0.5171	1004
HOME & LIVING	0.7633	0.8210	0.7911	648
PARENTING	0.6578	0.7271	0.6907	1319
PARENTS	0.4753	0.4697	0.4725	594
POLITICS	0.9174	0.8485	0.8816	5340
QUEER VOICES	0.7146	0.7311	0.7227	952
SPORTS	0.7573	0.8896	0.8181	761
STYLE & BEAUTY	0.8549	0.8567	0.8558	1472
TRAVEL	0.8203	0.8579	0.8387	1485
WELLNESS	0.7753	0.7946	0.7848	2692

Table 14: Kết quả đánh giá chi tiết theo từng nhãn của mô hình RoBERTa Embedding kết hợp LinearSVC

Kết quả trong Bảng 14 cho thấy mô hình vẫn đạt hiệu quả tốt ở các lớp có đặc trưng nội dung rõ ràng như **POLITICS**, **STYLE & BEAUTY**, **TRAVEL**, **FOOD & DRINK** và **SPORTS**. Đây cũng là nhóm nhãn đã cho kết quả tốt ở pipeline TF-IDF kết hợp LinearSVC.

Điểm cải thiện đáng chú ý nằm ở các lớp khó như **HEALTHY LIVING** và **PARENTS**. Với RoBERTa Embedding, F1-score của **HEALTHY LIVING** đạt 0.5171 và **PARENTS** đạt 0.4725, cao hơn đáng kể so với pipeline TF-IDF. Điều này cho thấy embedding từ RoBERTa hỗ trợ mô hình nắm bắt thông tin ngữ nghĩa tốt hơn, đặc biệt với các lớp có nội dung giao thoa.

Tuy nhiên, một số lớp như **BLACK VOICES**, **COMEDY** và **PARENTS** vẫn có F1-score thấp hơn so với các lớp còn lại. Tương tự pipeline TF-IDF kết hợp LinearSVC, nguyên nhân chủ yếu đến từ sự giao thoa nội dung giữa các nhãn, ranh giới chủ đề chưa thật sự rõ ràng và sự chênh lệch về số lượng mẫu giữa các lớp.

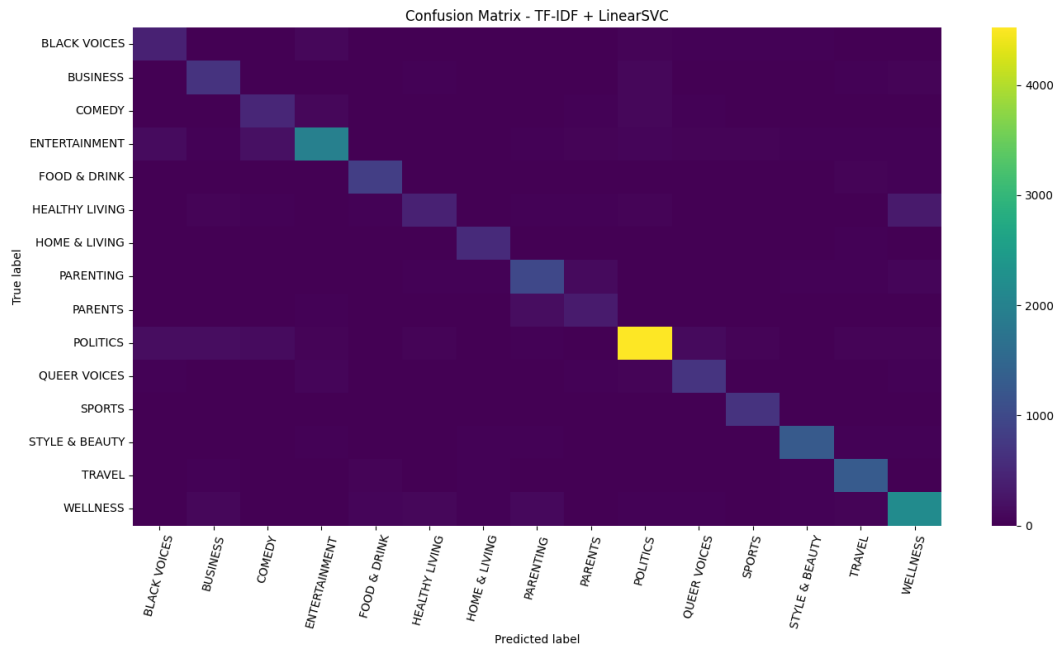


Figure 25: Ma trận nhầm lẫn của mô hình RoBERTa Embedding kết hợp LinearSVC trên tập kiểm thử

Ma trận nhầm lẫn ở Hình 25 cho thấy phần lớn dự đoán đúng tập trung trên đường chéo chính, đặc biệt ở các lớp có số lượng mẫu lớn hoặc đặc trưng nội dung rõ như POLITICS, ENTERTAINMENT, WELLNESS, STYLE & BEAUTY và TRAVEL. Các lỗi nhầm lẫn vẫn xuất hiện ở những nhóm nhãn gần nghĩa như HEALTHY LIVING, WELLNESS, PARENTING và PARENTS, tương tự hiện tượng đã quan sát ở pipeline TF-IDF kết hợp LinearSVC.

Tổng kết lại, RoBERTa Embedding kết hợp LinearSVC cho kết quả tốt hơn TF-IDF kết hợp LinearSVC, đặc biệt ở chỉ số Macro F1. Điều này cho thấy việc sử dụng embedding ngữ nghĩa giúp cải thiện khả năng phân loại trên các lớp khó. Tuy vậy, do bộ phân loại cuối vẫn là LinearSVC, mô hình vẫn còn hạn chế khi xử lý các nhãn có nội dung giao thoa mạnh.

6.2.5 Thử nghiệm

Trong phần thử nghiệm, nhóm giữ cố định phương pháp trích xuất đặc trưng bằng roberta-base và mean pooling, sau đó thử nghiệm các giá trị khác nhau của tham số C trong LinearSVC. Các giá trị được thử gồm 0.01, 0.1, 1.0, 5.0 và 10.0.

C	Accuracy	Macro F1	Weighted F1
0.01	0.7692	0.7152	0.7692
0.1	0.7691	0.7151	0.7692
1.0	0.7689	0.7151	0.7691
5.0	0.7688	0.7150	0.7690
10.0	0.7688	0.7150	0.7690

Table 15: Kết quả thử nghiệm tham số C của LinearSVC trên tập validation

Kết quả thử nghiệm cho thấy các giá trị C cho hiệu quả rất gần nhau, chứng tỏ embedding trích xuất từ **roberta-base** đã biểu diễn văn bản khá ổn định và có khả năng phân tách lớp tương đối tốt. Khi thay đổi C, mô hình LinearSVC chỉ điều chỉnh mức độ phạt lỗi phân loại, nên mức chênh lệch giữa các cấu hình không quá lớn.

Trong các giá trị được thử nghiệm, $C=0.01$ đạt Macro F1 cao nhất trên tập validation. Giá trị C nhỏ giúp mô hình được điều chuẩn mạnh hơn, hạn chế việc học quá sát tập train và giảm nguy cơ overfitting. Điều này phù hợp với trường hợp sử dụng embedding từ RoBERTa, vì vector đầu vào đã chứa nhiều thông tin ngữ nghĩa, nên bộ phân loại phía sau không cần quá phức tạp.

6.3 Sbert và Catboost

Để đảm bảo tính ổn định trong quá trình huấn luyện, dữ liệu được chuẩn hóa bằng cách thay thế các giá trị thiếu bằng chuỗi rỗng và ép kiểu toàn bộ văn bản sang kiểu **string**. Tương tự, nhãn phân loại cũng được chuyển sang dạng chuỗi để tránh lỗi không đồng nhất dữ liệu.

6.3.1 Biểu diễn văn bản bằng Sentence-BERT

Thay vì sử dụng các phương pháp biểu diễn truyền thống như Bag-of-Words hay TF-IDF, pipeline sử dụng mô hình Sentence-BERT để chuyển đổi văn bản thành vector embedding ngữ nghĩa.

Mô hình được sử dụng là: all-MiniLM-L6-v2

```
from sentence_transformers import SentenceTransformer

sbert_model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)
```

Mỗi văn bản d_i được ánh xạ thành vector embedding $\mathbf{x}_i$:

$$\mathbf{x}_i = \text{SBERT}(d_i)$$

Vector embedding giúp mô hình học được ngữ nghĩa tổng quát của câu, từ đó cải thiện hiệu quả phân loại, đặc biệt đối với các văn bản có cách diễn đạt khác nhau nhưng cùng ý nghĩa.

6.3.2 Huấn luyện mô hình CatBoost

Mô hình sử dụng hàm mất mát:

```
loss_function = MultiClass
```

Đồng thời áp dụng cân bằng lớp:

```
auto_class_weights = Balanced
```

Cấu hình tối ưu của pipeline được lựa chọn dựa trên chỉ số `val_macro_f1` trên tập validation. Trong các bộ tham số đã thử nghiệm, mô hình đạt kết quả tốt nhất khi sử dụng `depth=6`, `learning_rate=0.05` và `l2_leaf_reg=3`, với `val_macro_f1=0.633336`. Do đó, bộ tham số này được chọn làm cấu hình chính thức cho pipeline trong quá trình huấn luyện và đánh giá cuối cùng.

6.3.3 Đánh giá mô hình

Table 16: Kết quả phân loại của mô hình

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.4100	0.5700	0.4800	687
BUSINESS	0.4700	0.6900	0.5600	899
COMEDY	0.3800	0.5300	0.4400	810
ENTERTAINMENT	0.7500	0.6300	0.6900	2605
FOOD & DRINK	0.7100	0.8400	0.7700	951
HEALTHY LIVING	0.4500	0.3900	0.4200	1004
HOME & LIVING	0.6000	0.7200	0.6600	648
PARENTING	0.5900	0.6400	0.6100	1319
PARENTS	0.4000	0.4400	0.4200	593
POLITICS	0.9100	0.7500	0.8200	5341
QUEER VOICES	0.6700	0.7000	0.6800	952
SPORTS	0.6800	0.8200	0.7500	761
STYLE & BEAUTY	0.7700	0.7800	0.7800	1472
TRAVEL	0.7500	0.7600	0.7600	1485
WELLNESS	0.7400	0.6700	0.7000	2692
Accuracy			0.6900	22219
Macro Avg	0.6200	0.6600	0.6400	22219
Weighted Avg	0.7100	0.6900	0.6900	22219

6.3.4 Các thử nghiệm và đánh giá

Để tìm ra cấu hình phù hợp, nhiều bộ siêu tham số được thử nghiệm thủ công. Các tham số chính bao gồm:

$$\theta = \{\text{depth}, \text{learning_rate}, \text{l2_leaf_reg}\}$$

Trong đó:

- **depth**: độ sâu của cây quyết định.
- **learning_rate**: tốc độ cập nhật của boosting.
- **l2_leaf_reg**: hệ số regularization nhằm hạn chế overfitting.

Mỗi mô hình được huấn luyện tối đa 500 vòng lặp. Đồng thời, cơ chế *early stopping* được sử dụng với 50 vòng lặp để dừng huấn luyện khi mô hình không còn cải thiện trên tập validation.

`early_stopping_rounds = 50`

Cơ chế này giúp giảm thời gian huấn luyện và hạn chế hiện tượng quá khớp.

	depth	learning_rate	l2_leaf_reg	val_macro_f1	best_iteration	time_seconds
1	6	0.05	3	0.633336	482	1900.530643
3	8	0.03	5	0.630878	489	3739.206459
2	6	0.03	5	0.620673	499	1901.800241
0	4	0.05	3	0.619164	479	483.272013
4	6	0.01	10	0.581296	496	1868.237814

Dựa trên kết quả thử nghiệm, cấu hình tốt nhất của pipeline là `depth=6`, `learning_rate=0.05` và `l2_leaf_reg=3`, đạt `val_macro_f1=0.633336` trên tập validation. Đây là giá trị cao nhất trong các cấu hình đã thử, cho thấy mô hình có khả năng phân loại cân bằng nhất giữa các lớp. Cấu hình này được lựa chọn vì `depth=6` đủ lớn để học các quan hệ phi tuyến trong dữ liệu nhưng không quá phức tạp, `learning_rate=0.05` giúp mô hình học ổn định, và `l2_leaf_reg=3` cung cấp mức điều chuẩn phù hợp để hạn chế overfitting.

6.4 TF-IDF và CatBoost

6.4.1 Trích xuất đặc trưng

Cấu hình TF-IDF được sử dụng trong thí nghiệm này như sau:

- `ngram_range=(1,2)`: sử dụng cả unigram và bigram nhằm khai thác thông tin từ từ đơn lẻ và cụm hai từ.
- `max_features=300000`: giới hạn số lượng đặc trưng tối đa ở mức 300.000 để cân bằng giữa khả năng biểu diễn văn bản và chi phí tính toán.
- `max_df=0.8`: loại bỏ các từ xuất hiện quá phổ biến trong tập dữ liệu, do các từ này thường ít mang ý nghĩa phân biệt giữa các lớp.
- `sublinear_tf=True`: áp dụng biến đổi logarit lên tần suất từ nhằm giảm ảnh hưởng của các từ xuất hiện quá nhiều lần trong một văn bản.

Vectorizer chỉ được *fit* trên tập huấn luyện để tránh hiện tượng rò rỉ dữ liệu. Sau đó, cùng một vectorizer được dùng để biến đổi tập validation và tập test. Sau khi trích xuất đặc trưng, kích thước các ma trận TF-IDF thu được là:

- Tập huấn luyện: (103685, 300000)

- Tập validation: (22218, 300000)
- Tập kiểm thử: (22219, 300000)

Điều này cho thấy mỗi văn bản được biểu diễn trong không gian 300.000 chiều, tương ứng với 300.000 đặc trưng TF-IDF được lựa chọn.

6.4.2 Huấn luyện mô hình

Sau khi trích xuất đặc trưng TF-IDF, mô hình `CatBoostClassifier` được huấn luyện trên dữ liệu dạng sparse. Do bài toán là phân loại đa lớp, mô hình sử dụng `loss_function="MultiClass"` và `eval_metric="TotalF1"`. Các tham số chung được thiết lập như sau:

- `iterations=50`: số vòng lặp huấn luyện tối đa.
- `auto_class_weights="Balanced"`: tự động điều chỉnh trọng số lớp nhằm giảm ảnh hưởng của mất cân bằng dữ liệu.
- `early_stopping_rounds=10`: dừng sớm nếu mô hình không cải thiện trên tập validation.
- `random_seed=42`: cố định trạng thái ngẫu nhiên để đảm bảo khả năng tái lập kết quả.
- `thread_count=-1`: sử dụng toàn bộ tài nguyên CPU khả dụng trong quá trình huấn luyện.

Việc sử dụng `auto_class_weights="Balanced"` là cần thiết vì số lượng mẫu giữa các nhãn không đồng đều. Nếu không điều chỉnh trọng số lớp, mô hình có thể thiên lệch về các lớp có số lượng mẫu lớn hơn.

6.4.3 Các thử nghiệm đã thực hiện

Để lựa chọn cấu hình phù hợp, ba nhóm tham số CatBoost được thử nghiệm trên tập validation. Các tham số được thay đổi gồm độ sâu cây, tốc độ học và hệ số điều chuẩn `l2_leaf_reg`.

Depth	Learning rate	L2 leaf reg	Validation Macro F1	Thời gian (giây)
6	0.15	10	0.4478	1423.93
5	0.10	5	0.4100	516.69
4	0.10	5	0.4034	352.34

Table 17: Kết quả thử nghiệm các cấu hình CatBoost trên tập validation

Bảng 17 cho thấy cấu hình `depth=6`, `learning_rate=0.15` và `l2_leaf_reg=10` đạt Macro F1 cao nhất trên tập validation, với giá trị 0.4478. Tuy nhiên, cấu hình này cũng có thời gian huấn luyện lớn nhất, khoảng 1423.93 giây. Điều này phản ánh sự đánh đổi giữa hiệu quả mô hình và chi phí tính toán: cây sâu hơn có khả năng học quan hệ phức tạp hơn trong dữ liệu, nhưng đồng thời làm tăng đáng kể thời gian huấn luyện.

Các cấu hình có độ sâu nhỏ hơn, gồm `depth=4` và `depth=5`, cho thời gian huấn luyện ngắn hơn nhưng Macro F1 thấp hơn. Điều này cho thấy mô hình CatBoost với số vòng lặp giới hạn chưa khai thác hiệu quả toàn bộ không gian đặc trưng TF-IDF có kích thước lớn.

6.4.4 Đánh giá mô hình

Mô hình tốt nhất trên tập validation được sử dụng để dự đoán trên tập kiểm thử độc lập. Các chỉ số đánh giá gồm Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 được ưu tiên xem xét vì phản ánh hiệu quả trung bình trên từng lớp, phù hợp với bài toán phân loại đa lớp có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.4684
Macro Precision	0.5018
Macro Recall	0.4505
Macro F1	0.4571
Weighted F1	0.4900

Table 18: Kết quả đánh giá tổng quát của mô hình TF-IDF kết hợp CatBoost trên tập kiểm thử

Bảng 18 cho thấy mô hình đạt Accuracy 46.84% và Macro F1 45.71% trên tập kiểm thử. Kết quả này thấp hơn đáng kể so với kỳ vọng đối với biểu diễn TF-IDF có số chiều lớn. Mặc dù Macro Precision đạt 50.18%, Macro Recall chỉ đạt 45.05%, cho thấy mô hình bỏ sót khá nhiều mẫu ở nhiều lớp. Weighted F1 đạt khoảng 49.00%, cao hơn Macro F1, phản ánh việc mô hình vẫn hoạt động tốt hơn ở một số lớp có số lượng mẫu lớn hoặc đặc trưng dễ nhận diện.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.42	0.38	0.40	687
BUSINESS	0.46	0.37	0.41	899
COMEDY	0.44	0.36	0.39	810
ENTERTAINMENT	0.24	0.65	0.35	2605
FOOD & DRINK	0.59	0.50	0.54	951
HEALTHY LIVING	0.19	0.18	0.18	1004
HOME & LIVING	0.45	0.54	0.49	648
PARENTING	0.50	0.33	0.40	1319
PARENTS	0.26	0.43	0.33	593
POLITICS	0.89	0.51	0.65	5341
QUEER VOICES	0.82	0.61	0.70	952
SPORTS	0.46	0.48	0.47	761
STYLE & BEAUTY	0.60	0.69	0.64	1472
TRAVEL	0.63	0.41	0.50	1485
WELLNESS	0.58	0.31	0.41	2692

Table 19: Kết quả đánh giá chi tiết theo từng nhãn của mô hình TF-IDF kết hợp CatBoost

Kết quả theo từng nhãn trong Bảng 19 cho thấy mô hình đạt F1-score cao nhất ở hai lớp **QUEER VOICES** và **POLITICS**, lần lượt là 0.70 và 0.65. Đây là các lớp có hệ từ vựng tương đối đặc trưng, giúp mô hình dễ nhận diện hơn so với các nhóm chủ đề có nội dung giao thoa. Ngoài ra, **STYLE & BEAUTY** cũng đạt F1-score 0.64, cho thấy mô hình vẫn khai thác được một phần thông tin từ các đặc trưng TF-IDF ở những chủ đề có từ khóa rõ ràng.

Ở chiều ngược lại, **HEALTHY LIVING** là lớp yếu nhất với F1-score chỉ 0.18. Các lớp như **PARENTS**, **ENTERTAINMENT**, **COMEDY** và **PARENTING** cũng có F1-score thấp, dao động từ 0.33 đến 0.40. Đặc biệt, **ENTERTAINMENT** có recall cao 0.65 nhưng precision chỉ 0.24, cho thấy mô hình có xu hướng dự đoán quá nhiều mẫu vào lớp này, dẫn đến nhiều dự đoán sai.

Nhìn chung, mô hình TF-IDF kết hợp CatBoost chưa đạt hiệu quả cao trong thí nghiệm này. Nguyên nhân có thể đến từ việc CatBoost phải học trên không gian TF-IDF rất thưa và có số chiều lớn, trong khi số vòng lặp huấn luyện chỉ giới hạn ở 50 để kiểm soát thời gian tính toán. Với dạng dữ liệu văn bản được biểu diễn bằng TF-IDF, các mô hình tuyến tính như LinearSVC thường phù hợp hơn do có khả năng xử lý tốt dữ liệu sparse chiều cao. Trong khi đó, CatBoost có thể cần nhiều vòng lặp hơn, chiến lược giảm chiều, chọn lọc đặc trưng hoặc biểu diễn đặc trưng khác để khai thác hiệu quả hơn.

Tổng kết lại, mô hình TF-IDF kết hợp CatBoost cho thấy khả năng phân loại ở mức trung bình thấp trên tập kiểm thử. Mô hình nhận diện tương đối tốt một số lớp có từ khóa đặc trưng như **QUEER VOICES**, **POLITICS** và **STYLE & BEAUTY**,

nhưng gặp khó khăn với các lớp có nội dung giao thoa như **HEALTHY LIVING**, **PARENTS**, **PARENTING** và **WELLNESS**. Do đó, các hướng cải thiện tiếp theo nên tập trung vào tăng số vòng lặp huấn luyện, tinh chỉnh thêm tham số CatBoost, giảm chiều hoặc chọn lọc đặc trưng TF-IDF, cũng như thử nghiệm các biểu diễn văn bản giàu ngữ nghĩa hơn.

6.5 TF-IDF và Gradient Boosting

6.5.1 Pipeline mô hình

Pipeline gồm hai thành phần chính:

1. **TF-IDF** với `TfidfVectorizer(stop_words='english')` để biến đổi văn bản thành vector đặc trưng, đồng thời loại bỏ các từ dừng ít giá trị phân biệt.
2. **Gradient Boosting** với `GradientBoostingClassifier(subsample=0.8, random_state=42)` để học dần từ sai số thông qua nhiều cây quyết định yếu được huấn luyện tuần tự.

Sự kết hợp này được lựa chọn vì TF-IDF là một cách biểu diễn văn bản hiệu quả khi đặc trưng phân biệt nằm nhiều ở từ khóa, còn Gradient Boosting có khả năng học các quan hệ phi tuyến giữa đặc trưng và nhãn. Nói cách khác, TF-IDF giúp làm nổi bật tín hiệu ngôn ngữ, còn Gradient Boosting đảm nhiệm phần học ranh giới phân loại.

6.5.2 Tìm kiếm siêu tham số

Quá trình tinh chỉnh siêu tham số được thực hiện bằng `RandomizedSearchCV` với **10** lần thử. Trong quá trình này, mô hình được đánh giá bằng `PredefinedSplit` trên tập validation thay vì cross-validation truyền thống nhằm giảm thời gian tính toán mà vẫn giữ được một cách đánh giá nhất quán.

Không gian tham số thử nghiệm được trình bày trong Bảng 20.

Tham số	Giá trị thử nghiệm	Ý nghĩa
<code>tfidf__max_features</code>	1000, 3000	Số chiều TF-IDF
<code>tfidf__min_df</code>	5, 10	Ngưỡng loại từ hiếm
<code>clf__n_estimators</code>	50, 100	Số cây boosting
<code>clf__learning_rate</code>	0.1, 0.2	Tốc độ học
<code>clf__max_depth</code>	3, 5	Độ sâu cây

Table 20: Không gian tham số của pipeline TF-IDF kết hợp Gradient Boosting

6.5.3 Bộ tham số tốt nhất

Cấu hình tốt nhất thu được là:

```
{'tfidf__min_df': 5, 'tfidf__max_features': 3000,  
'clf__n_estimators': 100, 'clf__max_depth': 5, 'clf__learning_rate':  
0.2}
```

Kết quả này cho thấy mô hình hoạt động tốt hơn khi được giữ lại nhiều đặc trưng hơn, đồng thời không loại bỏ quá mạnh các từ xuất hiện ít nhưng vẫn mang tính chuyên biệt. Việc tăng số cây và độ sâu cây cũng giúp mô hình học được các ranh giới quyết định phức tạp hơn. Trong khi đó, `learning_rate=0.2` giúp mô hình cập nhật đủ nhanh trong giới hạn 100 cây. Nhìn chung, đây là một cấu hình khá cân bằng giữa khả năng biểu diễn, độ phức tạp mô hình và thời gian huấn luyện.

6.5.4 Kết quả trên tập kiểm thử

Với cấu hình tối ưu, mô hình đạt `Accuracy = 0.6684` trên tập kiểm thử, tức dự đoán đúng khoảng **66.84%** số mẫu. Ngoài `Accuracy`, báo cáo `classification_report` còn cho thấy `Macro Precision = 0.63`, `Macro Recall = 0.57`, `Macro F1 = 0.59` và `Weighted F1 = 0.66`. Với một bài toán gồm khoảng 22 nhãn có mức độ giao thoa nội dung khá cao, đây là kết quả ở mức tương đối tốt.

Chỉ số	Giá trị
Accuracy	0.6684
Macro Precision	0.63
Macro Recall	0.57
Macro F1	0.59
Weighted F1	0.66

Table 21: Các chỉ số đánh giá tổng quát của mô hình TF-IDF kết hợp Gradient Boosting trên tập kiểm thử

6.5.5 Phân tích theo nhãn

Những nhãn được dự đoán tốt nhất là `POLITICS` ($F1 = 0.79$), `STYLE & BEAUTY` (0.76), `QUEER VOICES` (0.73) và `FOOD & DRINK` (0.71). Điểm chung của các lớp này là có hệ từ vựng khá đặc trưng, vì vậy TF-IDF dễ khai thác được tín hiệu phân biệt.

Ở chiều ngược lại, các nhãn có kết quả thấp nhất là `HEALTHY LIVING` (0.26), `PARENTS` (0.28) và `BLACK VOICES` (0.43). Đây là những lớp có nội dung dễ chồng lấn với các chủ đề lân cận hoặc có đặc trưng ngôn ngữ chưa đủ tách biệt.

Nhãn	F1-score
POLITICS	0.79
STYLE & BEAUTY	0.76
QUEER VOICES	0.73
FOOD & DRINK	0.71
BLACK VOICES	0.43
PARENTS	0.28
HEALTHY LIVING	0.26

Table 22: Một số nhãn tiêu biểu trên tập kiểm thử của mô hình TF-IDF kết hợp Gradient Boosting

Sự chênh lệch giữa các nhãn chủ yếu đến từ ba nguyên nhân: mất cân bằng số lượng mẫu, mức độ đặc trưng của từ vựng và sự giao thoa ngữ nghĩa giữa các nhóm chủ đề gần nhau. Bên cạnh đó, Gradient Boosting cổ điển vốn cũng không phải lựa chọn tối ưu nhất cho dữ liệu TF-IDF có tính sparse cao.

6.5.6 Phân tích từ ma trận nhầm lẫn

Hình 26 cho thấy rõ mô hình không nhầm lẫn một cách ngẫu nhiên, mà sai số chủ yếu tập trung ở một vài nhóm nhãn có nội dung gần nhau. Nhờ đó, ma trận nhầm lẫn bổ sung rất tốt cho các chỉ số tổng quát như Accuracy hay F1-score, vì nó cho biết cụ thể mô hình đang nhầm ở đâu.

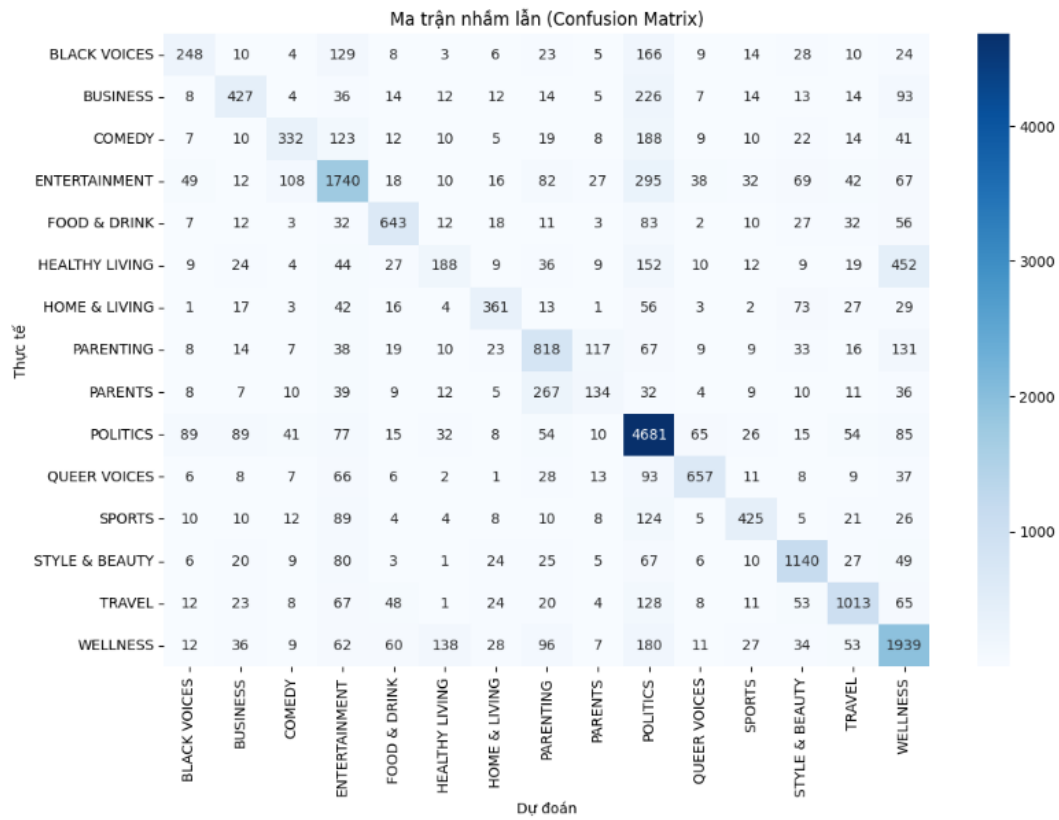


Figure 26: Ma trận nhầm lẫn của mô hình TF-IDF kết hợp Gradient Boosting trên tập kiểm thử

Điểm dễ nhận thấy nhất là mô hình có xu hướng dự đoán nhiều mẫu về nhãn POLITICS. Chẳng hạn, BUSINESS bị nhầm 226 mẫu, COMEDY 188 mẫu, BLACK VOICES 166 mẫu và WELLNESS 180 mẫu sang lớp này. Điều đó cho thấy POLITICS không chỉ là nhãn được nhận diện tốt, mà còn là nhãn mô hình khá dễ nghiêng về khi gặp những văn bản có tín hiệu chưa đủ rõ.

Một số cặp nhãn bị nhầm lẫn nhiều cũng phản ánh đúng bản chất dữ liệu. Cụ thể, PARENTING và PARENTS bị nhầm lẫn hai chiều khá mạnh, với 267 mẫu PARENTS bị dự đoán thành PARENTING và 117 mẫu theo chiều ngược lại. Tương tự, HEALTHY LIVING và WELLNESS cũng chòng lỉnh đáng kể, với 452 mẫu HEALTHY LIVING bị nhầm sang WELLNESS, trong khi chiều ngược lại là 138 mẫu. Cặp COMEDY và ENTERTAINMENT cũng có mức nhầm lẫn đáng chú ý, lần lượt là 123 và 108 mẫu.

Nhìn chung, các lỗi này là điều có thể hiểu được, vì những nhóm chủ đề như gia đình, sức khỏe, lối sống hay giải trí thường chia sẻ nhiều từ khóa và bối cảnh tương tự nhau. Khi chỉ dựa trên TF-IDF, mô hình chủ yếu học từ mức độ xuất hiện của từ và cụm từ, nên vẫn khó tách biệt hoàn toàn các nhãn có ranh giới ngữ nghĩa mờ.

6.5.7 Nhận xét tổng quan

Nhìn chung, pipeline TF-IDF kết hợp Gradient Boosting cho kết quả khá tốt ở những lớp có từ vựng đặc trưng rõ ràng, nhưng hiệu quả chưa đồng đều trên toàn bộ nhãn. Mô hình này phù hợp như một mốc so sánh mạnh trong bài toán phân loại văn bản đa lớp, song vẫn còn hạn chế khi xử lý các lớp có ranh giới ngữ nghĩa mờ. Nếu muốn cải thiện thêm, có thể cân nhắc các mô hình tuyến tính mạnh cho dữ liệu sparse hoặc các phương pháp biểu diễn ngữ nghĩa sâu hơn.

6.6 TF-IDF và thuật toán Random Forest

Trong pipeline này, dữ liệu văn bản sau tiền xử lý được biểu diễn bằng TF-IDF và sau đó được đưa vào mô hình Random Forest để thực hiện phân loại. Để đảm bảo tính ổn định trong quá trình huấn luyện, các giá trị thiếu trong cột `ml_clean_text` được thay thế bằng chuỗi rỗng. Đồng thời, nhãn phân loại `category` được mã hóa thành dạng số nguyên bằng `LabelEncoder` để phù hợp với đầu vào của mô hình học máy.

6.6.1 Biểu diễn văn bản bằng TF-IDF

Các tham số cấu hình của `TfidfVectorizer` được sử dụng như sau:

- `max_features=50000`: giới hạn tối đa 50.000 đặc trưng từ hoặc cặp từ được sử dụng.
- `ngram_range=(1, 2)`: sử dụng đồng thời unigram và bigram trong quá trình trích xuất đặc trưng.
- `min_df=2`: loại bỏ các thuật ngữ xuất hiện trong ít hơn 2 tài liệu.
- `max_df=0.95`: loại bỏ các thuật ngữ xuất hiện trong hơn 95% tổng số tài liệu, giúp giảm ảnh hưởng của các từ quá phổ biến.

Mỗi văn bản d_i được ánh xạ thành vector đặc trưng $\mathbf{x}_i$ theo công thức:

$$\mathbf{x}_i = \text{TF-IDF}(d_i)$$

Biểu diễn TF-IDF giúp mô hình nắm bắt tốt hơn tầm quan trọng của các từ trong từng tài liệu cũng như trong toàn bộ tập dữ liệu, từ đó hỗ trợ quá trình phân loại hiệu quả hơn.

6.6.2 Huấn luyện mô hình Random Forest

Sau khi văn bản được chuyển thành các vector TF-IDF, mô hình `RandomForestClassifier` được sử dụng để thực hiện phân loại. Random Forest là một phương pháp ensemble dựa trên nhiều cây quyết định, trong đó mỗi cây được huấn luyện trên một phần dữ liệu và tập đặc trưng khác nhau. Kết quả cuối cùng được xác định thông qua cơ chế bỏ phiếu của toàn bộ các cây, giúp mô hình có độ ổn định cao hơn so với một cây quyết định đơn lẻ.

Trong giai đoạn huấn luyện ban đầu, mô hình Random Forest được sử dụng với cấu hình mặc định, trong đó số lượng cây là `n_estimators=100`. Sau đó, để cải thiện hiệu quả phân loại, mô hình tiếp tục được tinh chỉnh siêu tham số thông qua quá trình tìm kiếm ngẫu nhiên.

6.6.3 Điều chỉnh siêu tham số

Để tối ưu hiệu suất của mô hình, `RandomizedSearchCV` được sử dụng nhằm tìm kiếm bộ siêu tham số phù hợp cho `RandomForestClassifier`. Quá trình tìm kiếm được thực hiện trên 10 tổ hợp tham số ngẫu nhiên (`n_iter=10`) với 3-fold cross-validation (`cv=3`) trên tập huấn luyện, sử dụng `accuracy` làm tiêu chí đánh giá.

Không gian tìm kiếm siêu tham số gồm:

- `n_estimators`: số lượng cây trong rừng, với các giá trị thử nghiệm là 50, 100, 200.
- `max_features`: số lượng đặc trưng tối đa được xem xét khi chia một nút, với các giá trị `sqrt` và `log2`.
- `max_depth`: độ sâu tối đa của cây, với các giá trị 10, 20 và `None`.
- `min_samples_split`: số lượng mẫu tối thiểu cần thiết để chia một nút nội bộ, với các giá trị 2 và 5.
- `min_samples_leaf`: số lượng mẫu tối thiểu tại một nút lá, với các giá trị 1 và 2.
- `bootstrap`: lựa chọn có hoặc không sử dụng lấy mẫu bootstrap, với các giá trị `True` và `False`.

Sau quá trình điều chỉnh, bộ siêu tham số tốt nhất được tìm thấy là:

```
{ 'n_estimators': 200, 'min_samples_split': 5, 'min_samples_leaf':  
                                  1,  
  'max_features': 'log2', 'max_depth': None, 'bootstrap': True }
```

Bộ tham số này cho thấy mô hình đạt kết quả tốt hơn khi tăng số lượng cây lên 200, sử dụng chiến lược chọn đặc trưng $\log_2$, cho phép cây phát triển không giới hạn độ sâu, đồng thời duy trì cơ chế bootstrap để tăng tính đa dạng giữa các cây. Mô hình Random Forest với bộ siêu tham số này sau đó được lưu lại để sử dụng trong bước đánh giá cuối cùng.

6.6.4 Đánh giá mô hình

Trên tập test, mô hình đạt **Accuracy = 0.70057**. Kết quả chi tiết được trình bày trong Bảng 23.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.81	0.18	0.29	687
BUSINESS	0.82	0.28	0.42	899
COMEDY	0.72	0.29	0.41	810
ENTERTAINMENT	0.64	0.82	0.72	2605
FOOD & DRINK	0.76	0.79	0.77	951
HEALTHY LIVING	0.64	0.24	0.35	1004
HOME & LIVING	0.91	0.58	0.71	648
PARENTING	0.66	0.52	0.58	1319
PARENTS	0.68	0.20	0.30	593
POLITICS	0.67	0.97	0.80	5341
QUEER VOICES	0.92	0.51	0.66	952
SPORTS	0.85	0.57	0.68	761
STYLE & BEAUTY	0.89	0.78	0.83	1472
TRAVEL	0.82	0.75	0.78	1485
WELLNESS	0.62	0.85	0.71	2692
accuracy		0.70		22219
macro avg	0.76	0.55	0.60	22219
weighted avg	0.72	0.70	0.67	22219

Table 23: Classification report của mô hình TF-IDF kết hợp Random Forest trên tập kiểm thử

Kết quả cho thấy sau khi điều chỉnh siêu tham số, mô hình Random Forest sử dụng đặc trưng TF-IDF đã cải thiện đáng kể hiệu suất. Cụ thể, độ chính xác trên tập test tăng từ 0.6813 lên 0.7006. Điều này cho thấy việc tối ưu siêu tham số có tác động tích cực đến khả năng tổng quát hóa của mô hình.

Xét chi tiết theo từng lớp, mô hình hoạt động tốt ở các nhóm nhãn như POLITICS, STYLE & BEAUTY, TRAVEL, FOOD & DRINK và WELLNESS, với F1-score tương đối cao. Ngược lại, các lớp như BLACK VOICES, COMEDY, PARENTS và HEALTHY LIVING vẫn có recall thấp, cho thấy mô hình còn gặp khó khăn khi phân biệt các

chủ đề có nội dung gần nhau hoặc có tín hiệu đặc trưng chưa thật sự rõ rệt trong không gian TF-IDF.

Nhìn chung, pipeline TF-IDF kết hợp Random Forest cho kết quả khá tốt sau khi tinh chỉnh siêu tham số và là một mốc so sánh có giá trị trong bài toán phân loại văn bản đa lớp. Kết quả này cũng cho thấy dù Random Forest không phải lúc nào cũng là lựa chọn tối ưu nhất cho dữ liệu văn bản thưa, việc lựa chọn cấu hình phù hợp vẫn có thể cải thiện đáng kể hiệu quả của mô hình.

6.7 TF-IDF và thuật toán LightGBM

Trong phần này, nhóm xây dựng pipeline phân loại văn bản bằng cách kết hợp phương pháp TF-IDF với mô hình LightGBM. TF-IDF được sử dụng để chuyển đổi văn bản đã tiền xử lý sang dạng vector số, trong khi LightGBM đảm nhận vai trò mô hình phân loại đa lớp cho nhãn `category`. Pipeline này được thiết kế nhằm đánh giá hiệu quả của một mô hình boosting trên đặc trưng văn bản dạng sparse.

6.7.1 Trích xuất đặc trưng

Dữ liệu đầu vào ban đầu là văn bản tự nhiên, do đó cần được biểu diễn dưới dạng vector số trước khi đưa vào mô hình. Trong pipeline này, nhóm sử dụng `TfidfVectorizer` để trích xuất đặc trưng từ văn bản.

Các siêu tham số chính của TF-IDF được đưa vào quá trình tìm kiếm gồm:

- `max_features`: giới hạn số lượng đặc trưng tối đa được giữ lại.
- `ngram_range`: xác định sử dụng unigram hoặc kết hợp thêm bigram.
- `min_df`: loại bỏ các từ xuất hiện quá ít trong tập dữ liệu.

Sau quá trình tối ưu, cấu hình TF-IDF tốt nhất sử dụng `ngram_range=(1,1)`, tức là chỉ dùng unigram. Điều này cho thấy trong pipeline LightGBM, việc sử dụng từ đơn mang lại hiệu quả tốt hơn so với việc mở rộng thêm bigram trong không gian đặc trưng. Ngoài ra, `max_features=5000` được chọn nhằm giảm số chiều đầu vào, giúp mô hình huấn luyện nhanh hơn và hạn chế nhiễu từ các đặc trưng ít quan trọng.

6.7.2 Huấn luyện mô hình

Sau khi văn bản được vector hóa bằng TF-IDF, mô hình `LGBMClassifier` được sử dụng để thực hiện phân loại đa lớp. LightGBM là mô hình boosting dựa trên cây quyết định, có khả năng học các quan hệ phi tuyến giữa các đặc trưng và nhãn đầu ra.

Pipeline huấn luyện được xây dựng theo dạng:

TF-IDF Vectorizer $\rightarrow$ LightGBM Classifier

Việc sử dụng pipeline giúp gộp bước trích xuất đặc trưng và huấn luyện mô hình trong cùng một quy trình thống nhất. Nhờ đó, quá trình huấn luyện, tối ưu siêu tham số và đánh giá mô hình được thực hiện nhất quán, tránh sai lệch giữa các bước xử lý dữ liệu.

6.7.3 Tối ưu siêu tham số

Để lựa chọn cấu hình phù hợp, nhóm sử dụng `RandomizedSearchCV` kết hợp với 3-fold cross-validation. Quá trình tìm kiếm được thực hiện trên cả tham số của TF-IDF và LightGBM, với 5 bộ tham số được chọn ngẫu nhiên để đánh giá.

Các nhóm tham số được tối ưu gồm:

- **TF-IDF**: `max_features`, `ngram_range`, `min_df`.
- **LightGBM**: `n_estimators`, `learning_rate`, `num_leaves`.

Kết quả cho thấy bộ tham số tốt nhất của pipeline là:

Tham số	Giá trị tốt nhất
<code>tfidf__ngram_range</code>	(1, 1)
<code>tfidf__min_df</code>	2
<code>tfidf__max_features</code>	5000
<code>model__num_leaves</code>	31
<code>model__n_estimators</code>	200
<code>model__learning_rate</code>	0.1

Table 24: Bộ tham số tốt nhất của pipeline TF-IDF kết hợp LightGBM

Bộ tham số này được lựa chọn vì cho kết quả tốt nhất trong quá trình cross-validation. Trong đó, `max_features=5000` giúp giữ lại các đặc trưng quan trọng nhất và giảm chi phí tính toán. Tham số `num_leaves=31` kiểm soát độ phức tạp của các cây trong LightGBM, còn `learning_rate=0.1` và `n_estimators=200` tạo sự cân bằng giữa tốc độ học và khả năng biểu diễn của mô hình.

6.7.4 Đánh giá mô hình

Sau khi chọn được bộ tham số tốt nhất, mô hình được đánh giá trên tập kiểm thử. Tương tự các pipeline trước, các chỉ số được sử dụng gồm Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn.

Kết quả thực nghiệm cho thấy mô hình đạt Accuracy khoảng 72.9% trên tập test. Weighted F1-score đạt khoảng 0.72, cho thấy pipeline TF-IDF kết hợp LightGBM có khả năng phân loại tương đối ổn định trên bài toán đa lớp.

...	precision	recall	f1-score	support
0	0.61	0.41	0.49	687
1	0.65	0.56	0.60	899
2	0.62	0.45	0.52	810
3	0.70	0.77	0.73	2605
4	0.76	0.76	0.76	951
5	0.47	0.28	0.35	1004
6	0.78	0.67	0.72	648
7	0.59	0.72	0.65	1319
8	0.49	0.28	0.36	593
9	0.83	0.90	0.86	5341
10	0.82	0.72	0.77	952
11	0.77	0.66	0.71	761
12	0.81	0.84	0.83	1472
13	0.78	0.78	0.78	1485
14	0.66	0.79	0.72	2692
accuracy			0.73	22219
macro avg	0.69	0.64	0.66	22219
weighted avg	0.72	0.73	0.72	22219

Figure 27: Classification report của mô hình TF-IDF kết hợp LightGBM trên tập kiểm thử

Từ classification report trong Hình 27, có thể thấy mô hình đạt kết quả tốt hơn ở các lớp có số lượng mẫu lớn hoặc đặc trưng từ vựng rõ ràng. Đây là hiện tượng tương tự các pipeline trước, khi các nhãn có nội dung đặc thù thường dễ được mô hình nhận diện hơn.

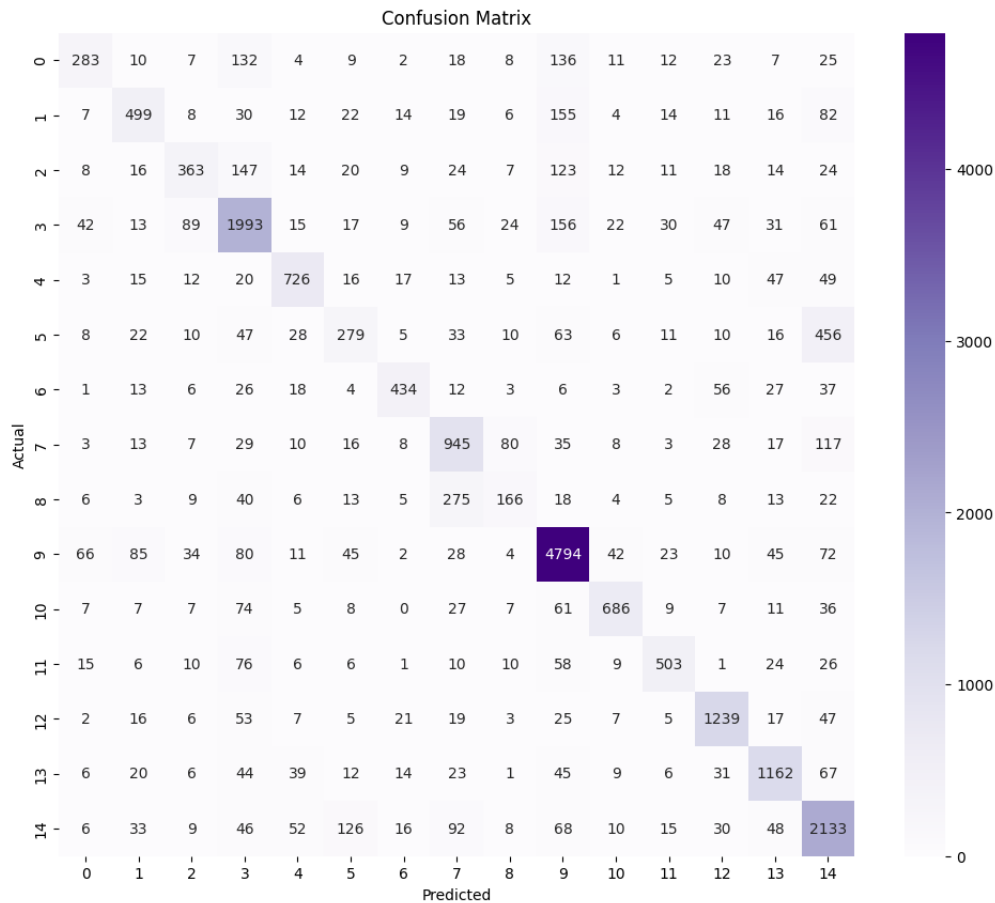


Figure 28: Ma trận nhầm lẫn của mô hình TF-IDF kết hợp LightGBM trên tập kiểm thử

Ma trận nhầm lẫn ở Hình 28 cho thấy phần lớn dự đoán đúng tập trung trên đường chéo chính. Tuy nhiên, mô hình vẫn xuất hiện nhầm lẫn ở một số cặp nhãn có nội dung gần nhau. Đây cũng là hạn chế chung trong bài toán phân loại tin tức nhiều lớp, đặc biệt khi các chủ đề chia sẻ nhiều từ vựng hoặc có ranh giới ngữ nghĩa không rõ ràng.

Tổng kết lại, pipeline TF-IDF kết hợp LightGBM cho kết quả tương đối tốt với Accuracy khoảng 72.9%. Việc sử dụng `RandomizedSearchCV` giúp lựa chọn được bộ tham số phù hợp cho cả bước vector hóa và mô hình phân loại. Tuy nhiên, do đặc trưng TF-IDF chủ yếu dựa trên tần suất từ, mô hình vẫn gặp khó khăn ở các lớp có nội dung giao thoa mạnh.

6.8 BoW và LinearSVC

Trong thí nghiệm này, mô hình học máy truyền thống được xây dựng để phân loại văn bản đã qua tiền xử lý vào các nhãn chủ đề tương ứng. Dữ liệu đầu vào gồm ba tập riêng biệt: tập huấn luyện, tập validation và tập kiểm thử. Trường văn bản được sử dụng là `ml_clean_text`, trong khi nhãn phân loại là `category`. Các giá trị rỗng trong trường văn bản được thay thế bằng chuỗi rỗng và toàn bộ nhãn được ép kiểu về chuỗi nhằm đảm bảo tính nhất quán trong quá trình huấn luyện và đánh giá.

6.8.1 Trích xuất đặc trưng

Ở mô hình này, văn bản được biểu diễn bằng phương pháp Bag-of-Words thông qua `CountVectorizer`. Khác với TF-IDF, BoW biểu diễn văn bản dựa trên tần suất xuất hiện của từ hoặc cụm từ trong văn bản mà không áp dụng trọng số nghịch đảo tần suất tài liệu. Cấu hình đặc trưng được sử dụng như sau:

- `max_features=250000`: giới hạn số lượng đặc trưng tối đa ở mức 250.000 nhằm cân bằng giữa khả năng biểu diễn văn bản và chi phí tính toán.
- `ngram_range=(1,2)`: sử dụng cả unigram và bigram, giúp mô hình khai thác thông tin từ từng từ riêng lẻ cũng như các cụm hai từ có ý nghĩa.
- `lowercase=False`: không chuyển đổi văn bản về chữ thường trong `CountVectorizer`, do dữ liệu đầu vào đã được tiền xử lý trước đó.

Vectorizer được đặt trong cùng một `Pipeline` với mô hình phân loại. Nhờ đó, quá trình trích xuất đặc trưng chỉ được *fit* trên tập huấn luyện, sau đó cùng một bộ biến đổi được sử dụng cho tập validation và tập kiểm thử. Cách làm này giúp tránh rò rỉ dữ liệu từ tập đánh giá vào quá trình huấn luyện.

6.8.2 Huấn luyện mô hình

Mô hình phân loại được sử dụng là `LinearSVC`. Đây là mô hình SVM tuyến tính, phù hợp với bài toán phân loại văn bản do dữ liệu văn bản sau khi vector hóa thường có số chiều lớn và rất thưa. Các tham số chính được sử dụng trong quá trình thử nghiệm gồm:

- `C`: hệ số điều chuẩn của mô hình, được thử nghiệm trên nhiều giá trị khác nhau để lựa chọn cấu hình tốt nhất.
- `class_weight="balanced"`: tự động điều chỉnh trọng số lớp theo tần suất xuất hiện, nhằm giảm ảnh hưởng của mất cân bằng dữ liệu.

- `max_iter=5000`: số vòng lặp tối đa cho quá trình tối ưu.
- `random_state=42`: cố định trạng thái ngẫu nhiên để đảm bảo khả năng tái lập kết quả.

Sau quá trình thử nghiệm trên tập validation, cấu hình tốt nhất đạt được với $C=0.03$. Cấu hình này cho Macro F1 trên tập validation bằng 0.6904, cao nhất trong các giá trị C được thử nghiệm.

6.8.3 Đánh giá mô hình

Mô hình BoW kết hợp LinearSVC được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 được ưu tiên xem xét vì phản ánh hiệu quả trung bình trên từng lớp, phù hợp với bài toán phân loại đa lớp có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.7600
Macro Precision	0.6973
Macro Recall	0.7047
Macro F1	0.6997
Weighted F1	0.7587

Table 25: Kết quả đánh giá tổng quát của mô hình BoW kết hợp LinearSVC trên tập kiểm thử

Bảng 25 cho thấy mô hình đạt Accuracy khoảng 76.00% và Macro F1 bằng 0.6997 trên tập kiểm thử. Kết quả này cho thấy BoW kết hợp LinearSVC vẫn là một mô hình cơ sở mạnh cho bài toán phân loại văn bản, đặc biệt khi kết hợp unigram và bigram với số lượng đặc trưng lớn. Tuy nhiên, sự chênh lệch giữa Weighted F1 và Macro F1 cho thấy hiệu quả phân loại giữa các lớp vẫn chưa hoàn toàn đồng đều.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.58	0.58	0.58	687
BUSINESS	0.62	0.66	0.64	899
COMEDY	0.58	0.54	0.56	810
ENTERTAINMENT	0.80	0.78	0.79	2605
FOOD & DRINK	0.75	0.83	0.78	951
HEALTHY LIVING	0.43	0.41	0.42	1004
HOME & LIVING	0.79	0.78	0.78	648
PARENTING	0.62	0.67	0.65	1319
PARENTS	0.41	0.37	0.39	593
POLITICS	0.90	0.87	0.88	5341
QUEER VOICES	0.82	0.78	0.80	952
SPORTS	0.75	0.84	0.79	761
STYLE & BEAUTY	0.86	0.87	0.86	1472
TRAVEL	0.82	0.83	0.82	1485
WELLNESS	0.73	0.76	0.75	2692

Table 26: Kết quả đánh giá chi tiết theo từng nhãn của mô hình BoW kết hợp LinearSVC

Kết quả chi tiết trong Bảng 26 cho thấy mô hình phân loại tốt nhất ở các lớp có đặc trưng từ vựng rõ ràng như **POLITICS**, **STYLE & BEAUTY**, **TRAVEL**, **QUEER VOICES** và **SPORTS**. Trong đó, **POLITICS** đạt F1-score 0.88, cao nhất trong toàn bộ các nhãn. Điều này cho thấy biểu diễn BoW vẫn phát huy hiệu quả tốt khi các lớp có hệ từ khóa đặc trưng và ít giao thoa về mặt nội dung.

Ngược lại, các lớp **HEALTHY LIVING**, **PARENTS** và **COMEDY** có F1-score thấp hơn, lần lượt là 0.42, 0.39 và 0.56. Đặc biệt, **PARENTS** và **HEALTHY LIVING** là hai lớp có nội dung dễ giao thoa với **PARENTING** và **WELLNESS**. Vì BoW chủ yếu dựa trên tần suất từ và cụm từ, mô hình có thể gặp khó khăn khi các lớp chia sẻ nhiều từ vựng chung nhưng khác nhau về sắc thái ngữ nghĩa.

6.8.4 Các thử nghiệm đã thực hiện

Để lựa chọn cấu hình phù hợp, nhiều giá trị của tham số **C** trong **LinearSVC** đã được thử nghiệm trên tập validation. Các giá trị được thử gồm 0.01, 0.03, 0.05, 0.1, 0.3, 0.5, 1.0, 2.0, 3.0 và 5.0. Kết quả cho thấy **C=0.03** đạt Macro F1 cao nhất trên tập validation.

C	Validation Macro F1	Thời gian huấn luyện (giây)
0.01	0.6883	16.64
0.03	0.6904	19.46
0.05	0.6885	21.55
0.10	0.6841	25.43
0.30	0.6756	38.52
0.50	0.6689	50.77
1.00	0.6634	77.81
2.00	0.6577	121.26
3.00	0.6538	158.69
5.00	0.6503	229.49

Table 27: Kết quả thử nghiệm tham số C của LinearSVC trên tập validation

Bảng 27 cho thấy khi C tăng từ 0.01 lên 0.03, Macro F1 tăng nhẹ từ 0.6883 lên 0.6904. Tuy nhiên, khi tiếp tục tăng C, hiệu quả mô hình giảm dần. Điều này cho thấy mô hình cần một mức điều chuẩn tương đối mạnh để tránh học quá mức trên không gian đặc trưng BoW có số chiều lớn. Các giá trị C cao như 2.0, 3.0 và 5.0 không những không cải thiện chất lượng dự đoán mà còn làm tăng đáng kể thời gian huấn luyện.

Tổng kết lại, mô hình BoW kết hợp LinearSVC đạt hiệu quả tương đối tốt và có kết quả gần với mô hình TF-IDF kết hợp LinearSVC. Ưu điểm của phương pháp này là đơn giản, dễ triển khai và hoạt động hiệu quả với các chủ đề có từ khóa đặc trưng. Tuy nhiên, hạn chế chính nằm ở việc BoW không biểu diễn được đầy đủ ngữ nghĩa và quan hệ ngữ cảnh giữa các từ. Do đó, mô hình vẫn gặp khó khăn ở các lớp có nội dung giao thoa như HEALTHY LIVING, PARENTS, PARENTING và WELLNESS.

6.9 BoW và Random Forest

Trong giai đoạn này, mô hình học máy truyền thống được xây dựng nhằm phân loại văn bản đã qua tiền xử lý vào các nhãn chủ đề tương ứng. Dữ liệu đầu vào gồm ba tập riêng biệt: tập huấn luyện, tập validation và tập kiểm thử. Trường văn bản sử dụng cho mô hình là `ml_clean_text`, trong khi nhãn phân loại là `category`. Các giá trị rỗng trong văn bản được thay thế bằng chuỗi rỗng và toàn bộ nhãn được ép kiểu về chuỗi để đảm bảo tính nhất quán khi huấn luyện.

6.9.1 Trích xuất đặc trưng

Trong mô hình này, văn bản được biểu diễn bằng phương pháp Bag-of-Words thông qua `CountVectorizer`. Khác với TF-IDF, Bag-of-Words biểu diễn văn bản

dựa trên tần suất xuất hiện của từ hoặc cụm từ trong văn bản. Cách biểu diễn này đơn giản, dễ triển khai và phù hợp để làm mô hình cơ sở cho bài toán phân loại văn bản.

Cấu hình BoW được sử dụng như sau:

- `max_features=50000`: giới hạn số lượng đặc trưng tối đa ở mức 50.000 nhằm giảm kích thước không gian đặc trưng và chi phí tính toán.
- `ngram_range=(1,2)`: sử dụng cả unigram và bigram để khai thác thông tin từ từng từ đơn lẻ cũng như các cụm hai từ liên tiếp.
- `lowercase=False`: không chuyển đổi văn bản về chữ thường trong bước vector hóa vì dữ liệu đã được xử lý trước ở giai đoạn tiền xử lý.

Vectorizer được đặt trong **Pipeline** cùng với mô hình Random Forest. Nhờ đó, quá trình trích xuất đặc trưng và huấn luyện mô hình được thực hiện thống nhất trong cùng một quy trình, hạn chế sai sót khi áp dụng lên tập validation và tập kiểm thử.

6.9.2 Huấn luyện mô hình

Mô hình phân loại được sử dụng là `RandomForestClassifier`. Đây là mô hình ensemble dựa trên nhiều cây quyết định, trong đó mỗi cây được huấn luyện trên các mẫu và đặc trưng khác nhau. Dự đoán cuối cùng được tổng hợp từ kết quả của nhiều cây, giúp mô hình ổn định hơn so với một cây quyết định đơn lẻ.

Các tham số chính được sử dụng trong quá trình thử nghiệm gồm:

- `n_estimators`: số lượng cây trong rừng, được thử với các giá trị 50, 100, 150 và 200.
- `max_depth`: độ sâu tối đa của mỗi cây, được thử với các giá trị 20, 30 và 50.
- `min_samples_split`: số mẫu tối thiểu cần có để tiếp tục tách một nút, được thử với các giá trị 2 và 5.
- `class_weight="balanced"`: tự động điều chỉnh trọng số lớp theo tần suất xuất hiện của từng nhãn, nhằm giảm ảnh hưởng của mất cân bằng dữ liệu.
- `random_state=42`: cố định trạng thái ngẫu nhiên để đảm bảo khả năng tái lập kết quả.
- `n_jobs=-1`: sử dụng toàn bộ số nhân CPU khả dụng để tăng tốc quá trình huấn luyện.

Sau quá trình thử nghiệm trên tập validation, cấu hình tốt nhất thu được là:

- `n_estimators=200`
- `max_depth=50`
- `min_samples_split=2`
- `class_weight="balanced"`

Cấu hình này đạt Macro F1 cao nhất trên tập validation, bằng 0.5646.

6.9.3 Đánh giá mô hình

Mô hình BoW kết hợp Random Forest được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 tiếp tục được ưu tiên xem xét vì phản ánh hiệu quả trung bình trên từng lớp, phù hợp với bài toán phân loại đa lớp có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.6200
Macro Precision	0.5600
Macro Recall	0.5900
Macro F1	0.5691
Weighted F1	0.6200

Table 28: Kết quả đánh giá tổng quát của mô hình BoW kết hợp Random Forest trên tập kiểm thử

Bảng 28 cho thấy mô hình đạt Accuracy khoảng 62% và Macro F1 bằng 0.5691 trên tập kiểm thử. Kết quả này cho thấy mô hình có khả năng học được một phần thông tin phân biệt giữa các chủ đề từ biểu diễn BoW. Tuy nhiên, hiệu quả tổng thể còn hạn chế so với các mô hình tuyến tính thường dùng cho dữ liệu văn bản thưa. Nguyên nhân có thể đến từ việc Random Forest không khai thác tốt không gian đặc trưng có số chiều lớn và rất thưa của Bag-of-Words.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.42	0.47	0.45	687
BUSINESS	0.48	0.53	0.50	899
COMEDY	0.56	0.44	0.50	810
ENTERTAINMENT	0.54	0.55	0.54	2605
FOOD & DRINK	0.60	0.78	0.68	951
HEALTHY LIVING	0.23	0.32	0.26	1004
HOME & LIVING	0.56	0.70	0.62	648
PARENTING	0.50	0.60	0.54	1319
PARENTS	0.33	0.27	0.29	593
POLITICS	0.88	0.71	0.78	5341
QUEER VOICES	0.72	0.71	0.72	952
SPORTS	0.59	0.69	0.64	761
STYLE & BEAUTY	0.68	0.80	0.74	1472
TRAVEL	0.67	0.67	0.67	1485
WELLNESS	0.66	0.56	0.60	2692

Table 29: Kết quả đánh giá chi tiết theo từng nhãn của mô hình BoW kết hợp Random Forest

Kết quả theo từng nhãn trong Bảng 29 cho thấy mô hình đạt kết quả tốt nhất ở các lớp như **POLITICS**, **STYLE & BEAUTY**, **QUEER VOICES**, **FOOD & DRINK** và **TRAVEL**. Trong đó, **POLITICS** đạt F1-score cao nhất, bằng 0.78. Đây là các chủ đề có hệ từ vựng tương đối đặc trưng, giúp biểu diễn BoW dễ ghi nhận các tín hiệu phân biệt.

Ở chiều ngược lại, hai lớp có kết quả thấp nhất là **HEALTHY LIVING** và **PARENTS**, với F1-score lần lượt là 0.26 và 0.29. Điều này cho thấy mô hình gặp khó khăn khi phân biệt các nhóm nội dung có mức độ giao thoa cao về từ vựng và ngữ nghĩa. Các nhãn như **HEALTHY LIVING**, **PARENTS**, **PARENTING** và **WELLNESS** đều liên quan đến đời sống, sức khỏe, gia đình hoặc chăm sóc cá nhân, nên nhiều từ khóa có thể xuất hiện đồng thời ở nhiều lớp khác nhau.

6.9.4 Các thử nghiệm đã thực hiện

Để lựa chọn cấu hình phù hợp, quá trình tinh chỉnh tham số được thực hiện trên tập validation với 24 cấu hình khác nhau. Các tham số được thay đổi gồm số lượng cây, độ sâu tối đa của cây và số mẫu tối thiểu để tách nút. Kết quả một số cấu hình tốt nhất được trình bày trong Bảng 30.

<code>n_estimators</code>	<code>max_depth</code>	<code>min_samples_split</code>	Macro F1	Thời gian (s)
200	50	2	0.5646	35.97
150	50	2	0.5630	28.66
200	50	5	0.5622	24.61
150	50	5	0.5617	20.13
100	50	2	0.5599	22.16
100	50	5	0.5596	16.47

Table 30: Một số kết quả tốt nhất khi tinh chỉnh tham số Random Forest trên tập validation

Bảng 30 cho thấy các cấu hình có `max_depth=50` đều đạt kết quả tốt hơn so với các cấu hình có độ sâu 20 hoặc 30. Điều này cho thấy cây cần đủ sâu để học được các tương tác phức tạp giữa các đặc trưng BoW. Khi độ sâu bị giới hạn quá thấp, mô hình có xu hướng underfitting và không biểu diễn tốt ranh giới giữa các lớp.

Việc tăng số lượng cây từ 50 lên 100, 150 và 200 nhìn chung giúp cải thiện Macro F1 trên tập validation. Cấu hình `n_estimators=200` và `max_depth=50` đạt kết quả tốt nhất, tuy nhiên thời gian huấn luyện cũng cao nhất trong nhóm các cấu hình tốt. Điều này thể hiện sự đánh đổi giữa hiệu quả mô hình và chi phí tính toán.

Đối với tham số `min_samples_split`, sự khác biệt giữa hai giá trị 2 và 5 không quá lớn. Tuy nhiên, trong cấu hình tốt nhất, `min_samples_split=2` cho Macro F1 cao hơn một chút so với `min_samples_split=5`. Điều này cho thấy việc cho phép cây tách nút linh hoạt hơn có thể giúp mô hình học được nhiều mẫu phân biệt hơn trong không gian đặc trưng BoW.

Tổng kết lại, mô hình BoW kết hợp Random Forest là một mô hình cơ sở có khả năng phân loại ở mức trung bình. Mô hình hoạt động tương đối tốt với các lớp có từ vựng đặc trưng, nhưng gặp nhiều khó khăn ở các lớp có nội dung giao thoa. Bên cạnh đó, Random Forest không phải lựa chọn tối ưu cho dữ liệu văn bản có không gian đặc trưng lớn và thưa. Do đó, các hướng cải thiện tiếp theo có thể bao gồm thử nghiệm TF-IDF, các mô hình tuyến tính như LinearSVC hoặc Logistic Regression, cũng như các mô hình biểu diễn ngữ nghĩa mạnh hơn.

6.10 BoW và LightGBM

Trong thí nghiệm này, mô hình học máy truyền thống được xây dựng để phân loại văn bản đã qua tiền xử lý vào các nhãn chủ đề tương ứng. Dữ liệu đầu vào gồm ba tập riêng biệt: tập huấn luyện, tập validation và tập kiểm thử. Trường văn bản được sử dụng là `ml_clean_text`, trong khi nhãn phân loại là `category`. Các giá trị rỗng trong trường văn bản được thay thế bằng chuỗi rỗng và toàn bộ nhãn

được ép kiểu về chuỗi nhằm đảm bảo tính nhất quán trong quá trình huấn luyện và đánh giá.

6.10.1 Trích xuất đặc trưng

Ở mô hình này, văn bản được biểu diễn bằng phương pháp Bag-of-Words thông qua `CountVectorizer`. BoW biểu diễn văn bản dựa trên tần suất xuất hiện của từ hoặc cụm từ trong văn bản. Cấu hình đặc trưng được sử dụng như sau:

- `max_features=50000`: giới hạn số lượng đặc trưng tối đa ở mức 50.000 nhằm cân bằng giữa khả năng biểu diễn văn bản và chi phí tính toán.
- `ngram_range=(1,2)`: sử dụng cả unigram và bigram, giúp mô hình khai thác thông tin từ từng từ riêng lẻ cũng như các cụm hai từ có ý nghĩa.
- `lowercase=False`: không chuyển đổi văn bản về chữ thường trong `CountVectorizer`, do dữ liệu đầu vào đã được tiền xử lý trước đó.
- `dtype=np.float32`: sử dụng kiểu dữ liệu số thực 32-bit nhằm giảm chi phí bộ nhớ khi biểu diễn ma trận đặc trưng thưa có số chiều lớn.

Vectorizer được đặt trong cùng một Pipeline với mô hình phân loại. Nhờ đó, quá trình trích xuất đặc trưng chỉ được *fit* trên tập huấn luyện, sau đó cùng một bộ biến đổi được sử dụng cho tập validation và tập kiểm thử. Cách làm này giúp tránh rò rỉ dữ liệu từ tập đánh giá vào quá trình huấn luyện.

6.10.2 Huấn luyện mô hình

Mô hình phân loại được sử dụng là `LightGBM`. Đây là mô hình gradient boosting trên cây quyết định, có khả năng xử lý dữ liệu lớn và thường cho hiệu quả tốt trong nhiều bài toán phân loại. Trong thí nghiệm này, `LightGBM` được huấn luyện trên đặc trưng BoW nhằm đánh giá khả năng kết hợp giữa biểu diễn văn bản truyền thống và mô hình boosting phi tuyến.

Các tham số chính được sử dụng trong quá trình thử nghiệm gồm:

- `boosting_type="gbdt"`: sử dụng Gradient Boosting Decision Tree.
- `n_estimators`: số lượng cây boosting, được thử nghiệm với các giá trị 50, 100, 150 và 200.
- `learning_rate`: tốc độ học của mô hình, được thử nghiệm với các giá trị 0.03, 0.05 và 0.1.

- `num_leaves`: số lá tối đa của mỗi cây, được thử nghiệm với các giá trị 15, 31 và 63.
- `random_state=42`: cố định trạng thái ngẫu nhiên để đảm bảo khả năng tái lập kết quả.
- `n_jobs=-1`: sử dụng toàn bộ tài nguyên CPU khả dụng để tăng tốc quá trình huấn luyện.

Trước khi huấn luyện, nhãn văn bản được mã hóa bằng `LabelEncoder` để phù hợp với đầu vào của mô hình `LightGBM`. Sau quá trình thử nghiệm trên tập validation, cấu hình tốt nhất đạt được với `n_estimators=200`, `learning_rate=0.1` và `num_leaves=63`. Cấu hình này cho Macro F1 trên tập validation bằng 0.6701, cao nhất trong các cấu hình được thử nghiệm.

6.10.3 Đánh giá mô hình

Mô hình BoW kết hợp `LightGBM` được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 được ưu tiên xem xét vì phản ánh hiệu quả trung bình trên từng lớp, phù hợp với bài toán phân loại đa lớp có phân bố nhãn không đồng đều.

Chỉ số	Giá trị
Accuracy	0.7500
Macro Precision	0.7100
Macro Recall	0.6600
Macro F1	0.6792
Weighted F1	0.7400

Table 31: Kết quả đánh giá tổng quát của mô hình BoW kết hợp `LightGBM` trên tập kiểm thử

Bảng 31 cho thấy mô hình đạt Accuracy khoảng 75.00% và Macro F1 bằng 0.6792 trên tập kiểm thử. Kết quả này cho thấy BoW kết hợp `LightGBM` có khả năng phân loại tương đối tốt, tuy nhiên vẫn thấp hơn so với một số mô hình tuyến tính thường phù hợp hơn với đặc trưng văn bản thưa có số chiều lớn. Sự chênh lệch giữa Weighted F1 và Macro F1 cho thấy mô hình vẫn có xu hướng hoạt động tốt hơn trên các lớp có số lượng mẫu lớn.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.65	0.44	0.53	687
BUSINESS	0.66	0.58	0.62	899
COMEDY	0.67	0.47	0.56	810
ENTERTAINMENT	0.72	0.80	0.76	2605
FOOD & DRINK	0.77	0.79	0.78	951
HEALTHY LIVING	0.46	0.28	0.35	1004
HOME & LIVING	0.82	0.71	0.76	648
PARENTING	0.61	0.72	0.66	1319
PARENTS	0.49	0.31	0.38	593
POLITICS	0.85	0.91	0.88	5341
QUEER VOICES	0.85	0.74	0.79	952
SPORTS	0.81	0.73	0.77	761
STYLE & BEAUTY	0.84	0.85	0.85	1472
TRAVEL	0.80	0.80	0.80	1485
WELLNESS	0.66	0.80	0.73	2692

Table 32: Kết quả đánh giá chi tiết theo từng nhãn của mô hình BoW kết hợp LightGBM

Kết quả chi tiết trong Bảng 32 cho thấy mô hình phân loại tốt nhất ở các lớp có đặc trưng từ vựng rõ ràng và số lượng mẫu tương đối lớn như **POLITICS**, **STYLE & BEAUTY**, **TRAVEL**, **QUEER VOICES** và **FOOD & DRINK**. Trong đó, **POLITICS** đạt F1-score 0.88, cao nhất trong toàn bộ các nhãn. Điều này cho thấy biểu diễn BoW vẫn phát huy hiệu quả tốt khi các lớp có hệ từ khóa đặc trưng và ít giao thoa về mặt nội dung.

Ngược lại, các lớp **HEALTHY LIVING**, **PARENTS** và **BLACK VOICES** có F1-score thấp hơn, lần lượt là 0.35, 0.38 và 0.53. Đặc biệt, **HEALTHY LIVING** và **PARENTS** là các lớp dễ giao thoa nội dung với **WELLNESS** và **PARENTING**. Điều này cho thấy LightGBM trên biểu diễn BoW vẫn gặp khó khăn khi các lớp chia sẻ nhiều từ vựng chung nhưng khác nhau về sắc thái ngữ nghĩa.

6.10.4 Các thử nghiệm đã thực hiện

Để lựa chọn cấu hình phù hợp, nhiều tổ hợp tham số của LightGBM đã được thử nghiệm trên tập validation. Các giá trị được thử gồm `n_estimators` thuộc {50, 100, 150, 200}, `learning_rate` thuộc {0.03, 0.05, 0.1} và `num_leaves` thuộc {15, 31, 63}. Tổng cộng có 36 cấu hình được đánh giá. Kết quả cho thấy cấu hình `n_estimators=200`, `learning_rate=0.1` và `num_leaves=63` đạt Macro F1 cao nhất trên tập validation.

<code>n_estimators</code>	<code>learning_rate</code>	<code>num_leaves</code>	Validation Macro F1	Thời gian huấn luyện (s)
200	0.10	63	0.6701	236.69
150	0.10	63	0.6694	154.19
100	0.10	63	0.6609	116.08
200	0.05	63	0.6600	202.75
200	0.10	31	0.6585	120.77
150	0.10	31	0.6542	106.62
150	0.05	63	0.6537	184.50
200	0.03	63	0.6470	208.73
100	0.10	31	0.6433	74.92
200	0.05	31	0.6429	126.96

Table 33: Mười cấu hình tốt nhất của mô hình BoW kết hợp LightGBM trên tập validation

Bảng 33 cho thấy các cấu hình sử dụng `learning_rate=0.1` và `num_leaves=63` thường đạt kết quả tốt hơn. Khi tăng `n_estimators` từ 100 lên 150 và 200, Macro F1 trên tập validation tiếp tục tăng, tuy nhiên thời gian huấn luyện cũng tăng đáng kể. Cấu hình tốt nhất đạt Validation Macro F1 bằng 0.6701 nhưng cần khoảng 236.69 giây để huấn luyện.

Tổng kết lại, mô hình BoW kết hợp LightGBM đạt hiệu quả tương đối tốt với Accuracy khoảng 75.00% và Macro F1 bằng 0.6792 trên tập kiểm thử. Ưu điểm của phương pháp này là có khả năng mô hình hóa quan hệ phi tuyến và tận dụng được đặc trưng BoW unigram–bigram. Tuy nhiên, hạn chế chính nằm ở chi phí huấn luyện cao hơn và hiệu quả chưa vượt trội trên dữ liệu văn bản thưa. Mô hình vẫn gặp khó khăn ở các lớp có nội dung giao thoa như `HEALTHY LIVING`, `PARENTS`, `PARENTING` và `WELLNESS`.

7 Pipeline học sâu

7.1 Tiền xử lý dữ liệu

Dữ liệu văn bản ban đầu có thể chứa giá trị thiếu, URL, thẻ HTML, ký tự đặc biệt và khoảng trắng dư thừa. Vì vậy, trước khi xây dựng mô hình học sâu, cần thực hiện các bước tiền xử lý nhằm chuẩn hóa dữ liệu, giảm nhiễu và chuyển văn bản sang định dạng phù hợp cho mô hình phân loại.

7.1.1 Lựa chọn thuộc tính

Từ bộ dữ liệu ban đầu, nghiên cứu sử dụng ba thuộc tính chính gồm:

- `category`: nhãn phân loại bài báo.

- **headline**: tiêu đề bài báo.
- **short_description**: mô tả ngắn của bài báo.

Hai trường **headline** và **short_description** được kết hợp để tạo thành một trường văn bản mới là **text**.

$$\text{text} = \text{headline} + \text{short_description} \quad (5)$$

Việc kết hợp này giúp văn bản đầu vào chứa nhiều thông tin ngữ nghĩa hơn so với chỉ sử dụng tiêu đề hoặc mô tả riêng lẻ.

7.1.2 Xử lý giá trị thiếu

Trong quá trình xử lý, các giá trị bị thiếu trong hai cột **headline** và **short_description** được thay thế bằng chuỗi rỗng. Bước này giúp tránh lỗi khi ghép văn bản và đảm bảo toàn bộ dữ liệu có thể được xử lý thống nhất.

$$\text{missing value} \rightarrow "" \quad (6)$$

Sau bước này, dữ liệu không còn giá trị thiếu trong các trường văn bản được sử dụng cho mô hình.

7.1.3 Kiểm tra dữ liệu trùng lặp

Sau khi tạo cột **text**, dữ liệu được kiểm tra các trường hợp trùng lặp nhằm đánh giá chất lượng dữ liệu đầu vào. Hai dạng trùng lặp được xem xét gồm:

- Trùng lặp toàn bộ dòng dữ liệu.
- Trùng lặp nội dung văn bản trong cột **text**.

Trong nghiên cứu này, các văn bản trùng lặp trong cột **text** được loại bỏ bằng phương pháp:

$$df = df.drop_duplicates(subset = ["text"]) \quad (7)$$

Việc loại bỏ dữ liệu trùng giúp hạn chế hiện tượng mô hình học lặp lại cùng một mẫu dữ liệu, đồng thời giảm nguy cơ rò rỉ dữ liệu giữa tập huấn luyện và tập kiểm tra.

7.1.4 Lựa chọn các nhãn phổ biến

Bộ dữ liệu gốc gồm nhiều nhóm chủ đề khác nhau và có phân phối không đồng đều giữa các lớp. Để giảm độ phức tạp của bài toán và tập trung vào các lớp có đủ số lượng mẫu, nghiên cứu lựa chọn 15 nhãn xuất hiện nhiều nhất trong tập dữ liệu.

$$\text{TOP_K} = 15 \quad (8)$$

Sau bước này, chỉ các mẫu thuộc 15 category phổ biến nhất được giữ lại để phục vụ quá trình huấn luyện, validation và kiểm tra mô hình. Việc lựa chọn này giúp mô hình học ổn định hơn và hạn chế ảnh hưởng từ các lớp có số lượng mẫu quá nhỏ.

7.1.5 Làm sạch văn bản

Dữ liệu văn bản trong cột `text` được chuẩn hóa nhằm loại bỏ các thành phần không cần thiết và giảm nhiễu trong quá trình học. Các bước làm sạch bao gồm:

- Chuyển toàn bộ văn bản về chữ thường.
- Loại bỏ URL.
- Loại bỏ thẻ HTML.
- Loại bỏ ký tự đặc biệt không cần thiết.
- Giữ lại chữ cái, chữ số, khoảng trắng và dấu nháy đơn.
- Chuẩn hóa khoảng trắng dư thừa.

Sau bước này, văn bản đã làm sạch được lưu vào cột `clean_text`. Khác với các pipeline học máy truyền thống, pipeline học sâu hiện tại không thực hiện loại bỏ stopwords hoặc lemmatization, vì mô hình BiLSTM có khả năng học quan hệ tuần tự và ngữ cảnh giữa các từ trong câu.

7.1.6 Mã hóa nhãn

Các nhãn trong cột `category` ban đầu ở dạng chuỗi, do đó cần được chuyển sang dạng số để đưa vào mô hình học sâu. Pipeline sử dụng `LabelEncoder` để ánh xạ mỗi category thành một chỉ số nguyên.

$$\text{category} \rightarrow \text{label} \quad (9)$$

Sau bước mã hóa, mỗi mẫu dữ liệu có một nhãn số tương ứng. Đồng thời, hệ thống lưu lại ánh xạ giữa nhãn số và tên category để phục vụ quá trình đánh giá và suy luận sau này.

7.1.7 Chia tập dữ liệu

Dữ liệu sau tiền xử lý được chia thành ba tập:

- Tập huấn luyện: 70%.
- Tập validation: 15%.
- Tập kiểm tra: 15%.

Quá trình chia dữ liệu sử dụng phương pháp phân tầng theo nhãn thông qua tham số `stratify`. Cách chia này giúp tỷ lệ các category trong từng tập dữ liệu gần tương đồng với phân phối ban đầu, từ đó làm cho quá trình huấn luyện và đánh giá mô hình ổn định hơn.

7.1.8 Tokenization và padding

Sau khi chia dữ liệu, văn bản trong cột `clean_text` được chuyển thành chuỗi số bằng `Tokenizer` với lượng từ vựng ko giới hạn và OOV với token ngoài vocabulary của Keras. `Tokenizer` chỉ được fit trên tập huấn luyện nhằm tránh rò rỉ thông tin từ tập validation và test.

$$\text{clean_text} \rightarrow \text{sequence of token ids} \quad (10)$$

Các chuỗi sau tokenization có độ dài khác nhau, do đó cần được chuẩn hóa về cùng một độ dài trước khi đưa vào mô hình. Pipeline sử dụng `pad_sequences` với độ dài tối đa là 30 token.

$$\text{MAX_LEN} = 30 \quad (11)$$

Các chuỗi ngắn hơn 30 token được thêm padding ở cuối, trong khi các chuỗi dài hơn 30 token bị cắt phần sau.

$$\text{padding} = \text{"post"}, \text{truncating} = \text{"post"}$$

Cách đặt padding và truncating ở cuối chuỗi phù hợp với dữ liệu tin tức, vì phần đầu văn bản thường chứa nhiều thông tin quan trọng, đặc biệt là tiêu đề bài viết. Bước này giúp toàn bộ dữ liệu đầu vào có cùng kích thước, phù hợp với yêu cầu của mô hình học sâu.

7.1.9 Xử lý mất cân bằng lớp

Mặc dù đã giữ lại 15 category phổ biến nhất, dữ liệu vẫn còn sự chênh lệch giữa các lớp. Vì vậy, pipeline sử dụng `class weight` để giảm ảnh hưởng của mất cân bằng dữ liệu trong quá trình huấn luyện.

Trọng số của mỗi lớp được tính theo công thức:

$$w_c = \frac{N}{K \times n_c} \quad (12)$$

Trong đó, N là tổng số mẫu huấn luyện, K là số lớp và n_c là số mẫu thuộc lớp c . Theo cách tính này, các lớp có ít mẫu hơn sẽ nhận trọng số lớn hơn, giúp mô hình chú ý hơn đến các category có tần suất thấp.

7.2 Xây dựng mô hình học sâu

Sau khi hoàn tất bước tiền xử lý, dữ liệu văn bản đã được chuyển thành các chuỗi số có cùng độ dài và nhãn đã được mã hóa dưới dạng số nguyên. Các đầu vào này được sử dụng để xây dựng mô hình học sâu cho bài toán phân loại chủ đề tin tức.

7.2.1 Đầu vào của mô hình

Đầu vào của mô hình là các chuỗi token sau khi đã được padding về cùng độ dài:

`X_train_pad, X_val_pad, X_test_pad`

Mỗi mẫu văn bản được biểu diễn dưới dạng một vector số có độ dài cố định:

$$\text{MAX_LEN} = 30 \quad (13)$$

Nhãn đầu ra là các giá trị số nguyên tương ứng với 15 category đã được lựa chọn:

$$y \in \{0, 1, 2, \dots, 14\} \quad (14)$$

Như vậy, bài toán được đưa về dạng phân loại đa lớp với 15 nhãn.

7.2.2 Lớp Embedding

Trước khi đưa vào mô hình tuần tự, mỗi token id được ánh xạ thành một vector embedding dày đặc. Lớp Embedding giúp mô hình học biểu diễn ngữ nghĩa của từ dựa trên dữ liệu huấn luyện.

```
Embedding(input_dim=vocab_size, output_dim=128)
```

Trong đó:

- `input_dim`: kích thước từ vựng.
- `output_dim = 128`: số chiều của vector embedding.

Sau lớp này, mỗi văn bản không còn là một chuỗi số nguyên mà trở thành một ma trận embedding, trong đó mỗi dòng là vector biểu diễn của một token.

7.2.3 Spatial Dropout

Sau lớp Embedding, mô hình sử dụng `SpatialDropout1D` với tỷ lệ dropout là 0.3:

```
SpatialDropout1D(0.3)
```

Khác với dropout thông thường, `SpatialDropout1D` loại bỏ toàn bộ một số chiều embedding trên tất cả các bước thời gian. Cách này giúp mô hình tránh phụ thuộc quá mạnh vào một vài chiều đặc trưng cụ thể, từ đó giảm nguy cơ overfitting.

7.2.4 Lớp BiLSTM

Thành phần chính của mô hình là lớp **Bidirectional LSTM**. LSTM có khả năng học quan hệ phụ thuộc theo thứ tự giữa các từ trong câu. Tuy nhiên, LSTM một chiều chỉ đọc chuỗi theo một hướng. Vì vậy, pipeline sử dụng BiLSTM để học ngữ cảnh theo cả hai chiều: từ trái sang phải và từ phải sang trái.

```
Bidirectional(LSTM(128, return_sequences=True))
```

Trong đó:

- 128: số đơn vị LSTM ở mỗi chiều.
- `return_sequences=True`: trả về hidden state tại tất cả các vị trí token.

Việc giữ lại toàn bộ chuỗi hidden state là cần thiết vì tầng Attention phía sau cần đánh giá mức độ quan trọng của từng token trong văn bản.

7.2.5 Cơ chế Masked Attention

Sau BiLSTM, mô hình sử dụng tầng **Masked Attention** để tập trung vào các token quan trọng nhất đối với quyết định phân loại.

Nếu chỉ dùng LSTM, mô hình có thể phụ thuộc nhiều vào hidden state cuối cùng. Trong khi đó, với văn bản tin tức, thông tin quan trọng có thể xuất hiện ở nhiều vị trí khác nhau trong tiêu đề hoặc mô tả ngắn. Attention giải quyết vấn đề này bằng cách gán trọng số cho từng token.

Với mỗi hidden state h_t tại vị trí t , mô hình tính điểm chú ý:

$$e_t = \tanh(Wh_t + b) \quad (15)$$

Sau đó, các điểm này được chuẩn hóa bằng hàm softmax để thu được trọng số chú ý:

$$\alpha_t = \frac{\exp(e_t)}{\sum_j \exp(e_j)} \quad (16)$$

Vector biểu diễn toàn bộ văn bản được tính bằng tổng có trọng số của các hidden state:

$$v = \sum_t \alpha_t h_t \quad (17)$$

Do dữ liệu đã được padding, các vị trí có giá trị token bằng 0 không mang thông tin thật. Vì vậy, tầng **MaskedAttentionLayer** tạo mask để bỏ qua các vị trí padding khi tính attention. Điều này giúp mô hình chỉ tập trung vào các token thật trong văn bản.

7.2.6 Dropout sau Attention

Sau tầng **Masked Attention**, mô hình sử dụng thêm một tầng **Dropout** với tỷ lệ 0.3:

Dropout(0.3)

Tầng này được đặt sau Attention nhằm giảm overfitting trên vector biểu diễn văn bản. Cụ thể, sau khi Attention tổng hợp thông tin từ các hidden state quan trọng của BiLSTM, Dropout sẽ ngẫu nhiên loại bỏ một phần đặc trưng trong vector này trong quá trình huấn luyện. Điều đó giúp mô hình không phụ thuộc quá mức vào một vài đặc trưng nổi bật và cải thiện khả năng tổng quát hóa trên dữ liệu mới.

7.2.7 Các lớp Dense và Dropout

Vector ngữ cảnh sau Attention được đưa qua các lớp fully-connected để học biểu diễn cấp cao hơn cho văn bản:

```
Dense(128, activation="relu")
```

Lớp Dense này giúp mô hình kết hợp các đặc trưng đã học từ BiLSTM và Attention thành một biểu diễn phù hợp hơn cho phân loại. Sau đó, mô hình tiếp tục sử dụng Dropout để giảm overfitting:

```
Dropout(0.3)
```

Ngoài ra, pipeline sử dụng regularization L2 nhằm hạn chế việc trọng số của mô hình tăng quá lớn trong quá trình huấn luyện.

$$\lambda = 10^{-4} \quad (18)$$

7.2.8 Lớp đầu ra

Vì bài toán có 15 category, lớp đầu ra là một lớp Dense với 15 neuron và hàm kích hoạt softmax:

```
Dense(num_classes, activation="softmax")
```

Softmax chuyển đầu ra của mô hình thành phân phối xác suất trên 15 lớp:

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)} \quad (19)$$

Trong đó, $K = 15$ là số category. Category có xác suất cao nhất được chọn làm nhãn dự đoán cuối cùng. Ngoài ra, pipeline có sử dụng regularization L2 nhằm hạn chế trọng số của mô hình tăng quá lớn trong quá trình huấn luyện:

$$\lambda = 10^{-4} \quad (20)$$

7.3 Tổng quan kiến trúc mô hình

Kiến trúc tổng quát của mô hình được trình bày trong Bảng 34.

Table 34: Kiến trúc mô hình BiLSTM kết hợp Attention

Thành phần	Cấu hình	Vai trò
Input	MAX_LEN = 30	Nhận chuỗi token sau padding
Embedding	dim = 128	Học biểu diễn vector cho từng token
SpatialDropout1D	rate = 0.3	Giảm overfitting ở tầng embedding
Bidirectional LSTM	128 units	Học ngữ cảnh hai chiều của văn bản
Masked Attention	Attention có mask padding	Tập trung vào token quan trọng và bỏ qua padding
Dropout	rate = 0.3	Giảm overfitting sau tầng Attention
Dense	128 units, ReLU	Học biểu diễn văn bản cấp cao
Dropout	rate = 0.3	Giảm overfitting trước tầng phân loại
Output	15 units, Softmax	Dự đoán xác suất cho 15 category

7.3.1 Biên dịch mô hình

Mô hình được biên dịch với hàm mất mát `sparse_categorical_crossentropy`. Hàm mất mát này phù hợp với bài toán phân loại đa lớp khi nhãn được biểu diễn dưới dạng số nguyên, không cần one-hot encoding.

```
loss = "sparse_categorical_crossentropy"
```

Bộ tối ưu được sử dụng là Adam với learning rate:

$$\eta = 2 \times 10^{-3} \quad (21)$$

Adam là lựa chọn phù hợp vì có khả năng tự điều chỉnh tốc độ học cho từng tham số, giúp mô hình hội tụ ổn định hơn so với gradient descent cơ bản.

Metric chính được theo dõi trong quá trình huấn luyện là accuracy:

```
metrics = ["accuracy"]
```

7.4 Quá trình huấn luyện mô hình

Sau khi mô hình được biên dịch, quá trình huấn luyện được thực hiện trên tập huấn luyện đã được padding với kích thước đầu vào cố định là 30 token. Tập validation được sử dụng để theo dõi khả năng tổng quát hóa của mô hình trong quá trình học, từ đó giúp phát hiện sớm hiện tượng overfitting.

Các tham số chính trong quá trình huấn luyện gồm:

- Số epoch tối đa: 20.
- Batch size: 128.
- Hàm mất mát: `sparse_categorical_crossentropy`.

- Bộ tối ưu: Adam.
- Learning rate ban đầu: 0.002.
- Metric theo dõi: accuracy.

Quá trình huấn luyện sử dụng tập huấn luyện gồm 103.467 mẫu và tập validation gồm 22.171 mẫu. Do dữ liệu giữa các category vẫn còn mất cân bằng, tham số `class_weight` được đưa vào hàm `fit`. Cách làm này giúp mô hình tăng mức độ quan tâm đến các lớp có số lượng mẫu ít, từ đó hạn chế tình trạng mô hình thiên lệch về các lớp xuất hiện nhiều.

7.4.1 Các callback sử dụng trong huấn luyện

Để kiểm soát quá trình huấn luyện và hạn chế overfitting, pipeline sử dụng một số callback như sau:

- **ModelCheckpoint**: lưu lại mô hình tốt nhất dựa trên giá trị `val_loss`. Mô hình tốt nhất được lưu dưới tên `bilstm_attention_best.keras`.
- **EarlyStopping**: dừng quá trình huấn luyện sớm nếu `val_loss` không cải thiện sau 5 epoch liên tiếp. Đồng thời, trọng số tốt nhất của mô hình được khôi phục lại.
- **ReduceLROnPlateau**: giảm learning rate khi `val_loss` không cải thiện. Cụ thể, learning rate được giảm một nửa sau mỗi epoch không cải thiện, với giá trị nhỏ nhất là 10^{-6} .
- **CSVLogger**: ghi lại lịch sử huấn luyện vào file `training_log.csv` để phục vụ quá trình phân tích kết quả.

Việc sử dụng các callback này giúp quá trình huấn luyện ổn định hơn. Trong đó, **ModelCheckpoint** đảm bảo mô hình tốt nhất không bị mất, **EarlyStopping** giúp tránh huấn luyện quá lâu khi mô hình đã bắt đầu overfitting, còn **ReduceLROnPlateau** hỗ trợ mô hình hội tụ tốt hơn khi giá trị loss không còn giảm rõ rệt.

7.4.2 Diễn biến quá trình huấn luyện

Kết quả huấn luyện cho thấy mô hình cải thiện nhanh trong những epoch đầu tiên. Ở epoch đầu tiên, mô hình đạt độ chính xác trên tập huấn luyện là 0.4009 và độ chính xác trên tập validation là 0.7171. Sang epoch thứ hai, độ chính xác trên tập huấn luyện tăng lên 0.7256, trong khi độ chính xác trên tập validation đạt 0.7278. Đồng thời, giá trị `val_loss` giảm xuống còn 0.9300, đây là giá trị tốt nhất trong toàn bộ quá trình huấn luyện.

Từ epoch thứ ba trở đi, độ chính xác trên tập huấn luyện tiếp tục tăng, tuy nhiên `val_loss` lại có xu hướng tăng dần. Điều này cho thấy mô hình bắt đầu học quá sâu vào dữ liệu huấn luyện, dẫn đến hiện tượng overfitting. Cụ thể, ở epoch thứ bảy, accuracy trên tập huấn luyện đạt 0.9045 nhưng `val_loss` tăng lên 1.1961. Vì vậy, cơ chế **EarlyStopping** đã dừng quá trình huấn luyện tại epoch thứ bảy và khôi phục lại trọng số tốt nhất của mô hình tại epoch thứ hai.

Bảng 35 trình bày tóm tắt kết quả huấn luyện qua các epoch.

Table 35: Kết quả huấn luyện mô hình BiLSTM kết hợp Attention

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.4009	1.9440	0.7171	0.9742
2	0.7256	0.9532	0.7278	0.9300
3	0.7926	0.6858	0.7243	0.9817
4	0.8451	0.4837	0.7153	1.0673
5	0.8789	0.3694	0.7195	1.1322
6	0.8962	0.3124	0.7216	1.1665
7	0.9045	0.2825	0.7233	1.1961

7.4.3 Biểu đồ loss trong quá trình huấn luyện

Hình 29 thể hiện sự thay đổi của hàm mất mát trên tập huấn luyện và tập validation qua từng epoch.

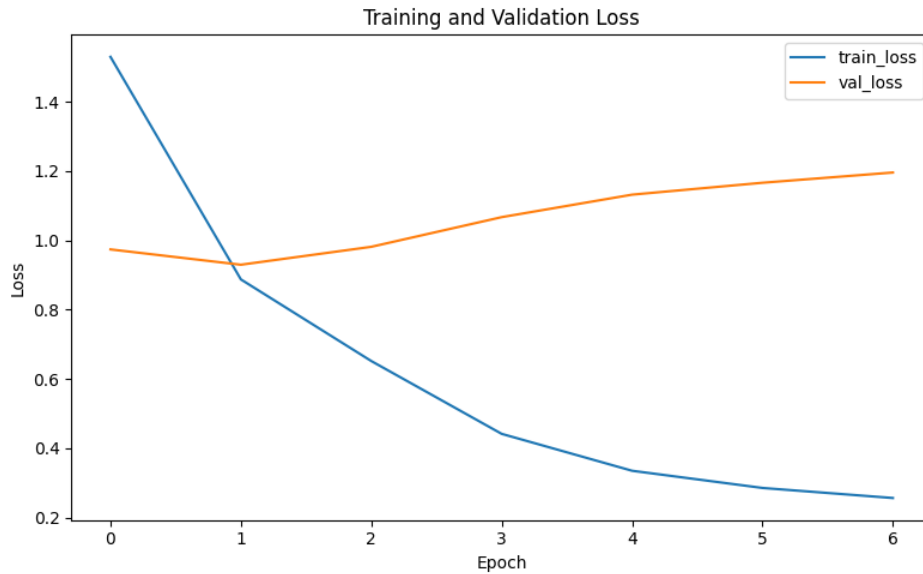


Figure 29: Biểu đồ Training Loss và Validation Loss của mô hình BiLSTM kết hợp Attention

Từ biểu đồ có thể thấy **train_loss** giảm liên tục qua các epoch, cho thấy mô hình học ngày càng tốt trên tập huấn luyện. Tuy nhiên, **val_loss** chỉ giảm đến epoch thứ hai, sau đó tăng dần ở các epoch tiếp theo. Sự khác biệt này là dấu hiệu rõ ràng của overfitting, khi mô hình tiếp tục cải thiện trên tập huấn luyện nhưng không còn cải thiện trên dữ liệu validation.

7.4.4 Biểu đồ accuracy trong quá trình huấn luyện

Hình 30 trình bày sự thay đổi của độ chính xác trên tập huấn luyện và tập validation.

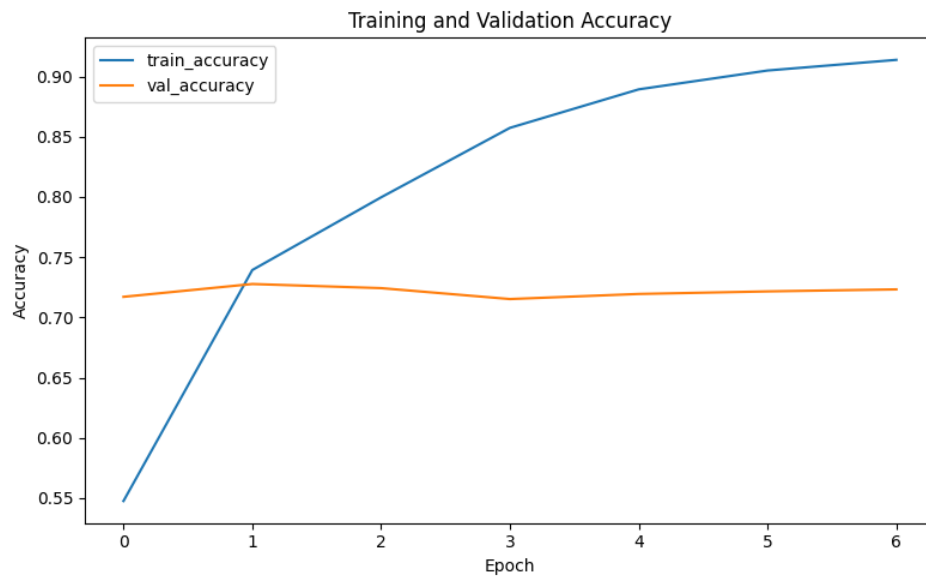


Figure 30: Biểu đồ Training Accuracy và Validation Accuracy của mô hình BiLSTM kết hợp Attention

Kết quả cho thấy accuracy trên tập huấn luyện tăng đều từ 0.4009 lên 0.9045. Trong khi đó, accuracy trên tập validation chỉ dao động quanh mức 0.71–0.73 và không tiếp tục cải thiện sau epoch thứ hai. Điều này củng cố nhận xét rằng mô hình đạt trạng thái tốt nhất ở epoch thứ hai, sau đó bắt đầu có xu hướng ghi nhớ dữ liệu huấn luyện thay vì học các đặc trưng tổng quát.

7.5 Đánh giá mô hình

Tương tự các pipeline trước, mô hình BiLSTM kết hợp Attention được đánh giá trên tập kiểm thử độc lập bằng các chỉ số Accuracy, Macro Precision, Macro Recall, Macro F1 và Weighted F1. Trong đó, Macro F1 tiếp tục được sử dụng để phản ánh khả năng phân loại cân bằng giữa các lớp.

Chỉ số	Giá trị
Accuracy	0.7364
Macro Precision	0.6728
Macro Recall	0.7130
Macro F1	0.6881
Weighted F1	0.7398

Table 36: Kết quả đánh giá tổng quát của mô hình BiLSTM kết hợp Attention trên tập kiểm thử

Bảng 36 cho thấy mô hình đạt Accuracy 73.64%, Weighted F1 73.98% và Macro F1 68.81%. Sự chênh lệch giữa Macro F1 và Weighted F1 cho thấy hiệu quả giữa các lớp chưa đồng đều, tương tự hiện tượng đã quan sát ở các pipeline trước. Đáng chú ý, Macro Recall đạt 71.30%, cao hơn Macro Precision 67.28%, cho thấy mô hình nhận diện được tương đối nhiều mẫu thuộc các lớp khác nhau nhưng vẫn còn một số dự đoán chưa chính xác.

Nhãn	Precision	Recall	F1-score	Support
BLACK VOICES	0.4815	0.6647	0.5585	686
BUSINESS	0.5368	0.6737	0.5975	898
COMEDY	0.4492	0.6238	0.5223	808
ENTERTAINMENT	0.7965	0.6912	0.7401	2604
FOOD & DRINK	0.7828	0.8149	0.7986	951
HEALTHY LIVING	0.5074	0.4074	0.4519	1004
HOME & LIVING	0.6891	0.8274	0.7519	643
PARENTING	0.6686	0.7057	0.6866	1315
PARENTS	0.4877	0.4401	0.4626	584
POLITICS	0.9336	0.7844	0.8525	5338
QUEER VOICES	0.7470	0.7878	0.7669	952
SPORTS	0.6713	0.8265	0.7409	761
STYLE & BEAUTY	0.8181	0.8457	0.8317	1452
TRAVEL	0.7734	0.8640	0.8162	1485
WELLNESS	0.7495	0.7373	0.7433	2691

Table 37: Kết quả đánh giá chi tiết theo từng nhãn của mô hình BiLSTM kết hợp Attention

Bảng 37 cho thấy mô hình đạt kết quả tốt ở các lớp như POLITICS, STYLE & BEAUTY, TRAVEL, FOOD & DRINK, QUEER VOICES và HOME & LIVING. Đây cũng là nhóm nhãn có xu hướng đạt hiệu quả cao ở các pipeline trước do đặc trưng nội dung tương đối rõ ràng.

Lớp POLITICS đạt F1-score cao nhất với 0.8525 và Precision đạt 0.9336, cho thấy các dự đoán thuộc lớp này có độ tin cậy cao. Một số lớp như TRAVEL, STYLE & BEAUTY, SPORTS và HOME & LIVING cũng có Recall cao, cho thấy mô hình nhận diện tốt phần lớn mẫu thật thuộc các lớp này.

Ngược lại, các lớp HEALTHY LIVING, PARENTS, COMEDY và BLACK VOICES có F1-score thấp hơn. Hiện tượng này tương tự các pipeline trên, chủ yếu do các lớp này có nội dung giao thoa, ranh giới chủ đề chưa rõ ràng hoặc số lượng mẫu ít hơn. Đặc biệt, HEALTHY LIVING có Recall chỉ đạt 0.4074, cho thấy nhiều mẫu thật thuộc lớp này vẫn chưa được mô hình nhận diện đúng.

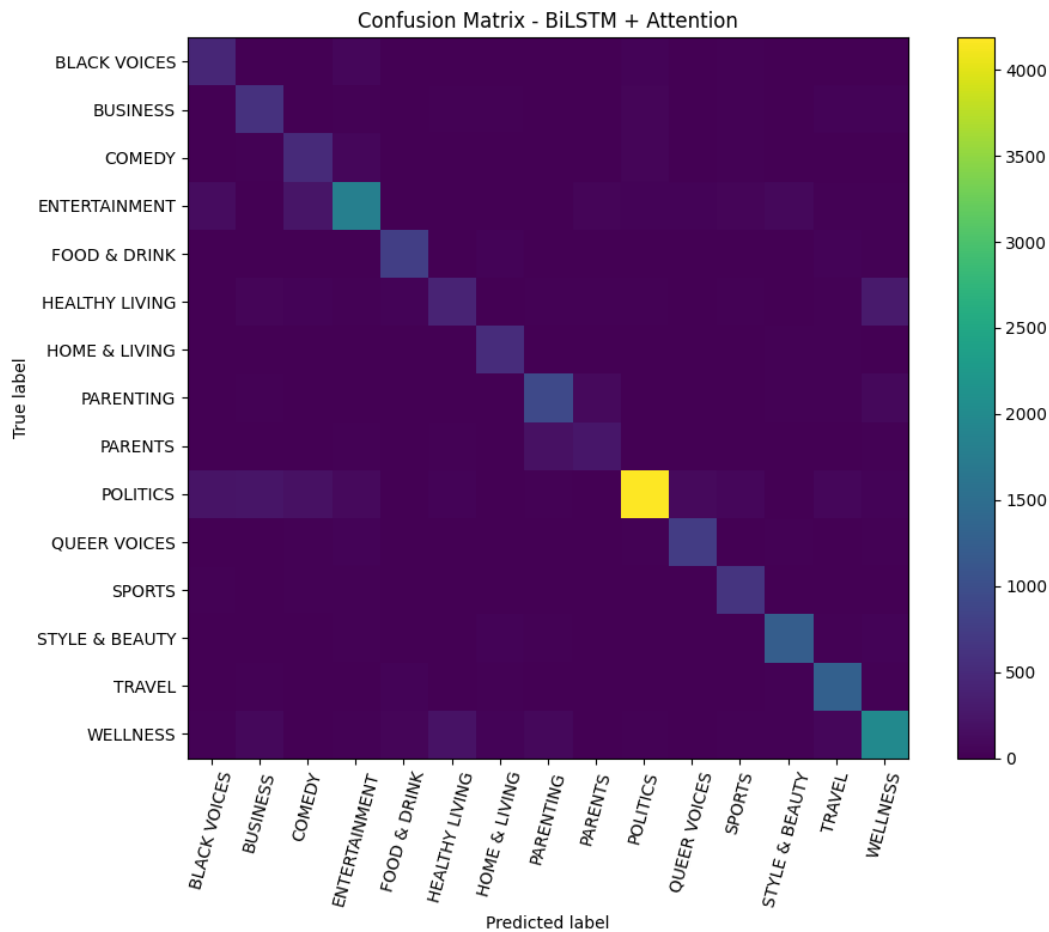


Figure 31: Ma trận nhầm lẫn của mô hình BiLSTM kết hợp Attention trên tập kiểm thử

Ma trận nhầm lẫn ở Hình 31 cho thấy phần lớn dự đoán đúng nằm trên đường chéo chính, đặc biệt ở các lớp có nhiều mẫu hoặc có nội dung rõ như **POLITICS**, **ENTERTAINMENT**, **WELLNESS**, **STYLE & BEAUTY** và **TRAVEL**. Các lỗi nhầm lẫn vẫn tập trung ở những nhóm nhãn gần nghĩa như **HEALTHY LIVING**, **WELLNESS**, **PARENTING** và **PARENTS**, tương tự hiện tượng đã xuất hiện ở các pipeline trước.

8 Phân tích và thảo luận kết quả thực nghiệm

Phần này thảo luận kết quả thực nghiệm của các pipeline phân loại văn bản đã triển khai. Trọng tâm phân tích không chỉ nằm ở mô hình có điểm số cao nhất, mà còn ở việc lý giải sự phù hợp giữa phương pháp biểu diễn văn bản và thuật toán phân loại. Do bài toán có phân phối nhãn không đồng đều, Accuracy được

dùng để đánh giá hiệu quả tổng thể, trong khi Macro F1 được xem là chỉ số quan trọng hơn để phản ánh khả năng phân loại cân bằng giữa các lớp.

8.1 Hiệu quả tổng thể của các pipeline

Bảng 38 tổng hợp kết quả của các pipeline trên cùng tập kiểm thử, với hai chỉ số chính là Accuracy và Macro F1.

Table 38: Tổng hợp kết quả các pipeline phân loại văn bản

Pipeline	Accuracy	Macro F1	Nhận xét chính
RoBERTa Embedding + LinearSVC	0.7686	0.7138	Tốt nhất tổng thể, cải thiện khả năng biểu diễn ngữ nghĩa
BoW + LinearSVC	0.7600	0.6997	Baseline mạnh, khai thác tốt tín hiệu từ khóa
TF-IDF + LinearSVC	0.7613	0.6988	Ổn định trên dữ liệu sparse nhiều chiều
BoW + LightGBM	0.7500	0.6800	Kết quả khá tốt, tốt hơn nhiều mô hình cây khác trên đặc trưng từ vệt
BiLSTM + Attention	0.7364	0.6881	Học được đặc trưng tuần tự nhưng còn hạn chế về độ ổn định
TF-IDF + LightGBM	0.7300	0.6600	Tương đối ổn, nhưng chưa khai thác tốt TF-IDF bằng LinearSVC
SBERT + CatBoost	0.6900	0.6400	Tốt hơn CatBoost với TF-IDF nhờ đặc trưng dense
TF-IDF + Random Forest	0.7006	0.6000	Accuracy khá nhưng chưa cân bằng giữa các lớp
TF-IDF + Gradient Boosting	0.6684	0.5900	Mức so sánh trung bình
BoW + Random Forest	0.6200	0.5691	Hạn chế trên không gian đặc trưng thưa
TF-IDF + CatBoost	0.4684	0.4571	Không phù hợp với TF-IDF chiều cao

Kết quả cho thấy **RoBERTa Embedding + LinearSVC** là pipeline tốt

nhất, đạt Accuracy = 0.7686 và Macro F1 = 0.7138. Đây là mô hình có Macro F1 cao nhất, cho thấy biểu diễn ngữ nghĩa từ Transformer giúp cải thiện khả năng phân biệt các lớp có ranh giới chủ đề chưa rõ ràng.

Hai pipeline truyền thống **BoW + LinearSVC** và **TF-IDF + LinearSVC** cũng đạt kết quả rất cạnh tranh, với Macro F1 xấp xỉ 0.70. Điều này cho thấy dữ liệu tin tức ngắn vẫn chứa nhiều tín hiệu từ vệt đủ mạnh để các mô hình tuyến tính khai thác hiệu quả.

Khi bổ sung LightGBM, có thể thấy **BoW + LightGBM** đạt Accuracy = 0.7500 và Macro F1 = 0.6800, là kết quả khá tốt trong nhóm mô hình cây. **TF-IDF + LightGBM** đạt Accuracy = 0.7300 và Macro F1 = 0.6600, thấp hơn BoW + LightGBM. Điều này cho thấy LightGBM có thể khai thác đặc trưng từ vệt ở mức tương đối tốt, nhưng vẫn chưa phù hợp bằng LinearSVC khi xử lý không gian đặc trưng văn bản thưa và nhiều chiều.

Một nhận xét quan trọng là độ phức tạp của mô hình không tỷ lệ thuận hoàn toàn với hiệu quả. Các mô hình tuyến tính như LinearSVC tuy đơn giản nhưng lại đạt kết quả rất ổn định. Trong khi đó, các mô hình cây như Random Forest, CatBoost, Gradient Boosting và LightGBM có khả năng học quan hệ phi tuyến nhưng hiệu quả phụ thuộc mạnh vào dạng đặc trưng đầu vào.

8.2 Tác động của phương pháp biểu diễn văn bản

Các phương pháp biểu diễn từ vệt gồm BoW và TF-IDF cho kết quả khá cạnh tranh. Với LinearSVC, BoW và TF-IDF đều đạt Macro F1 gần 0.70, cho thấy các tín hiệu từ khóa và cụm từ ngắn có vai trò quan trọng trong bài toán phân loại tin tức.

BoW có ưu điểm là đơn giản và giữ trực tiếp thông tin tần suất từ. TF-IDF bổ sung thêm cơ chế giảm trọng số của các từ quá phổ biến, nhờ đó làm nổi bật các từ có khả năng phân biệt chủ đề. Tuy nhiên, trong thí nghiệm này, sự khác biệt giữa BoW và TF-IDF khi kết hợp với LinearSVC không lớn. Điều này cho thấy nhiều chuyên mục tin tức có hệ từ khóa đặc trưng rõ ràng, nên chỉ riêng tần suất từ cũng đã cung cấp tín hiệu phân loại mạnh.

Đối với LightGBM, BoW cho kết quả tốt hơn TF-IDF. BoW + LightGBM đạt Macro F1 = 0.6800, trong khi TF-IDF + LightGBM đạt Macro F1 = 0.6600. Nguyên nhân có thể là LightGBM, với cơ chế học dựa trên cây quyết định, khai thác trực tiếp sự xuất hiện của từ trong BoW hiệu quả hơn so với trọng số TF-IDF đã được chuẩn hóa. Tuy nhiên, cả hai pipeline LightGBM vẫn chưa vượt qua LinearSVC, cho thấy mô hình tuyến tính vẫn phù hợp hơn với đặc trưng văn bản sparse.

Khác với BoW và TF-IDF, các embedding ngữ nghĩa như RoBERTa và SBERT biểu diễn văn bản bằng vector dense có khả năng mã hóa ngữ cảnh. RoBERTa cho kết quả tốt nhất vì không chỉ dựa vào sự xuất hiện của từ khóa mà còn nắm

bất quan hệ ngữ nghĩa giữa các từ trong câu. Lợi thế này đặc biệt quan trọng với các lớp có nội dung giao thoa như **HEALTHY LIVING**, **WELLNESS**, **PARENTS** và **PARENTING**.

8.3 Tác động của thuật toán phân loại

Xét theo thuật toán, **LinearSVC** là lựa chọn hiệu quả và ổn định nhất trong nghiên cứu. Mô hình này đạt kết quả cao với cả đặc trưng sparse như BoW, TF-IDF và đặc trưng dense như RoBERTa embedding. Điều này phù hợp với đặc thù phân loại văn bản, nơi không gian đặc trưng có số chiều lớn nhưng nhiều lớp có thể được phân tách tốt bằng ranh giới tuyến tính.

Trong nhóm mô hình cây, **LightGBM** cho kết quả tốt hơn so với Random Forest, CatBoost và Gradient Boosting khi sử dụng đặc trưng BoW hoặc TF-IDF. BoW + LightGBM đạt Macro F1 = 0.6800, cho thấy LightGBM có khả năng học tốt hơn các mô hình cây truyền thống trong không gian đặc trưng từ vệt. Tuy nhiên, kết quả này vẫn thấp hơn các pipeline LinearSVC, cho thấy boosting trên cây chưa phải lựa chọn tối ưu nhất cho dữ liệu văn bản sparse.

Random Forest và Gradient Boosting đạt kết quả thấp hơn do khó khai thác hiệu quả không gian đặc trưng thưa, nhiều chiều. CatBoost thể hiện rõ sự phụ thuộc vào dạng đặc trưng đầu vào: khi dùng TF-IDF, mô hình đạt kết quả thấp, nhưng khi dùng SBERT embedding, kết quả được cải thiện đáng kể. Điều này cho thấy các mô hình boosting thường phù hợp hơn với đặc trưng dense hoặc đặc trưng đã được nén thông tin, thay vì ma trận TF-IDF có số chiều lớn.

Nhìn chung, kết quả thực nghiệm cho thấy sự tương thích giữa đặc trưng và thuật toán quan trọng hơn việc chỉ lựa chọn mô hình phức tạp. LinearSVC phù hợp nhất với đặc trưng văn bản sparse, trong khi LightGBM có thể xem là một lựa chọn khá tốt trong nhóm mô hình cây, đặc biệt khi kết hợp với BoW.

8.4 Phân tích theo nhóm nhân

Hiệu quả theo từng nhân cho thấy các mô hình đều phân loại tốt hơn ở các lớp có đặc trưng từ vệt rõ. Các lớp như **POLITICS**, **STYLE & BEAUTY**, **TRAVEL**, **FOOD & DRINK**, **QUEER VOICES** và **SPORTS** thường đạt F1-score cao. Đây là những nhóm chủ đề có nhiều từ khóa đặc trưng, giúp cả mô hình tuyến tính, mô hình cây và mô hình embedding nhận diện tương đối tốt.

Ngược lại, các lớp như **HEALTHY LIVING**, **WELLNESS**, **PARENTS** và **PARENTING** thường có F1-score thấp hơn và dễ bị nhầm lẫn. Nguyên nhân chủ yếu đến từ sự chồng lấn nội dung giữa các chủ đề. Ví dụ, **HEALTHY LIVING** và **WELLNESS** đều liên quan đến sức khỏe, lối sống và chăm sóc cá nhân; **PARENTS** và **PARENTING** cùng xoay quanh gia đình và nuôi dạy con cái. Khi các lớp chia sẻ nhiều từ vệt, các mô hình dựa trên BoW hoặc TF-IDF khó xác định ranh giới rõ ràng.

Hiện tượng Macro F1 thấp hơn Weighted F1 xuất hiện ở nhiều pipeline, cho thấy các mô hình có xu hướng hoạt động tốt hơn trên các lớp lớn hoặc dễ nhận diện. Do đó, nếu chỉ dựa vào Accuracy, đánh giá chất lượng mô hình có thể bị lạc quan quá mức. Macro F1 là chỉ số phù hợp hơn để phản ánh khả năng phân loại công bằng giữa các nhãn.

8.5 So sánh học máy truyền thống và học sâu

Pipeline BiLSTM + Attention đạt Accuracy = 0.7364 và Macro F1 = 0.6881. Kết quả này cho thấy mô hình học sâu có khả năng học đặc trưng tuần tự và ngữ cảnh của văn bản, nhưng chưa vượt qua các mô hình tuyến tính mạnh như LinearSVC.

Một nguyên nhân có thể là dữ liệu văn bản trong bài toán này tương đối ngắn, trong khi nhiều nhãn có thể được nhận diện khá tốt thông qua từ khóa và cụm từ đặc trưng. Ngoài ra, BiLSTM được huấn luyện từ đầu nên cần nhiều dữ liệu và quá trình tinh chỉnh kỹ hơn để đạt hiệu quả cao. Trong khi đó, RoBERTa tận dụng tri thức ngôn ngữ đã được tiền huấn luyện trên tập dữ liệu lớn, nên biểu diễn văn bản tốt hơn ngay cả khi chỉ dùng bộ phân loại tuyến tính phía sau.

Từ kết quả này, có thể thấy học sâu hiệu quả hơn khi được sử dụng theo hướng khai thác mô hình tiền huấn luyện thay vì huấn luyện toàn bộ từ đầu. Pipeline RoBERTa Embedding + LinearSVC là minh chứng rõ ràng cho nhận định này.

8.6 Mức độ phù hợp giữa đặc trưng và thuật toán

Bảng 39 tóm tắt mức độ phù hợp giữa từng nhóm đặc trưng và thuật toán phân loại dựa trên kết quả thực nghiệm.

Table 39: Mức độ phù hợp giữa đặc trưng và thuật toán

Đặc trưng	Thuật toán phù hợp	Nhận xét
BoW	LinearSVC, LightGBM	LinearSVC tốt nhất; LightGBM cũng đạt kết quả khá với đặc trưng từ vựng
TF-IDF	LinearSVC	Ổn định trên dữ liệu sparse nhiều chiều, dễ triển khai
TF-IDF	LightGBM	Có thể sử dụng, nhưng hiệu quả thấp hơn BoW + LightGBM và TF-IDF + LinearSVC
RoBERTa/BERT Embedding	LinearSVC	Tốt nhất tổng thể, cải thiện các lớp có giao thoa ngữ nghĩa
SBERT Embedding	CatBoost	Phù hợp hơn TF-IDF với mô hình cây nhờ đặc trưng dense
BoW/TF-IDF	Random Forest	Hạn chế do đặc trưng thưa và số chiều cao
BoW/TF-IDF	CatBoost/Gradient Boosting	Không tối ưu nếu chưa giảm chiều hoặc chọn lọc đặc trưng
Token sequence	BiLSTM + Attention	Có tiềm năng nhưng cần kiểm soát overfitting tốt hơn

Bảng 39 cho thấy không có thuật toán nào tối ưu trong mọi trường hợp. LinearSVC phù hợp nhất với đặc trưng văn bản sparse và cả embedding dense. LightGBM là mô hình cây có kết quả khá tốt khi kết hợp với BoW, nhưng vẫn chưa đạt mức hiệu quả của LinearSVC. Các mô hình cây nhìn chung cần đặc trưng có mật độ thông tin cao hơn hoặc đã được giảm chiều để hoạt động hiệu quả hơn.

8.7 Kết luận

Kết quả thực nghiệm cho thấy **RoBERTa Embedding + LinearSVC** là pipeline hiệu quả nhất cho bài toán phân loại chuyên mục tin tức. Mô hình này đạt Macro F1 cao nhất và cải thiện khả năng nhận diện các lớp có ranh giới ngữ nghĩa mờ. Tuy nhiên, **BoW + LinearSVC** và **TF-IDF + LinearSVC** vẫn là các baseline rất mạnh, phù hợp khi cần mô hình đơn giản, nhanh và dễ giải thích.

Việc bổ sung LightGBM cho thấy mô hình boosting này có thể đạt kết quả khá tốt với đặc trưng từ vựng, đặc biệt là BoW + LightGBM với Macro F1 = 0.6800. Tuy nhiên, LightGBM vẫn chưa vượt qua LinearSVC trên cùng nhóm đặc trưng, cho thấy đặc trưng sparse của văn bản phù hợp hơn với mô hình tuyến tính.

Nhìn chung, hiệu quả phân loại phụ thuộc chủ yếu vào sự phù hợp giữa phương pháp biểu diễn văn bản và thuật toán phân loại. Với bài toán này, hướng tiếp cận tốt nhất là sử dụng biểu diễn ngữ nghĩa tiền huấn luyện từ Transformer kết hợp với bộ phân loại tuyến tính. Trong trường hợp ưu tiên tốc độ, tính đơn giản và khả năng diễn giải, BoW hoặc TF-IDF kết hợp LinearSVC vẫn là lựa chọn thực tiễn và hiệu quả.

9 Mã nguồn

Toàn bộ mã nguồn của đề tài được lưu trữ tại GitHub:
<https://github.com/HuyHoang1712/news-category-classification>.

Tài liệu tham khảo

- [1] R. Misra, “News Category Dataset,” *arXiv preprint arXiv:2209.11429*, 2022.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [5] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186, 2019.
- [10] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [11] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992, 2019.
- [12] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *International Conference on Learning Representations*, 2015.
- [14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015.